

队伍编号	MC2606711
题号	D

多场景多目标约束下的货物运输三维装箱与车辆配置优化研究

摘 要

本文针对真实物流场景下多车型、多约束、多目标的货物运输装箱优化问题，研究短途货物运输场景下的单车型三维装箱、多车型联合配送以及模型通用性与参数敏感性分析问题。根据题目中货物尺寸、重量、可堆叠性、方向约束、易碎规则及车辆容积、载重和运输成本等条件，建立了“**下层单车几何装箱—上层模式精确覆盖**”的两层优化框架，并设计了相应的求解算法与程序实现流程。

针对问题一，本文首先建立单车三维装箱模型，在满足边界、不重叠、支撑稳定性、定向件投影、易碎件单层放置、局部承重与整车载重等约束条件下，以单车满容率与满载率同时尽可能高为目标，提出“**短板优先—平均提升—失衡抑制**”的分层优化目标。随后，针对固定单车型完成全部货物运输且车辆数最少的问题，将单车可行装载方案抽象为模式，建立模式精确覆盖模型。计算得到：以车型 1 为装载对象时，单车空间利用率为 **83.24%**，载重利用率为 **39.95%**；以车型 2 为装载对象时，单车空间利用率为 **87.22%**，载重利用率为 **53.07%**。在固定单车型条件下，完成附件 1 全部货物运输最少分别需要 **20 辆车型 1** 和 **9 辆车型 2**。

针对问题二，首先，以总运输车辆数最少为目标，构建联合模式精确覆盖模型，并采用“最少车辆数—最低成本—均衡优化”的三级求解结构；其次，以总运输成本最低为目标，构建“最低成本—最少车辆数—均衡优化”的对称分层求解模型。计算结果表明：当前者为目标时，最优方案为采用 **9 辆车型 2**；当以后者为目标时，最优方案为采用 **8 辆车型 2** 和 **1 辆车型 1**，总运输成本为 **6050 元**。

针对问题三，本文在问题一和问题二求解结果基础上，撰写面向物流企业管理者的技术报告，系统总结模型框架、求解思路及其工程应用价值，并通过**参数敏感性实验**分析定向件比例、易碎件比例、货物重量、货物尺寸和货物数量变化对装载率与运输成本的影响。结果表明，**货物尺寸变化对车辆数、运输成本及装载率影响最显著**，易碎件比例次之，定向件比例和货物数量变化影响居中，而货物重量变化主要影响载重利用率，对车辆数和总成本影响相对有限。进一步结合附件 2 数据进行**通用性验证**，结果说明所建模型与算法具有较好的稳定性、迁移性与实际应用价值。

关键词：三维装箱；混合整数线性规划；多目标优化；参数敏感性分析；物流运输决策

目录

一、 问题重述	1
1.1 问题背景	1
1.2 问题分析	1
二、 模型假设	2
三、 符号说明	2
四、 数据预处理	3
五、 问题一模型建立与求解	3
5.1 第一问模型构建	3
5.1.1 符号与变量定义	4
5.1.2 单车协同优化目标	4
5.1.3 姿态约束	5
5.1.4 边界约束与不重叠约束	5
5.1.5 支撑与稳定性约束	5
5.1.6 定向件投影面积约束	6
5.1.7 易碎件与标准件规则约束	6
5.1.8 承重约束与载荷传递	6
5.1.9 整车载重约束	7
5.1.10 固定车型的最少车辆模型	7
5.1.11 固定车辆数下的均衡优化	8
5.2 第一问求解方法	8
5.2.1 求解思路概述	8
5.2.2 单车装箱的构造型求解	8
5.2.3 顶层易碎件优先机制	9
5.2.4 候选动作的评分与多策略择优	9
5.2.5 固定单车型最少车辆的模式化求解	9
5.2.6 固定最优车辆数下的均衡优化	9
5.3 模型求解结果: 针对单个车厢货物摆放问题	10
5.3.1 车型 1 为装载对象	10
5.3.2 车型 2 为装载对象	11
5.4 模型求解结果: 针对单一车型装货最小辆数问题	11
5.4.1 仅有车型 1	11
5.4.2 仅有车型 2	13
5.4.3 仅有车型 1 或 2 完成货物装完且用车最少两方案空间利用率及载重利用率对比	14
六、 问题二模型的建立与求解	14
6.1 模型建立	14
6.1.1 问题二的建模思路	14
6.1.2 下层: 单车装载模式模型	14
6.1.3 上层: 多车型联合覆盖模型	15
6.1.4 目标一: 总车辆数最少模型	15
6.1.5 目标二: 总运输成本最低模型	16

6.2	第二问求解方法	16
6.2.1	求解框架概述	16
6.2.2	模式库构造与裁剪	16
6.2.3	上层基础模型：联合模式覆盖	17
6.2.4	目标一：总车辆数最少的分层求解	17
6.2.5	目标二：总运输成本最低的分层求解	17
6.2.6	均衡优化的评价指标	18
6.2.7	回退机制与稳定性设计	18
6.2.8	本节小结	18
6.3	模型求解结果：总运输车辆最少为目标，装箱策略优化	18
6.4	总运输成本最少为目标，装箱策略优化模型求解结果	19
七、	问题一和问题二模型通用性的验证	20
7.1	问题一模型求解结果：针对单个车厢货物摆放问题（以附件 2 中 4.2m 箱货及 6.8m 箱货为例）	20
7.1.1	车型 1(4.2m 箱货) 为装载对象	20
7.1.2	车型 2(6.8m 箱货) 为装载对象	20
7.2	模型求解结果：针对单一车型装货最小辆数问题	21
7.2.1	仅有车型 1(4.2m 箱货)	21
7.2.2	仅有车型 2(6.8m 箱货)	22
7.3	问题二模型求解结果：总运输车辆最少为目标（以附件 2 中 4.2m 箱货及 6.8m 箱货为例）	23
7.4	问题二模型求解结果：总运输成本最少为目标（以附件 2 中 4.2m 箱货及 6.8m 箱货为例）	23
八、	问题三技术报告	24
8.1	模型与算法介绍	24
8.2	参数敏感性实验设计	25
8.2.1	定向件比例变化	25
8.2.2	易碎件比例变化	26
8.2.3	货物重量变化	26
8.2.4	货物尺寸变化	27
8.2.5	货物数量变化	27
8.3	敏感度排序	27
8.4	管理建议	28
九、	模型的评价	28
9.1	模型的优点	28
9.2	模型不足与改进	28
9.3	模型应用价值	28

一、 问题重述

1.1 问题背景

随着现代物流业与制造业的快速发展，运输装载效率已成为影响企业运营成本、周转效率和绿色低碳水平的重要因素^[1]。短途配送中，货车厢体空间有限，待装货物尺寸不一、重量不均、堆叠与摆放方向受限，部分还存在易碎、不可倒置等要求，使装箱成为兼具几何、重量与操作规则约束的复杂三维组合优化问题。装载方案不佳会造成空间浪费、车辆载效偏低，增加车次与运输成本，影响配送效率并加剧碳排放。物流企业常通过重泡货混装实现车辆满容满载，提升单车收益、摊薄单位成本。因此装箱优化需兼顾空间与载重利用率、用车数量及总成本，在多车型条件下制定合理配载与装箱方案，具有重要现实意义与研究价值。

基于上述背景，本文围绕真实物流运输场景，针对不同车型、不同优化目标和不同运输组织方式下的货物装箱问题展开研究，建立多场景、多目标的三维装箱优化模型，并设计相应求解算法，在满足尺寸、重量、堆叠和方向等约束条件下，分别实现单车装载效率最大化、车辆数量最少化以及运输成本最低化，为物流企业提供具有可操作性的货物运输装箱决策依据。

1.2 问题分析

本题研究对象是短途规则货物运输场景下的装箱与调度优化问题。与普通“体积尽量装满”的装箱问题不同，本题同时受到货物类型差异、堆叠支撑关系、承重上限以及运输成本等多方面因素影响，因此本质上是一个带有稳定性约束的三维装箱—调度联合优化问题。

问题一分析：对于问题一（1），其关键在于“满容率”和“满载率”为一个双目标协同优化问题：一方面尽量提高空间利用率，另一方面尽量提高载重利用率，并保持二者相对均衡。与此同时，由于货物存在易碎件和定向件等特殊规则，单车方案不能简单依赖总体积排序，而必须显式考虑平台构造、堆叠顺序和局部空间适配性。初步看来，单车层面更适合采用一种构造型装箱方法：在保证几何可行与支撑稳定的前提下，逐步选择货物、姿态和放置位置，并通过局部评价函数兼顾空间利用率和载重利用率。对于问题一（2），如果直接在多车层面逐件搜索，不仅搜索空间巨大，而且不利于判断最少车辆数是否已经达到全局最优。因此，初步分析可将该问题分解为两步：先由单车装箱器生成若干高质量的合法单车方案，再将这些方案抽象为“模式”，通过模式组合的方式完成全部需求的精确覆盖。这样一来，问题一（2）就从一个难以直接求解的多车三维装箱问题，转化为一个更适合整数规划处理的固定车型模式覆盖问题。

问题二分析：问题二在问题一的基础上进一步放宽条件，允许两种车型联合参与运输，并分别要求给出“总车辆数最少”和“总运输成本最低”的方案。从目标看，大车型通常在容积和运力上更占优势，但单次运输成本较高；小车型单次成本较低，却可能因为装载能力有限而导致总车数增加。因此，这两个目标应当分开研究，而不宜直接混成一个简单加权目标，否则会掩盖二者之间的决策差异。基于这一考虑，问题二更适合采用分层优化思路：分别以“总车辆数最少”和“总运输成本最低”为主目标，再在主目标固定的前提下考虑另一个指标，并进一步比较两类方案的车型结构和装载差异。

问题三分析：问题三不再是新的数学模型求解，而是对前两问结果的综合提炼、解释与应用延伸。因此，问题三需要从问题一中提炼出单车装载质量、固定车型最少车辆数等信息，从问题二中提炼出多车型联合下的最少车数方案、最低成本方案及其差异，再进一步结合参数敏感性分析，形成能够服务企业管理决策的技术结论。换言之，问题三的核心不是“再求一个最优解”，而是把前两问的建模成果转化为可解释、可沟通、可应用的管理建议。

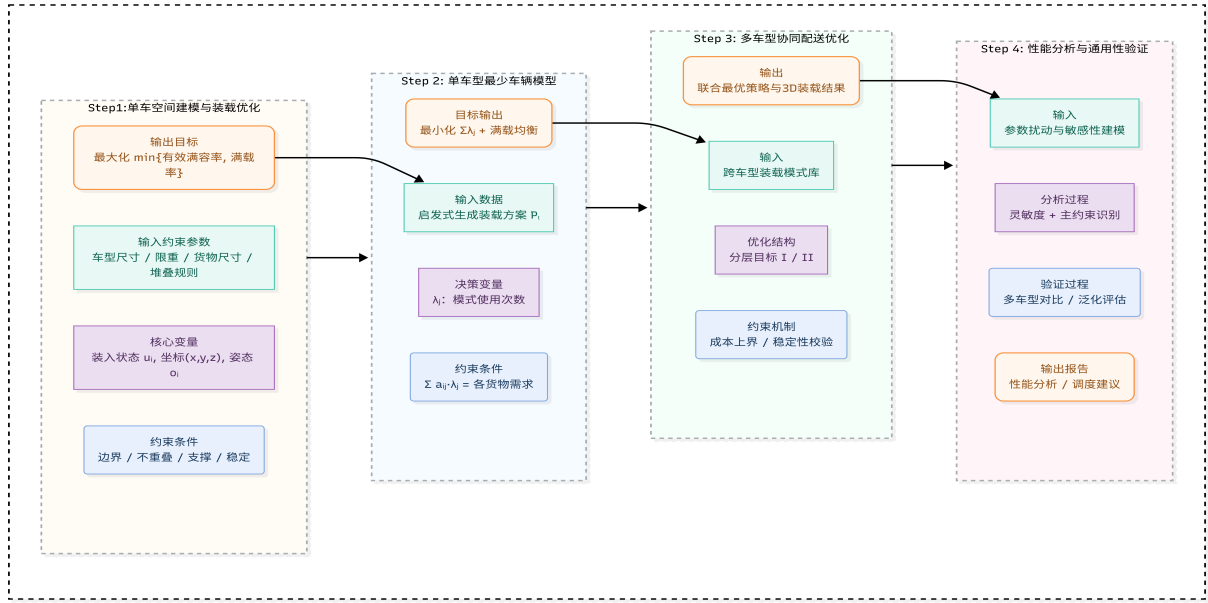


图 1: 多层次三维模型加载与传输优化框架

二、 模型假设

1. 所有货物均抽象为刚性长方体，运输过程中不发生压缩、弯折或形变。
2. 所有货物仅允许正交摆放，即货物棱边与车厢坐标轴保持平行。
3. 单件货物不可拆分，任一货物只能整体装入某一辆车。
4. 若货物不直接落地，则必须由一个或若干个同高支撑面联合承托，不允许悬空。

三、 符号说明

符号	意义	符号	意义
K	车型集合	k	车型索引
L_k, W_k, H_k	车型 k 车厢内部的长、宽、高	H_k^{eff}	车型 k 的有效装载高度
V_k	车型 k 的有效装载体积	R_k	车型 k 的额定载重
c_k	车型 k 的单次运输成本	G	货物类别集合
g	货物类别索引	n_g	第 g 类货物的总件数
v_g	第 g 类货物的单件体积	m_g	第 g 类货物的单件质量
P_k	车型 k 的单车可行模式库	P	联合模式库
a_{gp}	模式 p 中第 g 类货物的装载件数	y_p	模式 p 的采用次数
N	总车辆数	C	总运输成本
ϕ_p	模式 p 的短板利用率	Φ_p	模式 p 的综合评价价值

四、 数据预处理

题目给出两种厢式货车：车型 1 的内部尺寸为 $420 \times 210 \times 220$ cm，额定载重 6000 kg，单次运输成本 450 元；

车型 2 的内部尺寸为 $680 \times 245 \times 250$ cm，额定载重 10000 kg，单次运输成本 700 元。根据题意，车顶需预留 3 cm 安全间隙，因此两种车型的有效高度分别为 217 cm 和 247 cm，对应有效容积为

$$V_1^{\text{eff}} = 420 \times 210 \times 217 = 19.1394 \text{ m}^3, \quad (4.1)$$

$$V_2^{\text{eff}} = 680 \times 245 \times 247 = 41.1502 \text{ m}^3. \quad (4.2)$$

附件中 5 类货物的数量已更新为原始数据的 10 倍，分别为 $G1 = 800$ ， $G2 = 1000$ ， $G3 = 300$ ， $G4 = 400$ ， $G5 = 500$ 。按其尺寸与重量计算，全部货物总体积为

$$V_{\text{all}} = 287.35 \text{ m}^3, \quad (4.3)$$

总重量为

$$W_{\text{all}} = 41100 \text{ kg}. \quad (4.4)$$

由此可得，在不考虑几何损坏和堆叠限制时，固定车型下的理论车辆下界分别为

$$LB_1 = \max \left(\left\lceil \frac{287.35}{19.1394} \right\rceil, \left\lceil \frac{41100}{6000} \right\rceil \right) = 16, \quad (4.5)$$

$$LB_2 = \max \left(\left\lceil \frac{287.35}{41.1502} \right\rceil, \left\lceil \frac{41100}{10000} \right\rceil \right) = 7. \quad (4.6)$$

可以看到，两种车型的体积下界都明显大于重量下界，说明本题首先是一个以空间约束为主导的装箱问题，而不是以载重约束为主导的问题。这一结论直接决定了后续建模中应优先关注空间布局、支撑结构和堆叠方式，而不是单纯追求重量装载。

从运输经济性看，若以单位有效容积成本考量，则车型 1 约为

$$\frac{450}{19.1394} \approx 23.51 \text{ 元/m}^3, \quad (4.7)$$

车型 2 约为

$$\frac{700}{41.1502} \approx 17.01 \text{ 元/m}^3. \quad (4.8)$$

因此，车型 2 在单位容积成本上更具优势，这为问题二中“最低运输成本”目标下可能更多采用车型 2 提供了初步依据；但车型 1 的车身更紧凑，局部适配性更强，“最少车辆数”与“局部填充效率”之间仍可能具有补充作用。

五、 问题一模型建立与求解

5.1 第一问模型构建

第一问本质上是单车三维装箱协同优化问题^[2]，和固定车型下的多车覆盖优化问题。因此，本文采用“下层单车装箱模型—上层单车型覆盖模型”的两层建模思路：先建立满足几何与稳定性要求的单车装箱模型，再在此基础上构造最少车辆模型。

5.1.1 符号与变量定义

设车型集合为 $\mathcal{T} = \{1, 2\}$ 。对任意车型 $t \in \mathcal{T}$ ，其长度、宽度、高度、额定载重分别记为

$$L_t, \quad W_t, \quad H_t, \quad Q_t. \quad (5.1)$$

考虑顶部需预留 3 cm 安全间隙，则车厢有效高度与有效体积分别为

$$H_t^{\text{eff}} = H_t - 3, \quad V_t^{\text{eff}} = L_t W_t H_t^{\text{eff}}. \quad (5.2)$$

设货物类型集合为

$$G = \{G1, G2, G3, G4, G5\}, \quad (5.3)$$

其中每类货物 $g \in G$ 的数量、单件体积、单件重量分别记为

$$n_g, \quad v_g, \quad m_g. \quad (5.4)$$

在单车问题中，将待装货物逐件编号为 $i = 1, 2, \dots, N$ 。对任意货物 i ，定义 0-1 变量

$$u_i = \begin{cases} 1, & \text{若货物 } i \text{ 被装入当前车辆,} \\ 0, & \text{否则.} \end{cases} \quad (5.5)$$

设货物 i 左后下角坐标为

$$(x_i, y_i, z_i). \quad (5.6)$$

实际摆放尺寸为

$$(l_i, w_i, h_i). \quad (5.7)$$

若货物 i 的可行姿态集合为 Ω_i ，则定义姿态变量

$$o_{ir} = \begin{cases} 1, & \text{若货物采用姿态 } r, \quad r \in \Omega_i, \\ 0, & \text{否则.} \end{cases} \quad (5.8)$$

此外，为描述堆叠关系，引入支撑关系变量。若货物 i 直接落地，则记其支撑来源为地板；若由其他货物承托，则其支撑来源可为单一支撑体，也可在特定条件下由若干同高标准件顶面联合构成支撑区域。这样就能统一描述普通支撑与组合支撑两种情形。

5.1.2 单车协同优化目标

定义单车有效满容率为

$$\rho_V = \frac{\sum_{i=1}^N v_i u_i}{V_t^{\text{eff}}}, \quad (5.9)$$

定义单车满载率为

$$\rho_W = \frac{\sum_{i=1}^N m_i u_i}{Q_t}. \quad (5.10)$$

若直接采用线性加权和 $\alpha \rho_V + \beta \rho_W$ ，容易产生某一指标显著偏高而另一指标偏低的失衡方案。为更符合“同时达到”的题意，本文采用“短板优先”的协同目标思想。令

$$\lambda = \min\{\rho_V, \rho_W\}, \quad (5.11)$$

则首先希望最大化 λ ，使较低的一项指标尽可能高。在此基础上，再提高整体装载

水平，并尽量减小二者差异。因此，可将目标写成分层形式：

$$\max \lambda, \quad (5.12)$$

$$\max \frac{\rho_V + \rho_W}{2}, \quad (5.13)$$

$$\min |\rho_V - \rho_W|. \quad (5.14)$$

上述三层目标的含义分别为：优先提升短板，其次提高总体平均水平，最后控制两类利用率之间的失衡程度。

5.1.3 姿态约束

货物的允许摆放姿态由其类别决定^[3]。标准件和易碎件允许正交旋转，而定向件仅允许原始方向。设货物 i 的可行姿态集合为 Ω_i ，则每件被装入车辆的货物必须且只能选择一种姿态：

$$\sum_{r \in \Omega_i} o_{ir} = u_i. \quad (5.15)$$

若 $u_i = 0$ ，则该货物未装入，所有姿态变量均为 0；若 $u_i = 1$ ，则恰有一种姿态被选中。由此，货物的实际尺寸可表示为

$$l_i = \sum_{r \in \Omega_i} l_{ir} o_{ir}, \quad w_i = \sum_{r \in \Omega_i} w_{ir} o_{ir}, \quad h_i = \sum_{r \in \Omega_i} h_{ir} o_{ir}. \quad (5.16)$$

5.1.4 边界约束与不重叠约束

任意被装入货物都必须完全位于车厢内部，因此有

$$0 \leq x_i \leq L_t - l_i \quad (5.17)$$

$$0 \leq y_i \leq W_t - w_i \quad (5.18)$$

$$0 \leq z_i \leq H_t^{\text{eff}} - h_i. \quad (5.19)$$

其中 H_t^{eff} 已经扣除了顶部 3 cm 安全间隙。这样就自动保证了货物顶部不会超过安全高度。此外，任意两件不同货物 i, j 在空间中不得重叠，因此必须满足在 x 、 y 、 z 三个方向中至少有一个方向分离，即

$$(x_i + l_i \leq x_j) \vee (x_j + l_j \leq x_i) \vee (y_i + w_i \leq y_j) \quad (5.20)$$

$$\vee (y_j + w_j \leq y_i) \vee (z_i + h_i \leq z_j) \vee (z_j + h_j \leq z_i). \quad (5.21)$$

需要说明的是，在实际求解中，本文转而采用支撑表面分割机制结构性地保证该约束成立，后续求解过程 4.2 会详细说明。

5.1.5 支撑与稳定性约束

若货物 i 直接落地，则有

$$z_i = 0. \quad (5.22)$$

若货物 i 由某一支撑体 k 顶面承托，则需满足高度方向紧贴关系

$$z_i = z_k + h_k, \quad (5.23)$$

并且其底面矩形完整落在支撑体顶面矩形范围内，即

$$x_k \leq x_i, \quad x_i + l_i \leq x_k + l_k, \quad (5.24)$$

$$y_k \leq y_i, \quad y_i + w_i \leq y_k + w_k. \quad (5.25)$$

进一步地，为防止出现边缘支撑但重心外移的情况，要求货物底面中心投影仍位于支撑体顶面范围内，即

$$\left(x_i + \frac{l_i}{2}, y_i + \frac{w_i}{2}\right) \in [x_k, x_k + l_k] \times [y_k, y_k + w_k]. \quad (5.26)$$

上述约束保证了货物在支撑面上的几何稳定性。

5.1.6 定向件投影面积约束

对定向件，由于其放置方向固定且稳定性要求更高，仅要求“重心”落在支撑面上仍不足以保证稳定安全。因此，本文进一步为定向件引入投影面积约束。设定向件 i 的底面积为

$$A_i^{\text{base}} = l_i w_i, \quad (5.27)$$

其与下方支撑区域的有效重叠面积为 A_i^{ov} ，则要求

$$\frac{A_i^{\text{ov}}}{A_i^{\text{base}}} \geq 1. \quad (5.28)$$

即定向件底面必须被完整支撑，不允许出现悬挑。

5.1.7 易碎件与标准件规则约束

题目附件规定， $G3$ 属于易碎件，只能单层放置，不可承重，并且底面必须贴合地板或标准件顶面。由此可知，若货物 i 属于易碎件，则其支撑来源只能是地板或标准件：

$$\text{Support}(i) \in \{0\} \cup S, \quad (5.29)$$

其中 S 表示标准件集合， 0 表示地板。同时，任何其他货物都不能以易碎件作为支撑体，即

$$i \in F \Rightarrow \nexists j, \text{ s.t. } i \in \text{Support}(j), \quad (5.30)$$

其中 F 表示易碎件集合。标准件则没有上述限制，既可以落地，也可以堆叠，并且可以作为上层货物的平台。

5.1.8 承重约束与载荷传递

附件规定下层货物承重上限为 500 kg/m^2 。由于模型采用厘米为长度单位，因此换算后有

$$500 \text{ kg/m}^2 = 0.05 \text{ kg/cm}^2. \quad (5.31)$$

若某支撑货物 k 的顶面积为 A_k （单位 cm^2 ），则其最大允许承重为

$$C_k = 0.05 A_k. \quad (5.32)$$

设压在货物 k 上方的累计总质量为 Load_k ，则必须满足

$$\text{Load}_k \leq C_k. \quad (5.33)$$

若货物采用组合支撑，则其重量不由单一父节点承担，而应按照底面与各支撑面的重叠面积比例进行分配。设货物 i 由支撑集合 P_i 承托，且与其中支撑体 k 的重叠面积为 A_{ik}^{ov} ，则 k 承受的直接分摊载荷定义为

$$\delta_{ik} = m_i \cdot \frac{A_{ik}^{\text{ov}}}{\sum_{p \in P_i} A_{ip}^{\text{ov}}}. \quad (5.34)$$

该分摊载荷随后沿支撑链逐层向下传递，并在每一层都进行承重检验。这样，承重约束不仅限制直接父支撑体，也限制更底层支撑货物的累计受力。

5.1.9 整车载重约束

除局部承重外，整车总重量也不能超过额定载重，因此有

$$\sum_{i=1}^N m_i u_i \leq Q_t. \quad (5.35)$$

由前面数据处理得到，本题总体上属于空间主导型装箱问题，整车载重约束通常不是最终约束，但它仍然是可行性不可缺少的一部分。

5.1.10 固定车型的最少车辆模型

第一问的第二部分要求：若只允许使用一种车型，则必须把全部货物装完，并使车辆数最少。由于单车内部的三维几何约束过于复杂，本文不直接建立“多辆车一逐件生长”的统一大模型，而是采用模式化建模思路。

设在固定车型 t 下，通过单车装箱模型生成的可行单车模式集合为

$$P_t = \{P_1, P_2, \dots, P_M\}. \quad (5.36)$$

对每个模式 P_j ，定义其装载计数向量为

$$\mathbf{a}_j = (a_{1j}, a_{2j}, a_{3j}, a_{4j}, a_{5j})^T, \quad (5.37)$$

其中 a_{gj} 表示模式 P_j 中装入第 g 类货物的件数。再定义整数变量

$$x_j \in \mathbb{Z}_+, \quad (5.38)$$

表示模式 P_j 在最终运输方案中被采用的次数。则固定车型的最少车辆模型可写为

$$\min \sum_{j=1}^M x_j \quad (5.39)$$

满足

$$\sum_{j=1}^M a_{gj} x_j = n_g, \quad g \in G, \quad (5.40)$$

$$x_j \in \mathbb{Z}_+, \quad j = 1, 2, \dots, M. \quad (5.41)$$

这表明：在固定车型条件下，问题一的第二部分实质上是一个模式精确覆盖问题。每个模式本身代表一辆合法装满的车，因此模式数之和就足所需车辆数。

5.1.11 固定车辆数下的均衡优化

仅仅求得最少车辆数，还可能出现个别车辆利用率很低、整体装载质量不均衡的情况。因此，在固定最优车辆数 N_t^* 后，本文进一步对模式组合做二次均衡优化。定义模式 P_j 的短板利用率为

$$\phi_j = \min \left(\frac{\sum_{g \in G} a_{gj} v_g}{V_t^{\text{eff}}}, \frac{\sum_{g \in G} a_{gj} m_g}{Q_t} \right). \quad (5.42)$$

在满足精确覆盖和车辆总数固定的条件下，进一步提高各模式的 ϕ_j ，并综合考虑模式内货类丰富性、协同装载程度与装载件数，从而得到更均衡的最少车辆方案。

这样，问题一的最终建模结构就完整地分成了两个层次：第一层是单车三维装箱模型，第二层是固定车型的模式精确覆盖与均衡优化模型。

5.2 第一问求解方法

5.2.1 求解思路概述

对于第一小问，本文直接以单车装箱模型为核心，通过多策略择优构造高质量单车方案；对于第二小问，本文先利用单车装箱器生成一批高质量“单车模式”，再通过整数规划在这些模式上完成精确覆盖，以获得最少车辆数，并在固定最优车辆数下进一步提高整体装载均衡性。这样，求解过程自然形成“下层单车装箱—上层模式覆盖”的两层框架。

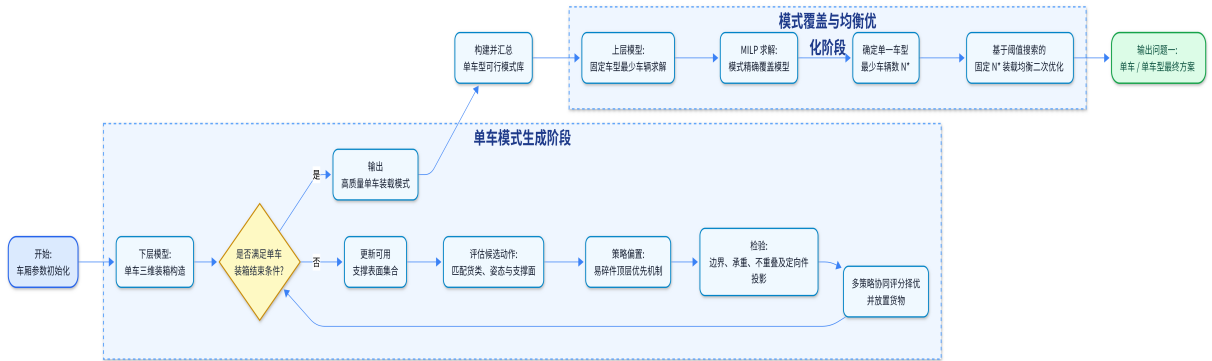


图 2: 问题一技术路线图

5.2.2 单车装箱的构造型求解

由于题目中的货物总量有 3000 件，如果在求解阶段继续逐件展开全部货物，会导致状态规模迅速膨胀。因此，单车求解器不直接维护“3000 个单件对象”，而是维护五类货物的剩余数量向量，并在每一步只决定“下一件应当取哪一类货物、以何种姿态、放到哪个支撑面上”。这样做相当于把原始问题从逐件装箱转化为按货类计数驱动的序贯决策过程，显著降低了搜索维度。

在空间表示上，求解器将车厢底板和所有允许承重货物的顶面统一表示为“可用支撑表面”集合。初始时，每辆车只有一个支撑表面，即车厢底板。之后每放入一件货物，

就将原表面中被占用的区域切分出去，再生成若干新的剩余矩形表面；若该货物允许承重，则把它的顶面加入新的支撑表面集合。通过不断重复“选表面—放货物—切表面—加顶面”这一过程，单车装箱问题被转化为对支撑表面集合的动态更新问题。这样一来，不重叠约束不再需要显式枚举所有货物对，而是由表面切分过程结构性地保证。

5.2.3 顶层易碎件优先机制

在问题一中， $G3$ 属于易碎件，它只能放在地板或标准件顶面上，且一旦放置后，其上方就不能再继续堆叠货物。因此， $G3$ 的放置时机不是普通的“局部放得下就放”，而会直接影响上层空间是否被过早封顶。为此，本文在单车求解中专门设置了顶层易碎件优先机制。

该机制在每一步都会同时评估两类动作：第一类是“当前立即在某个标准件顶面上放置 $G3$ ”；第二类是“暂不放置 $G3$ ，而是继续在当前表面上堆放标准件，以构造更高的平台，之后再放置 $G3$ ”。若当前立即放置 $G3$ 所能达到的最终顶部不低于继续造平台后再放置 $G3$ 所能达到的高度，则直接放置 $G3$ ；否则优先继续堆放标准件，为易碎件预留更优的平台。这样做的本质，是在局部决策中引入了前瞻性优化，避免候选过程中放置易碎件而导致车厢上部空间浪费。

5.2.4 候选动作的评分与多策略择优

单车求解的关键在于：在任一步中，往往会同时存在多个可放置货类、多个可用表面和多个可行姿态，因此必须构造合理的候选评价函数，决定当前最优动作。本文采用的主评分思想与单车目标函数保持一致：优先提高满容率与满载率中的较低者，其次提高二者平均水平，再抑制失衡程度，并适当考虑柔积项。这样，每个候选动作都不只是看“当前装进去多少体积”，而是在它对单车整体双目标协同效果的贡献。

除主评分外，求解器还叠加了若干与货类结构有关的附加偏置。例如：对标准件，会适度奖励其作为后续平台构造者的作用；对定向件，在仍有易碎件待装时，会适度惩罚其过早占用标准件顶面；对易碎件，则额外考虑其最终所处的顶层高度和未来平台损失。这些附加项并不改变问题的主目标，而是通过启发式方式引导搜索向更贴近题目物理逻辑的方向推进。

为了避免单一策略导致局部偏差，单车问题并非只运行一次构造算法，而是采用多种策略分别装箱，然后择优保留。最终程序中使用了若干不同的单车策略，包括偏平衡的策略、偏体积利用率的策略、偏载重利用率的策略以及偏协同柔积的策略。对每种策略各运行一次完整单车装箱过程，再按“短板利用率—平均利用率—倾斜性—协同柔积”的次序对结果排序，最终选取最优方案作为问题（1）的输出。这种“多策略并行试探 + 结果择优”的方法，能够有效降低单一启发式排序带来的偶然性。

5.2.5 固定单车型最少车辆的模式化求解

下层单车装箱器生成的模式库并不是随机完成的，而是先用基础贪心整车方案作为种子，确保模式库至少包含一个可靠可行方案；然后再对若干货类子集和数量截断水平生成“局部需求样本”，在多种模式策略下调用单车装箱器，得到更多高质量且结构不同的单车模式；若多个模式对应相同的装载计数，则只保留评分更优的模式。这样得到的模式库既具有多样性，又能控制规模。

5.2.6 固定最优车辆数下的均衡优化

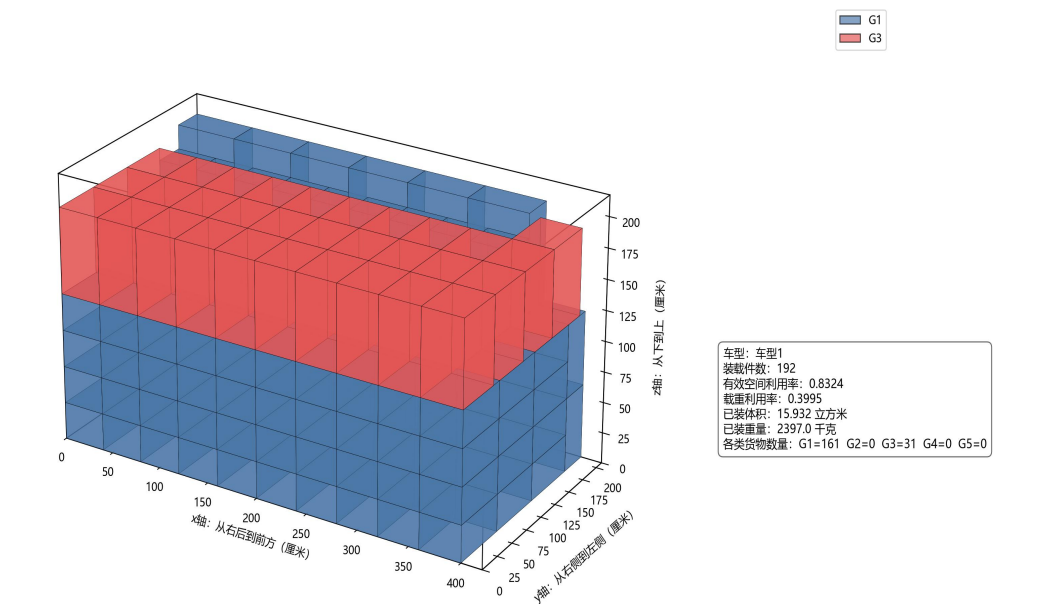
在固定最优车辆数的条件下，先通过阈值搜索找出“仍能实现精确覆盖的最高短板阈值”，以筛掉明显装载质量较差的模式；接着，在保留下来的模式子集上，以综合评分

最大为目标重新求解精确覆盖模型。综合评分不仅考虑模式的短板利用率，还考察体积—重量协同乘积、货类混装程度以及装入件数，从而使最终车辆数虽不变，但各车装载质量更加均衡。

5.3 模型求解结果: 针对单个车厢货物摆放问题

5.3.1 车型 1 为装载对象

以车型 1 为装载对象，目标为单车同时达到满容率与满载率最大化，具体装箱方案示意图如下：



该图中部分货物坐标展示如下：

表 1: 车型 1 部分货物坐标展示

货物编号	X 坐标	Y 坐标	Z 坐标	长 (cm)	宽 (cm)	高 (cm)
G1	0	60	60	40	60	30
G1	0	60	90	40	60	30
G3	0	50	120	40	50	70
G1	40	0	0	40	60	30

而附件中货物长宽高如下：

表 2: 附件一货物数据展示

货物编号	类型	长 (cm)	宽 (cm)	高 (cm)	单重 (kg)	数量
G1	标准件	60	40	30	12	800
G2	标准件	50	35	25	8	1000
G3	易碎件	70	50	40	15	300
G4	定向件	80	60	50	25	400
G5	定向件	40	40	60	18	500

可以看出坐标表内记录货物的长宽高与附件中货物长宽高有区别，可以看出货物是以什么样的姿态摆放的。

以车型 1 为装载对象，以上装箱方案经过算法计算其空间利用率为 83.24%、载重利用率 39.95%

5.3.2 车型 2 为装载对象

以车型 2 为装载对象，目标为单车同时达到满容率与满载率最大化，具体装箱方案示意图如下：

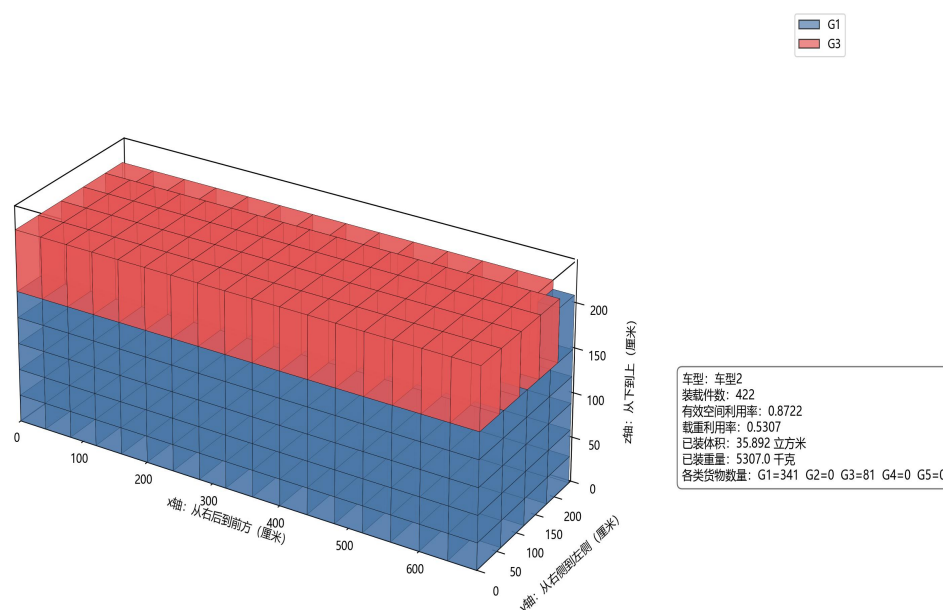


图 4: 车型 2 货物 3D 摆放图

该图中部分货物坐标展示如下：

表 3: 车型 2 部分货物坐标展示

货物编号	X 坐标	Y 坐标	Z 坐标	长 (cm)	宽 (cm)	高 (cm)
G1	0	60	0	40	60	30
G3	0	50	150	40	50	70
G1	0	120	60	40	60	30
G1	40	120	90	40	60	30

同样的，可以看出坐标表内记录货物的长宽高与附件中货物长宽高有区别，可以看出货物是以什么样的姿态摆放的。

以车型 2 为装载对象，以上装箱方案经过算法计算其空间利用率为 87.22%、载重利用率 53.07%

5.4 模型求解结果: 针对单一车型装货最小辆数问题

5.4.1 仅有车型 1

把附件 1 中的货物装完且所用车辆最少情况下，需要 20 辆车型 1，每一辆车具体装箱方案如下图：

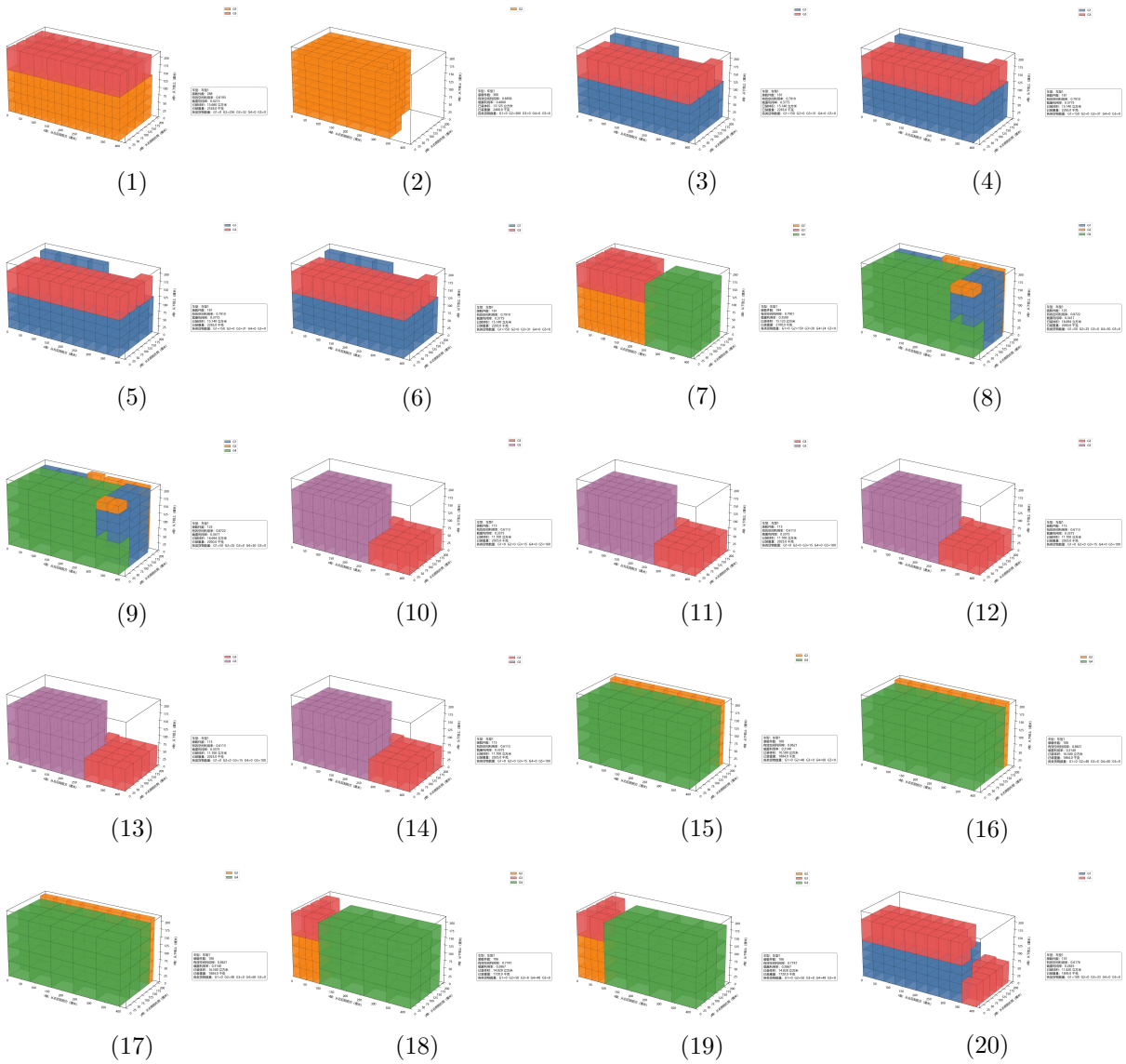


图 5: 最少车辆方案下各车辆装载结果总览图 (仅有车型 1)

逐一计算出每辆车货物摆放坐标及长宽高数据，展示部分数据如下表：

表 4: 仅有车型 1 部分货物坐标展示

货物编号	X 坐标	Y 坐标	Z 坐标	长 (cm)	宽 (cm)	高 (cm)
G2	0	0	105	25	50	35
G2	25	0	105	25	50	35
G8	0	0	140	50	40	70
G2	25	50	70	25	50	35

同样可以看出坐标表内记录货物的长宽高与附件中货物长宽高有区别，可以看出货物是以什么样的姿态摆放的。

以上就是目标只有车型 1 把附件 1 中的货物装完且所用车辆最少的方案，采用 20 辆车型 1，再计算这种方案下每一辆车型 1 的空间利用率、载重利用率如下表：

表 5: 仅有车型 1 空间利用率及载重利用率

车辆编号	空间利用率 (%)	载重利用率 (%)	车辆编号	空间利用率 (%)	载重利用率 (%)
1	81.93	42.13	11	74.57	31.50
2	77.54	41.02	12	74.57	31.50
3	82.09	40.83	13	74.57	31.50
4	81.62	39.22	14	74.57	31.50
5	78.40	38.13	15	74.57	31.50
6	87.22	34.17	16	74.57	31.50
7	61.13	33.75	17	74.57	31.50
8	61.13	33.75	18	74.57	31.50
9	84.16	33.50	19	70.81	31.50
10	67.94	33.45	20	70.81	31.50

可以计算这种方案下 20 辆车型 1 中，空间利用率最大值是 87.22%，空间利用率最小值是 61.13%，空间利用率平均值是 75.07%。载重利用率最大值是 42.13%，载重利用率最小值是 31.50%，载重利用率平均值是 34.25%。

5.4.2 仅有车型 2

把附件 1 中的货物装完且所用车辆最少情况下，需要 9 辆车型 2，每一辆车具体装箱方案如下图：

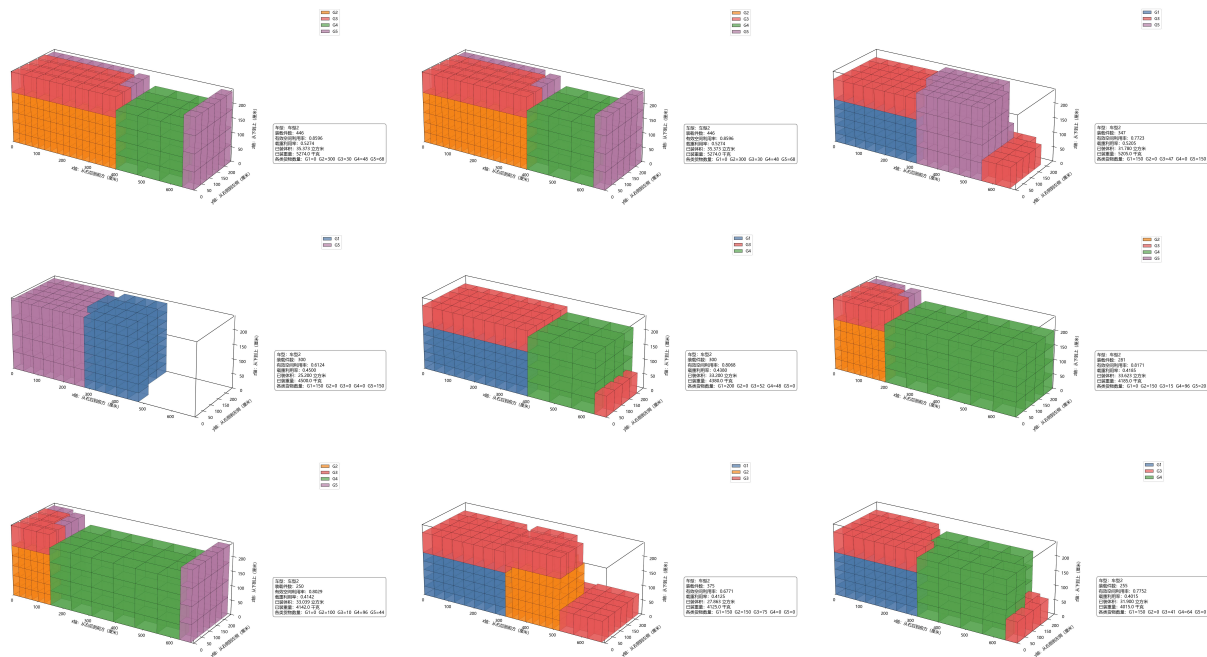


图 6: 最少车辆方案下各车辆装载结果总览图 (仅有车型 2)

逐一计算出其货物摆放坐标及长宽高数据，展示部分数据如下表：

表 6: 仅有车型 2 部分货物坐标展示

货物编号	X 坐标	Y 坐标	Z 坐标	长 (cm)	宽 (cm)	高 (cm)
G2	0	0	0	25	50	35
G2	25	0	0	25	50	35
G3	0	0	175	50	40	35
G2	25	50	35	25	50	35

同样可以看出坐标表内记录货物的长宽高与附件中货物长宽高有区别，可以看出货物是以什么样的姿态摆放的。

以上就是目标只有车型 2 把附件 1 中的货物装完且所用车辆最少的方案，采用 9 辆车型 2，再计算这种方案下每一辆车型 2 的空间利用率、载重利用率如下表：

表 7: 仅有车型 2 空间利用率及载重利用率

车辆编号	空间利用率 (%)	载重利用率 (%)	车辆编号	空间利用率 (%)	载重利用率 (%)
1	85.96	42.13	6	81.71	31.50
2	85.96	41.02	7	80.29	31.50
3	77.23	40.83	8	67.71	31.50
4	61.24	39.22	9	77.52	31.50
5	80.68	38.13	无	无	无

可以计算这种方案下 9 辆车型 2 中，空间利用率最大值是 85.96%，空间利用率最小值是 61.24%，空间利用率平均值是 77.59%。载重利用率最大值是 42.13%，载重利用率最小值是 31.50%，载重利用率平均值是 36.37%。

5.4.3 仅有车型 1 或 2 完成货物装完且用车最少两方案空间利用率及载重利用率对比

表 8: 仅有车型 1 或 2 条件下空间利用率及载重利用率对比

车型	空间利用率 最大值 (%)	空间利用率 最小值 (%)	空间利用率 平均值 (%)	载重利用率 最大值 (%)	载重利用率 最小值 (%)	载重利用率 平均值 (%)
1(20 辆)	87.22%	61.13%	75.07%	42.13%	31.50%	34.25%
2(9 辆)	85.96%	61.24%	77.59%	42.13%	31.50%	36.37%

六、 问题二模型的建立与求解

6.1 模型建立

6.1.1 问题二的建模思路

总体上与问题一一样，均采用“下层单车装箱 + 上层模式覆盖”的分层思想。

6.1.2 下层：单车装载模式模型

第二问的下层模型与第一问的单车模型保持一致，故不再赘述。

与第一问不同的是，每个模式还对应所属车型 $t(j)$ 、单车成本 $c_j = C_{t(j)}$ ，以及该模式自身的满容率和满载率。

6.1.3 上层：多车型联合覆盖模型

在上层，将车型 1 与车型 2 分别生成的单车模式合并，构成联合模式库

$$\mathcal{P} = \{P_1, P_2, \dots, P_M\}. \quad (6.1)$$

对每个模式 $P_j \in \mathcal{P}$ ，定义整数变量

$$x_j \in \mathbb{Z}_+, \quad (6.2)$$

表示该模式在最终运输方案中被采用的次数。这样，问题二就被转化为：如何在联合模式库中选取若干模式及其使用次数，使得五类货物总需求被恰好覆盖，同时满足给定的优化目标。其基本覆盖约束写为

$$\sum_{j=1}^M a_{gj} x_j = n_g, \quad g \in G. \quad (6.3)$$

由于每个模式本身代表一辆已经合法装满的车，因此变量 x_j 的和就直接代表总车辆数：

$$N = \sum_{j=1}^M x_j. \quad (6.4)$$

类似地，总运输成本可以表示为

$$Z = \sum_{j=1}^M c_j x_j. \quad (6.5)$$

因此，问题二的求解从几何装箱本身转化为一个带有精确覆盖约束的多目标整数规划问题。

6.1.4 目标一：总车辆数最少模型

在目标一中，题目要求在两种车型联合使用的情况下，使总车辆数最少。因此主目标函数写为

$$\min N = \min \sum_{j=1}^M x_j. \quad (6.6)$$

同时满足前述精确覆盖约束

$$\sum_{j=1}^M a_{gj} x_j = n_g, \quad g \in G, \quad (6.7)$$

以及非负整数约束

$$x_j \in \mathbb{Z}_+, \quad j = 1, 2, \dots, M. \quad (6.8)$$

该模型求得的是两种车型联合条件下的最少车辆总数。需要注意的是，在最少车辆数相同的前提下，仍可能存在多种不同的车型组合与模式组合。

6.1.5 目标二：总运输成本最低模型

在目标二中，题目要求在全局货物运输完成的前提下，使总运输成本最低。因此主目标函数改写为

$$\min Z = \min \sum_{j=1}^M c_j x_j. \quad (6.9)$$

满足同样的精确覆盖约束

$$\sum_{j=1}^M a_{gj} x_j = n_g, \quad g \in G, \quad (6.10)$$

以及

$$x_j \in \mathbb{Z}_+, \quad j = 1, 2, \dots, M. \quad (6.11)$$

6.2 第二问求解方法

6.2.1 求解框架概述

同样采用问题一两层求解框架求解，第二问的下层仍然采用第一问已经建立的单车三维装箱模型，不再赘述。

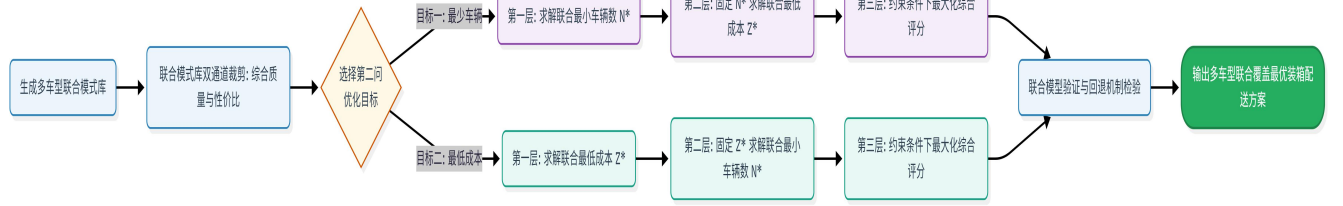


图 7: 问题二技术路线图

6.2.2 模式库构造与裁剪

通过“货类子集 + 数量截断”的方式构造若干局部需求子问题，再对这些子问题运行多种装箱策略，得到结构多样的单车模式。

最终求解中，问题二采用的货类子集不仅包含单货类、双货类和三货类组合，还特别加入了若干针对标准件、定向件和易碎件混装情形的组合；数量截断网格则取 (None, 30, 80, 150, 300)，以便在不同规模上采样高质量模式。

其次，对同一车型中“装载计数完全相同”的模式进行去重，只保留评分更优者。设模式 P 的有效空间利用率、载重利用率、装载货类数、装入件数和单次运输成本分别为 $v(P), w(P), m(P), n(P), c(P)$ ，定义模式评分为

$$S(P) = \left(\min\{v(P), w(P)\}, \frac{v(P) + w(P)}{2}, v(P)w(P), v(P), m(P), n(P), -c(P) \right). \quad (6.12)$$

比较时采用字典序规则，优先提高短板利用率，其次提高平均利用率与协同利用水平，并兼顾混装丰富度、装入件数及运输成本。由此可在重复模式中保留装载质量更高的代表列。

再次，对单车型模式库和联合模式库分别进行“双通道裁剪”：一方面按主评分保留一批装载质量较高的模式；另一方面按单位成本效率再保留一批“性价比”较高的模

式。

最终求解过程中，单车型模式库的主评分保留上限与成本评分保留上限分别为 90 和 60，联合模式库对应上限分别为 150 和 100。这样的裁剪策略，使模式库既保留了高质量模式，又不会因为规模过大而削弱上层 MILP 的可解性。

6.2.3 上层基础模型：联合模式覆盖

上层求解统一基于 6.1.3 的联合覆盖模型展开。

6.2.4 目标一：总车辆数最少的分层求解

当主目标是“总车辆数最少”时，求解过程采用三层结构。第一层，直接在联合模式库上求解最少车辆数模型：

$$\min N = \sum_{j \in \mathcal{P}} x_j. \quad (6.13)$$

满足全部精确覆盖约束。若这一阶段 MILP 求解失败，则退回到多车型贪心回退方案，保证至少能输出一个可行解。第二层，在最优车辆总数 N^* 已确定后，继续最小化总运输成本，即求解

$$\min Z = \sum_{j \in \mathcal{P}} c_j x_j. \quad (6.14)$$

并额外加入约束

$$\sum_{j \in \mathcal{P}} x_j = N^*. \quad (6.15)$$

这样得到的是“在最少车辆数条件下，总成本尽量小”的方案。

第三层，在固定车辆数和总成本的条件下，再进行均衡优化。其核心思路是：先计算每个模式的短板利用率

$$\phi_j = \min\{\rho_V(P_j), \rho_W(P_j)\}, \quad (6.16)$$

然后通过二分搜索求出在当前车辆数与成本约束下仍可行的最高短板阈值 τ^* 。在满足

$$\phi_j \geq \tau^* \quad (6.17)$$

的模式子库上，再最大化综合质量评分，从而提升方案中最差车辆的装载质量与整体均衡性。这一阶段的意义在于：在不牺牲最少车辆数和相应最低成本的前提下，使车辆之间的装载利用程度尽可能协调。

6.2.5 目标二：总运输成本最低的分层求解

当主目标改为“总运输成本最低”时，求解结构与目标一对称，但实现上更强调成本上界控制与最优性验证。第一层，在联合模式库上直接求解最低成本模型：

$$\min Z = \sum_{j \in \mathcal{P}} c_j x_j \quad (6.18)$$

满足全部精确覆盖约束。与目标一同，这一阶段程序还额外利用一个已知可行上界作为参考，若“总车辆最少方案”求解的总成本已经低于当前成本上界，则将其作为成本上界，以避免因为求解时间限制而把明显更差的方案误判为最低成本方案。由于成本是离散的，且车型成本分别为 450 元和 700 元，因此总成本只会一步步上涨。为进一步验证当前最好成本是否还能降低，最终求解器并不仅依赖一次成本最小化 MILP，而是先快速检查在“再便宜一个成本步长”是否存在可行方案；若存在，则利用二分式搜索

继续压缩成本上界；若不存在，则接受当前成本上界作为最优值。这样做的作用，是在 MILP 可能因为时间限制未能给出严格最优证明时，仍尽量避免把偏高的成本误判为全局最低成本。

第二层，在最小总成本 Z^* 已确定后，再最小化总车辆数：

$$\min N = \sum_{j \in P} x_j \quad (6.19)$$

并加入约束

$$\sum_{j \in P} c_j x_j = Z^*. \quad (6.20)$$

这样得到的是“在最低成本条件下，车辆数量尽量少”的方案。

第三层，同样在固定成本与车辆数条件下进行均衡优化，方法与目标一完全一致，即先提升最差模式的短板利用率阈值，再在满足该阈值的模式集上最大化工综合质量评分。这样，目标二最终得到的不是单纯“最便宜”的方案，而是“成本最低且车辆尽量少、装载质量尽量均衡”的方案。

6.2.6 均衡优化的评价指标

第二问的三层均衡优化中，综合质量评价并非单一追求某项利用率最大，而是兼顾短板利用率、体载协同度、货类混装丰富性与单车装载件数。若仅优化单一指标，易出现车辆装载优劣悬殊的不均衡问题，而论文展示与实际配送均更看重方案的整体稳定性。均衡优化阶段先以短板利用率 ϕ_j 为阈值筛选方案，再通过综合评分函数对保留模式排序。最终输出最小与平均空间利用率、最小与平均载重利用率、最小与平均短板利用率等统计量，作为第二问评价装载方案优劣的主要依据。

6.2.7 回退机制与稳定性设计

由于问题二的上层 MILP 规模较大，且同时涉及模式库质量和时间限制控制，因此求解中必须设计必要的回退机制，以保证无论何种情况下都能输出可行方案。

当“最少车辆数”第一阶段求解失败时，回退到多车型贪心方案；当“最低成本”阶段未能得到比当前可行上界更低的成本时，直接返回已知更优行上界；当均衡优化阶段未能在筛选后的模式子库中找到可行解时，则退回到模式库再次求解；若仍失败，则保留前一阶段方案。通过这种层层回退设计，整个求解过程始终保持“先追求更优，失败则回退到已知可行解”的稳健结构。

6.2.8 本节小结

综上，第二问的求解并不是简单地在两种车型之间做经验性选择，而是一个“下层模式生成—上层精确覆盖—分层多目标优化—必要时回退修正”的系统过程。对“总车辆数最少”目标，采用“最少车数—最低成本—均衡优化”的三级求解结构；对“总运输成本最低”目标，采用“最低成本—最少车数—均衡优化”的对称三级结构。通过这种方法，第二问既能保持单车装箱方案的几何真实性，又能在全局层面严谨地比较不同车型组合的经济性与效率。

6.3 模型求解结果：总运输车辆最少为目标，装箱策略优化

本文求解出，在同时可选用车型 1、车型 2 的条件下，为了把附件 1 中的货物运完，以总运输车辆最少为目标，装箱策略是采用 9 辆车型 2，每辆车型 2 货物摆放示意图如下：

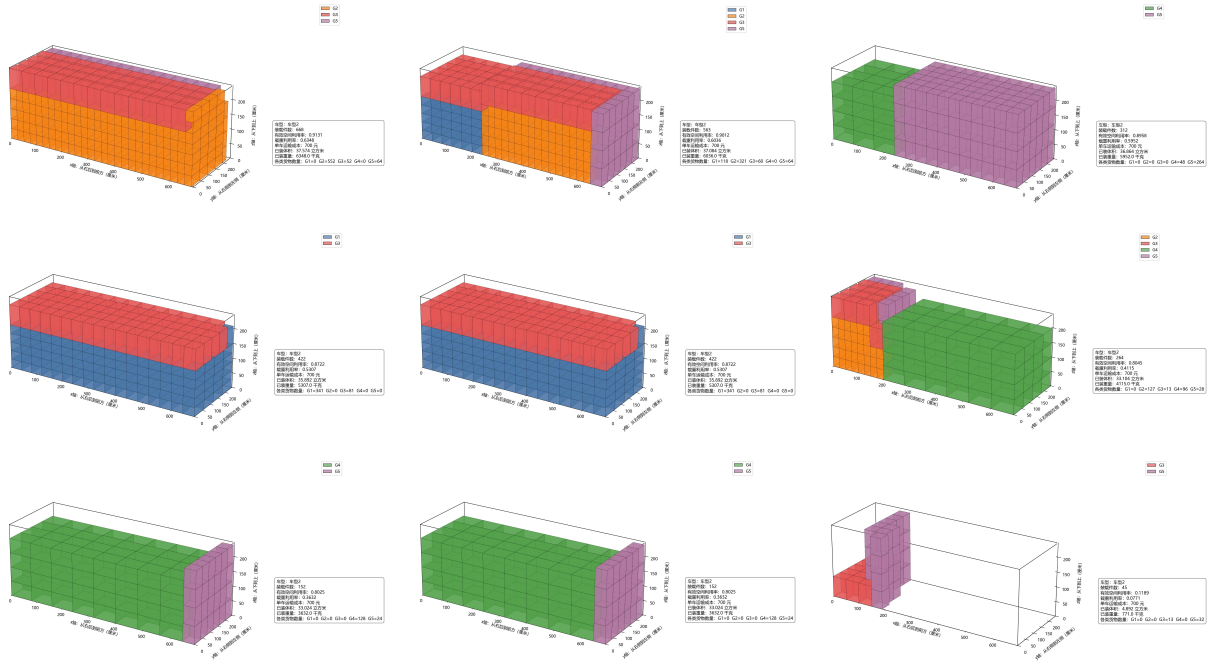


图 8: 总运输车辆最少条件下货物摆放图

6.4 总运输成本最少为目标，装箱策略优化模型求解结果

本文求解出，在同时可选车型 1、车型 2 的条件下，为了把附件 1 中的货物运完，以总运输成本最少为目标，装箱策略是采用 8 辆车型 2 和 1 辆车型 1，每辆车货物摆放示意图如下：

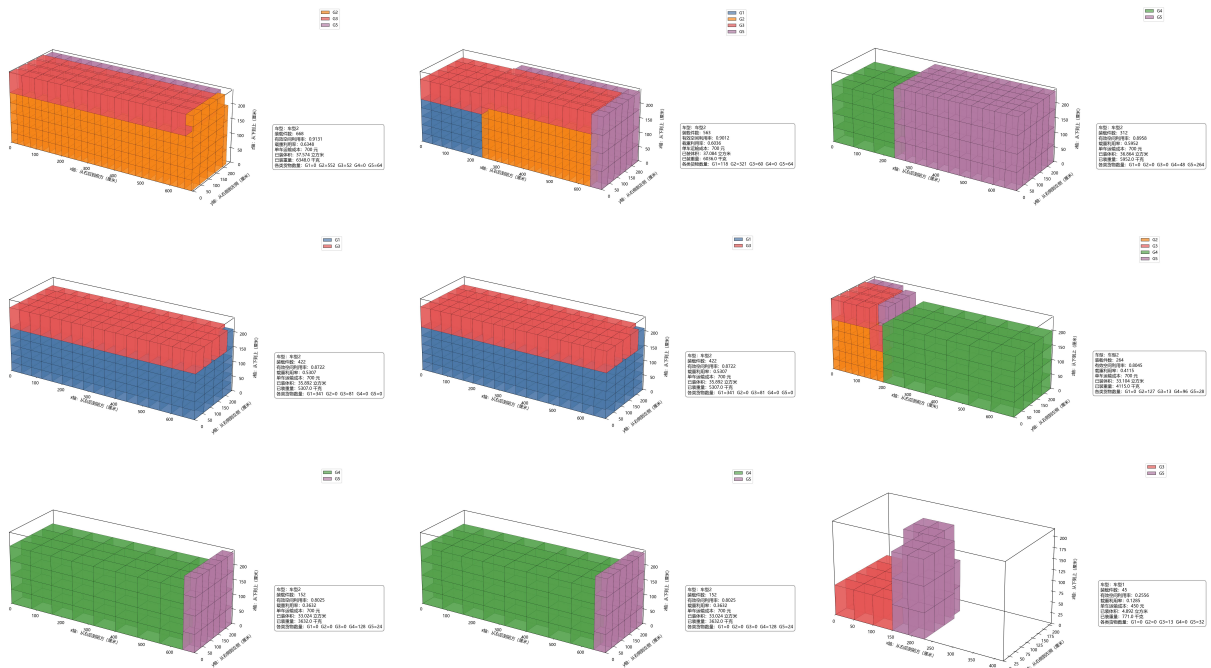


图 9: 总运输成本最少条件下货物摆放图

每辆车型 2 运输成本是 700，每辆车型 1 运输成本是 450，而总运输成本最少为目标装箱策略优化该方案下，采用 8 辆车型 2 和 1 辆车型 1，则总运输成本为 6050 元。

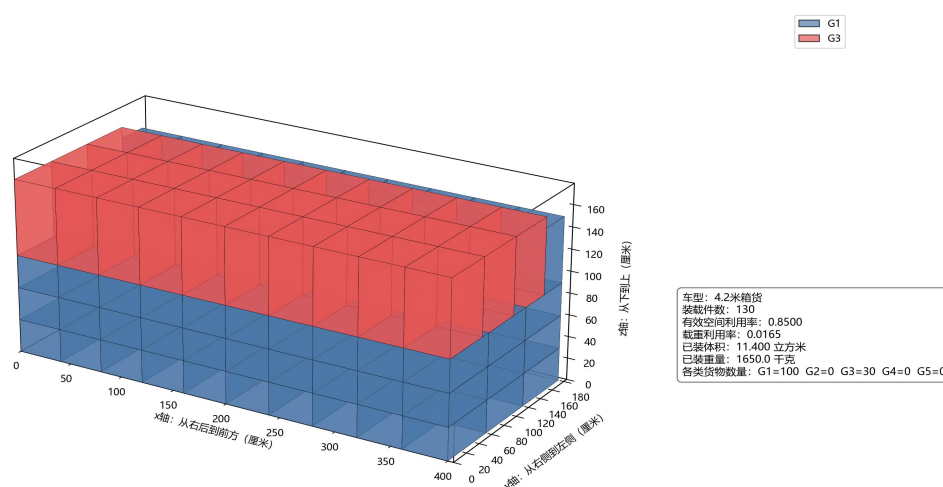
七、 问题一和问题二模型通用性的验证

为验证上述问题一和问题二模型通用性，选用附件 2 中更大规模测试数据，以附件 2 中 4.2m 箱货及 6.8m 箱货为例进行验证，由于附件中没有记录承重，假设其承重 10000kg，记 4.2 米箱货为车型 1，6.8 米箱货为车型 2。根据上文问题一和问题 2 建立的模型及求解方法，同样对 4.2m 箱货及 6.8m 箱货进行问题一及问题二的求解。

7.1 问题一模型求解结果: 针对单个车厢货物摆放问题 (以附件 2 中 4.2m 箱货及 6.8m 箱货为例)

7.1.1 车型 1(4.2m 箱货) 为装载对象

以车型 1 为装载对象，目标为单车同时达到满容率与满载率最大化，具体装箱方案示意图如下：



该图中部分货物坐标展示如下：

表 9: 车型 1 部分货物坐标展示

货物编号	X 坐标	Y 坐标	Z 坐标	长 (cm)	宽 (cm)	高 (cm)
G1	0	0	60	40	60	30
G3	0	0	90	40	50	70
G1	0	60	60	40	60	30
G3	0	50	90	40	50	70

可以看出坐标表内记录货物的长宽高与附件中货物长宽高有区别，由此可看出货物是以何姿态摆放的。

以车型 1 为装载对象，以上装箱方案经过算法计算其空间利用率为 85.00%、载重利用率 1.65%

7.1.2 车型 2(6.8m 箱货) 为装载对象

以车型 2 为装载对象，目标为单车同时达到满容率与满载率最大化，具体装箱方案示意图如下：

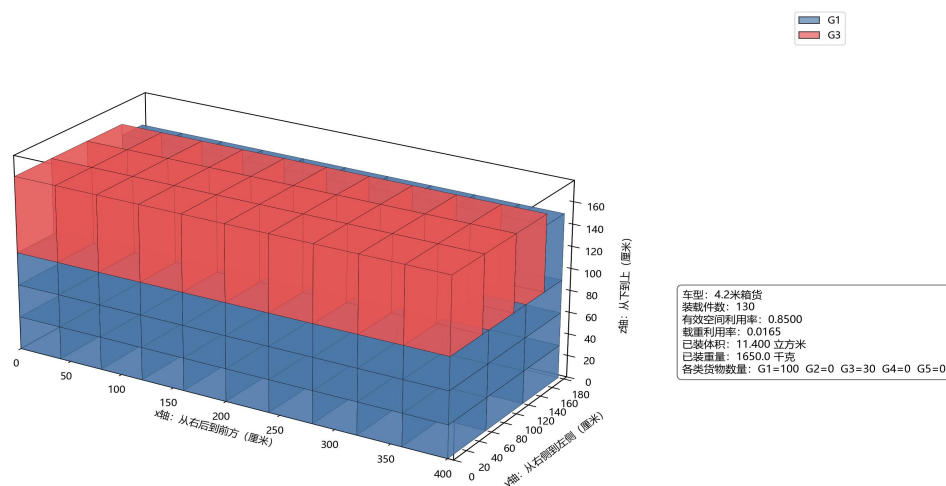


图 11: 车型 2 货物 3D 摆放图

该图中部分货物坐标展示如下:

表 10: 车型 2 部分货物坐标展示

货物编号	X 坐标	Y 坐标	Z 坐标	长 (cm)	宽 (cm)	高 (cm)
G1	0	0	90	40	60	30
G3	0	0	120	40	50	70
G1	0	60	0	40	60	30
G1	0	60	30	40	60	30

同样的,可以看出坐标表内记录货物的长宽高与附件中货物长宽高有区别,可以看出货物是以什么样的姿态摆放的。

以车型 2 为装载对象,以上装箱方案经过算法计算其空间利用率为 87.72%、载重利用率 4.47%

7.2 模型求解结果: 针对单一车型装货最小辆数问题

7.2.1 仅有车型 1(4.2m 箱货)

把附件 1 中的货物装完且所用车辆最少情况下,需要 26 辆车型 1,这里展示部分车具体装箱方案如下图:

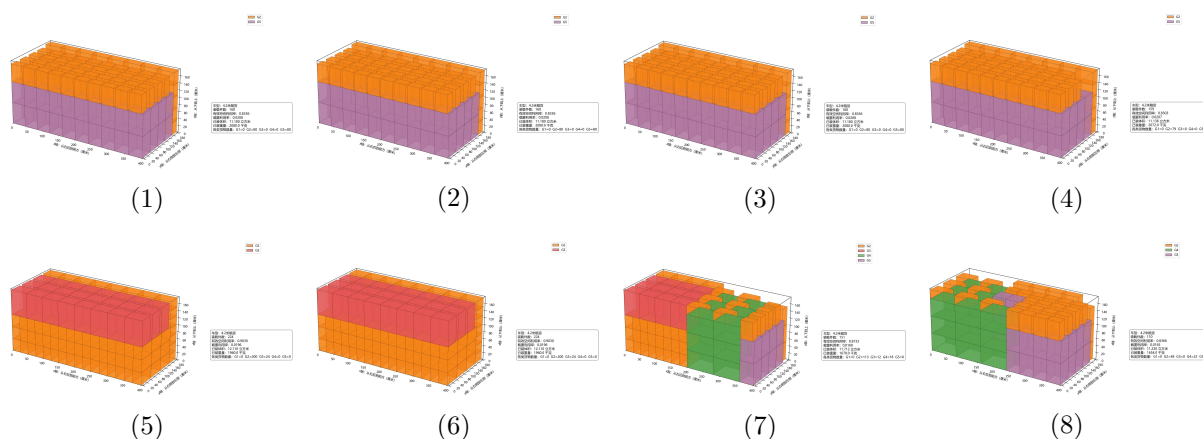


图 12: 仅有车型 1 每辆车装载方案图 (部分)

逐一计算出每辆车货物摆放坐标及长宽高数据，展示部分数据如下表：

表 11: 仅有车型 1 部分货物坐标展示

货物编号	X 坐标	Y 坐标	Z 坐标	长 (cm)	宽 (cm)	高 (cm)
G5	25	0	60	40	40	60
G5	0	40	0	40	40	60
G5	25	80	60	40	40	60
G5	25	120	60	40	40	60

同样可以看出坐标表内记录货物的长宽高与附件中货物长宽高有区别，可以看出货物是以什么样的姿态摆放的。

以上就是目标只有车型 1 把附件 1 中的货物装完且所用车辆最少的方案，采用 26 辆车型 1，再计算这种方案下每一辆车型 1 的空间利用率、载重利用率如下表：

表 12: 仅有车型 1 空间利用率及载重利用率部分展示

车辆编号	空间利用率 (%)	载重利用率 (%)	车辆编号	空间利用率 (%)	载重利用率 (%)
1	85.00	1.65	5	83.36	2.08
2	87.72	4.47	6	83.03	2.07
3	83.36	2.08	7	90.30	1.96
4	83.36	2.08	8	90.30	1.96

7.2.2 仅有车型 2(6.8m 箱货)

把附件 1 中的货物装完且所用车辆最少情况下，需要 10 辆车型 2，每一辆车具体装箱方案如下图：

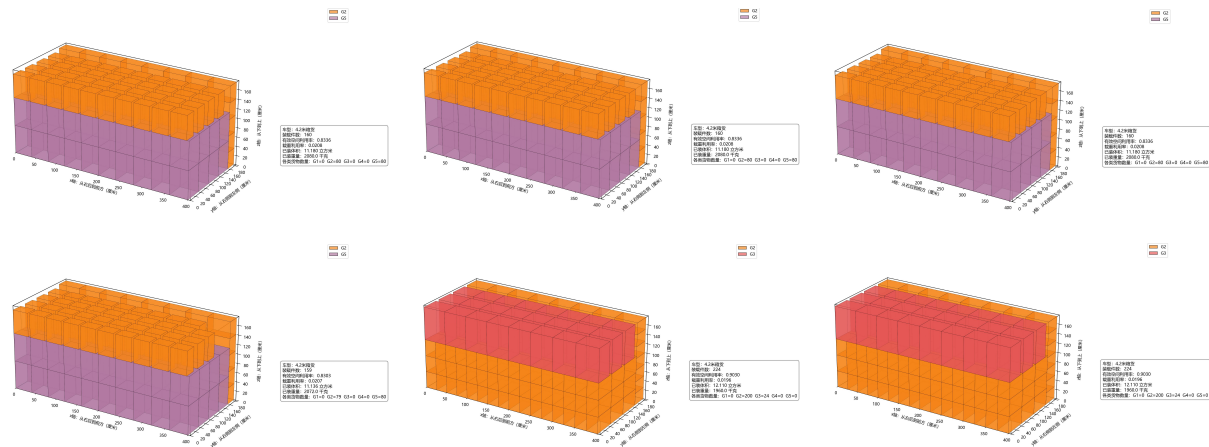


图 13: 仅有车型 2 每辆车装载方案图 (部分)

逐一计算出其货物摆放坐标及长宽高数据，展示部分数据如下表：

表 13: 仅有车型 2 部分货物坐标展示

货物编号	X 坐标	Y 坐标	Z 坐标	长 (cm)	宽 (cm)	高 (cm)
G5	0	40	60	40	40	60
G5	0	40	120	40	40	60
G5	0	80	0	40	40	60
G5	0	80	60	40	40	60

同样可以看出坐标表内记录货物的长宽高与附件中货物长宽高有区别，可以看出货物是以什么样的姿态摆放的。

以上就是目标只有车型 2 把附件 1 中的货物装完且所用车辆最少的方案，采用 10 辆车型 2，再计算这种方案下每一辆车型 2 的空间利用率、载重利用率如下表：

表 14: 仅有车型 2 部分货物坐标展示

车辆编号	空间利用率 (%)	载重利用率 (%)	车辆编号	空间利用率 (%)	载重利用率 (%)
1	82.75	5.41	6	87.72	4.47
2	86.23	4.97	7	88.25	4.17
3	86.85	4.47	8	87.29	4.15
4	87.72	4.47	9	91.55	3.42

7.3 问题二模型求解结果: 总运输车辆最少为目标 (以附件 2 中 4.2m 箱货及 6.8m 箱货为例)

本文求解出，在同时可选用车型 1(4.2m 箱货)、车型 2(6.8m 箱货) 的条件下，为了把附件 1 中的货物运完，以总运输车辆最少为目标，装箱策略是采用 2 辆车型 1、9 辆车型 2。每辆车货物摆放示意图 (部分) 如下：

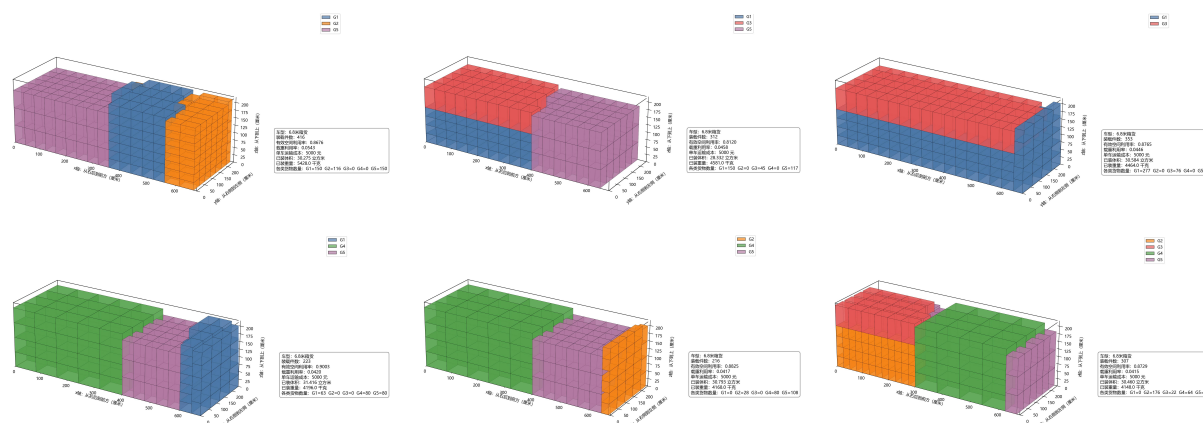


图 14: 总运输车辆最少货物摆放图 (部分)

7.4 问题二模型求解结果: 总运输成本最少为目标 (以附件 2 中 4.2m 箱货及 6.8m 箱货为例)

本文求解出，在同时可选用车型 1(4.2m 箱货)、车型 2(6.8m 箱货) 的条件下，为了把附件 1 中的货物运完，以总运输成本最少为目标，装箱策略是采用 1 辆车型 1 和 9 辆车型 2，每辆车货物摆放示意图 (部分) 如下：

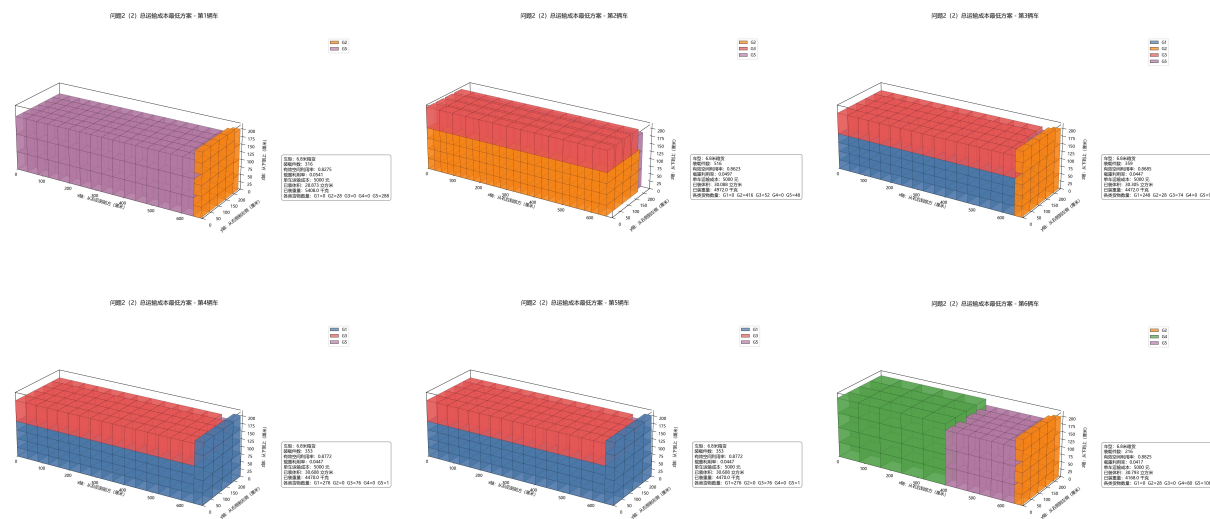


图 15: 总运输成本最少货物摆放图 (部分)

每辆车型 1 运输成本是 2800，每辆车型 2 运输成本是 5000，而总运输成本最少为目标装箱策略优化该方案下，采用 1 辆车型 1 和 9 辆车型 2，则总运输成本为 47800 元。

通过上述模型通用性检验，我们发现，本文所建立的“下层单车几何装箱—上层模式精确覆盖”框架并不依赖于原始数据中某一固定车型或某一特定货物组合，而是具有较强的数据适配能力与场景迁移能力。

八、 问题三技术报告

8.1 模型与算法介绍

该模型采用的是“下层几何装箱 + 上层模式精确覆盖”的分层框架。下层负责解决单辆车内部的真实装箱问题。在满足车厢边界、不重叠、支撑稳定性、承重链、易碎件单层放置和定向件完整投影等约束的前提下，生成合法且质量较高的单车装载模式；上层则把这些单车模式抽象为“列”，通过混合整数线性规划精确决定每种模式使用多少次，从而求解“固定车型最少车辆数”以及“联合车型最少车辆/最低成本”问题。前者采用“单车多策略择优 + 模式库 + MILP 精确覆盖”，并将 $G3$ 的合法支撑平台统一约束为标准件 $G1/G2$ ；后者则进一步扩为“两车型联合模式库 + 最少车辆/最低成本分层优化”的框架。

相比之下，单纯依靠经验装车或简单贪心规则，往往只能保证当前货物可以放下，却难以兼顾后续平台构造和整体装载质量，容易导致局部合理而全局低效，最终表现为车辆数偏多或运输成本偏高。

另一方面，若直接建立统一的大规模三维精确整数规划模型，又会将所有货物坐标、不重叠关系、支撑关系、车型选择和车辆分配同时耦合，导致模型维度过高、求解难度过大。

综上，该模型算法具有以下优点：第一，**单车层面更真实**：保留了表面分割、易碎件顶层前瞻、定向件完整投影以及组合支撑下按面积分摊载荷的机制，避免了“局部能放、整体不稳”的问题。第二，**上层优化更严格**：第一问不再只用启发式贪心，而是升级为“模式库 + MILP 精确覆盖 + 固定车数均衡优化”；第二问则进一步把“最少车数”和“最低成本”都写成分层优化问题。第三，**工程稳健性更强**：第二问采用更丰富的截断网格和模式子集生成模式库，并加入“双通道裁剪”“成本上界检查”和“失败回退”机制，因此在精度、稳定性和可解释性上都更好。

8.2 参数敏感性实验设计

表 15: 参数敏感性实验设计

实验参数	控制方式	倍率取值
定向件比例	保持总件数不变，调整 G4+G5 总量，并由 G1+G2 反向补足	0.60 / 0.80 / 1.00 / 1.20 / 1.40
易碎件比例	保持总件数不变，调整 G3 总量，并由 G1+G2 反向补足	0.50 / 0.75 / 1.00 / 1.25 / 1.50
货物重量	全部货类重量统一乘以倍率	0.80 / 0.90 / 1.00 / 1.10 / 1.20
货物尺寸	全部货类三维尺寸统一乘以倍率并取整	0.90 / 0.95 / 1.00 / 1.05 / 1.10
货物数量	全部货类数量统一乘以倍率并取整	0.80 / 0.90 / 1.00 / 1.10 / 1.20

8.2.1 定向件比例变化

表 16: 定向件比例变化敏感性结果

倍率	总成本 (元)	车辆数	车型 1	车型 2	平均空间利用率	平均载重利用率
0.60	4900	7	0	7	0.8680	0.5289
0.80	5800	9	2	7	0.7815	0.4591
1.00	6050	9	1	8	0.7911	0.4624
1.20	6750	10	1	9	0.7452	0.4318
1.40	7000	10	0	10	0.7890	0.4518

在保持总件数不变的条件下，定向件比例由基准场景的 1.00 倍下降到 0.60 倍时，总运输成本由 6050.0 元降至 4900.0 元，车辆数由 9 辆降至 7 辆；当定向件比例提高到 1.4 倍时，总成本升至 7000.0 元。这表明定向件由于摆放姿态固定、姿态受限严格，会压缩可利用的几何组合空间，导致运输成本与发车次数上升。从装载率看，平均空间利用率在 0.6 倍时达到 0.8680，在 1.2 倍时降至 0.7452，说明模型在高定向件占比下更依赖大车型补位。

8.2.2 易碎件比例变化

表 17: 易碎件比例变化敏感性结果

倍率	总成本 (元)	车辆数	车型 1	车型 2	平均空间利用率	平均载重利用率
0.50	5350	8	1	7	0.8820	0.5246
0.75	5800	9	2	7	0.8291	0.4830
1.00	6050	9	1	8	0.7911	0.4624
1.25	6750	10	1	9	0.7222	0.4171
1.50	7450	11	1	10	0.6664	0.3817

易碎件比例对方案质量的影响更为显著。由 0.50 倍上升到 1.50 倍时，总成本由 5350.0 元增加到 7450.0 元，车辆数由 8 辆上升到 11 辆。同时，平均空间利用率由 0.8820 降到 0.6664。其根本原因在于易碎件不能承重，且只能放在地板或标准件顶面，这会显著削弱上层可堆叠空间，提前触发增车。

8.2.3 货物重量变化

表 18: 货物重量变化敏感性结果

倍率	总成本 (元)	车辆数	车型 1	车型 2	平均空间利用率	平均载重利用率
0.80	6050	9	1	8	0.7911	0.3699
0.90	6050	9	1	8	0.7911	0.4161
1.00	6050	9	1	8	0.7911	0.4624
1.10	6050	9	1	8	0.7911	0.5086
1.20	6050	9	1	8	0.7911	0.5549

表 6-3 货物重量变化敏感性结果

在重量整体变化 $\pm 20\%$ 的区间内，模型输出的总成本始终保持为 6050.0 元，车辆数始终为 9 辆，平均空间利用率也保持在 0.7911 左右。但平均载重利用率会随着重量倍率近似线性变化，由 0.80 倍时的 0.3699 提高到 1.20 倍时的 0.5549。这说明当前数据下典型的空间主导型任务中，重量变化主要影响满载率评价，而不会在中等幅度内改变车型组合与运输成本。

8.2.4 货物尺寸变化

表 19: 货物尺寸变化敏感性结果

倍率	总成本 (元)	车辆数	车型 1	车型 2	平均空间利用率	平均载重利用率
0.90	5550	9	3	6	0.6102	0.4786
0.95	5600	8	0	8	0.7502	0.5138
1.00	6050	9	1	8	0.7911	0.4624
1.05	7650	12	3	9	0.7450	0.3673
1.10	8600	13	2	11	0.7486	0.3272

表 6-4 货物尺寸变化敏感性结果

尺寸变化是五类参数中影响最强的一类。相对于基准场景，尺寸缩小到 0.95 倍时，车辆数最少，仅为 8 辆；同时尺寸缩小到 0.90 倍时，总成本最低，为 5550.0 元。相反，当尺寸扩大到 1.10 倍时，总成本上升到 8600.0 元，车辆数增加到 13 辆。与此同时，平均载重利用率在 1.10 倍场景下降到 0.3272。尺寸放大会同时提高体积需求并恶化几何适配性，因此其对运输成本和装载率的放大效应最明显。

8.2.5 货物数量变化

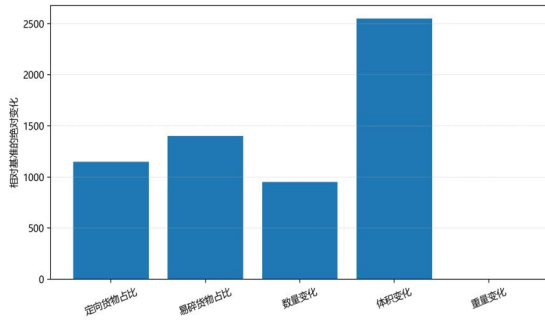
表 20: 货物数量变化敏感性结果

倍率	总成本 (元)	车辆数	车型 1	车型 2	平均空间利用率	平均载重利用率
0.80	5100	8	2	6	0.7736	0.4434
0.90	5350	8	1	7	0.8284	0.4806
1.00	6050	9	1	8	0.7911	0.4624
1.10	6750	10	1	9	0.7720	0.4531
1.20	7000	10	0	10	0.8380	0.4932

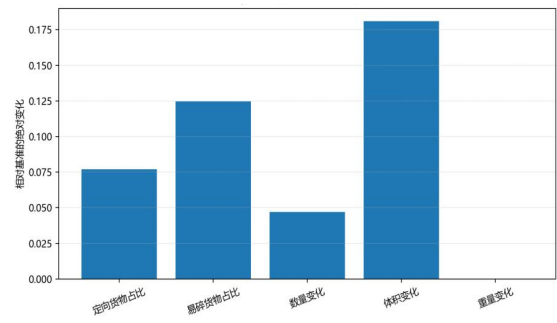
当货物数量按比例缩放时，总成本总体呈阶梯式上升：从 0.80 倍场景的 5100.0 元增加到 1.20 倍场景的 7000.0 元，车辆数由 8 辆增加到 10 辆。不过，由于车辆数是整数、车型选择具有离散跳变特征，平均空间利用率并非严格单调，而是在大车型占比变化时出现波动。这说明数量增长首先体现为发车需求增加，其次才通过车型组合变化影响平均装载率。

8.3 敏感度排序

由批量实验结果可知：尺寸变化对总运输成本、平均空间利用率和平均载重利用率的影响最强；易碎件比例次之；定向件比例和数量变化居中；重量变化在当前主导任务窗口内几乎不改变发车数和成本，但会显著改变平均载重利用率。这一结果进一步验证了本题是空间主导型装箱问题，而不是载重主导型问题。



(a) 各参数对总运输成本的敏感度排序图



(b) 各参数对装载率的敏感度排序图

图 16: 各参数敏感度排序结果图

8.4 管理建议

(1) 当订单中易碎件或定向件比例上升时，应优先考虑大容积、封闭式箱货车型，以减少因支撑受限带来的增车风险。

(2) 企业在订单预测与尺寸录入时应重点控制尺寸误差，因为尺寸放大对运输成本的冲击最为显著。

(3) 在当前数据结构下，重量波动对总成本影响有限，因此调度重点应优先放在空间组织与车型匹配上。

九、 模型的评价

9.1 模型的优点

本文模型的优点在于约束表达较为完整，具有较强的现实针对性。同时求解框架兼顾了几何真实性与全局优化能力，且具有较强的应用性。此前在技术报告已有详细说明，不再赘述。

9.2 模型不足与改进

本文模型仍存在一定不足。其一，上层优化建立在模式库基础上，因此最终结果在一定程度上依赖模式生成的充分性；其二，模型主要描述静态装箱稳定性，未进一步考虑车辆转弯、制动、振动等动态运输因素；其三，货物被统一抽象为规则长方体，未考虑托盘、包装误差和异形件等更复杂情况。

9.3 模型应用价值

从应用角度看，本文模型具有较高的实际价值。一方面，它可以直接用于装车决策支持。下层模型输出的逐件坐标与支撑关系可直接转化为仓库装车指导方案，帮助现场人员减少试装次数，提高装车标准化程度。另一方面，它也可用于企业层面的运输组织优化。第一问的模型可帮助企业判断在固定车型条件下的单车装载上限与最少车辆需求；第二问的模型则可辅助企业在“两车型联合使用”条件下判断是“优先少用车”还是“优先省成本”，以及两种目标之间的差距。这对于制定发车策略、采购车型结构和安排日常调度都具有直接参考意义。此外，本文的“下层装箱—上层模式精确覆盖”框架具有较好的可迁移性。只要货物仍以规则箱体为主、车辆类型有限、且企业关心车辆数与成本这类指标，该框架就可以迁移到仓配一体、区域配送或电商干线集货等类似场景中。

参考文献

- [1] DYCKHOFF H. A typology of cutting and packing problems[J]. European Journal of Operational Research, 1990, 44(2): 145-159.
- [2] MARTELLO S, PISINGER D, VIGO D. The three-dimensional bin packing problem[J]. Operations Research, 2000, 48(2): 256-267.
- [3] BORTFELDT A, WÄSCHER G. Constraints in container loading—A state-of-the-art review[J]. European Journal of Operational Research, 2013, 229(1): 1-20.
- [4] CRAINIC T G, PERBOLI G, TADEI R. Extreme point-based heuristics for three-dimensional bin packing[J]. INFORMS Journal on Computing, 2008, 20(3): 368-384.
- [5] GILMORE P C, GOMORY R E. A linear programming approach to the cutting-stock problem[J]. Operations Research, 1961, 9(6): 849-859.
- [6] BISCHOFF E E, RATCLIFF M S W. Issues in the development of approaches to container loading[J]. Omega, 1995, 23(4): 377-390.
- [7] BORTFELDT A, GEHRING H. A hybrid genetic algorithm for the container loading problem[J]. European Journal of Operational Research, 2001, 131(1): 143-161.

十、附录

文件名列表

第一题：附件一	Q1_求解器
附件二	Q1_输出脚本
附件三	问题一摆放图及空间载重利用率数据
第二题：附件四	Q2_求解器
附件五	Q2_输出脚本
附件六	问题二最佳方案摆放图及空间载重利用率数据

附录一 问题一_求解器.py

介绍：用于求解问题一，采用基于表面分割的三维装箱启发式算法，并结合模式库与MILP 精确覆盖方法，完成单车装载优化和单一车型下最少车辆方案求解

```
from __future__ import annotations

import copy
import math

import csv
from pathlib import Path

import numpy as np
from scipy.optimize import Bounds, LinearConstraint, milp
from dataclasses import dataclass, field
from functools import lru_cache
from itertools import combinations, permutations
from typing import Dict, Iterable, List, Optional, Sequence, Tuple

# 全局参数
SAFETY_GAP_CM = 3
BEARING_LIMIT_KG_PER_M2 = 500.0
BEARING_LIMIT_KG_PER_CM2 = BEARING_LIMIT_KG_PER_M2 / 10000.0 # 0.05 kg / cm^2
MAX_SURFACES = 400
CANDIDATE_SURFACE_LIMIT = 140
DIRECTIONAL_PROJECTION_RATIO = 1.0 # 保守处理：定向件必须整底完全投影在支撑面内
ITEM_ORDER = ("G1", "G2", "G3", "G4", "G5")
FRAGILE_CODE = "G3"
STANDARD_SUPPORT_BUILDERS = ("G1", "G2")
FRAGILE_PLATFORM_LOOKAHEAD_DEPTH = 4
FRAGILE_TOP_LINEAR_BONUS = 4.0
FRAGILE_TOP_QUADRATIC_BONUS = 6.0
FRAGILE_NEAR_TOP_BONUS = 2.0
FRAGILE_NEAR_TOP_MARGIN_CM = 20
COMPOSITE_SURFACE_PAIR_LIMIT = 120
```

一、数据结构

```
@dataclass(frozen=True)
class TruckType:
    name: str
    L: int
    W: int
    H: int
    max_weight: float
    trip_cost: float

    @property
    def effective_H(self) -> int:
        return self.H - SAFETY_GAP_CM

    @property
    def raw_volume(self) -> int:
        return self.L * self.W * self.H

    @property
    def eff_volume(self) -> int:
        return self.L * self.W * self.effective_H

@dataclass(frozen=True)
class ItemType:
    code: str
    category: str # standard / fragile / directional
    dims: Tuple[int, int, int]
    weight: float
    quantity: int

    @property
    def volume(self) -> int:
        a, b, c = self.dims
        return a * b * c

@dataclass
class PlacedBox:
    pid: int
    type_code: str
    category: str
    weight: float
    x: int
    y: int
    z: int
    l: int
    w: int
    h: int
    orientation_name: str
    parent_pid: Optional[int] = None
    support_pids: Tuple[int, ...] = ()
    load_on_top: float = 0.0

    @property
    def volume(self) -> int:
```

```

        return self.l * self.w * self.h

    @property
    def top_area_cm2(self) -> int:
        return self.l * self.w

    @property
    def bearing_capacity(self) -> float:
        if self.category == "fragile":
            return 0.0
        return BEARING_LIMIT_KG_PER_CM2 * self.top_area_cm2

@dataclass(frozen=True)
class Surface:
    x: int
    y: int
    z: int
    L: int
    W: int
    parent_pid: Optional[int] = None
    support_pids: Tuple[int, ...] = ()
    source_surfaces: Tuple["Surface", ...] = ()

    @property
    def area(self) -> int:
        return self.L * self.W

@dataclass(frozen=True)
class PackingPolicy:
    name: str
    mode: str # balanced / volume / weight / product
    category_bonus: Dict[str, float] = field(default_factory=dict)

# 二、基础数据（从附件中提取）
def build_problem_data() -> Tuple[List[TruckType], Dict[str, ItemType]]:
    def _find_file(filename: str) -> Path:
        candidates = [
            Path(filename),
            Path.cwd() / filename,
            Path("/mnt/data") / filename, # 当前对话附件环境
        ]
        for p in candidates:
            if p.exists():
                return p
        raise FileNotFoundError(f"未找到附件文件: {filename}")

    truck_file = _find_file("附件1_车型数据.csv")
    cargo_file = _find_file("附件1_货物数据.csv")

    trucks = []
    with open(truck_file, "r", encoding="gbk", newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            truck_name = row["车型名称"].split("_")[0] if row["车型名称"] else f"车型{row['车型编号']}"

```

```

        trucks.append(
            TruckType(
                truck_name,
                int(row["长(cm)"]),
                int(row["宽(cm)"]),
                int(row["高(cm)"]),
                int(row["额定载重(kg)"]),
                float(row["单次运输成本(元)"]),
            )
        )

    type_mapping = {
        "标准件": "standard",
        "易碎件": "fragile",
        "定向件": "directional",
    }

    item_types = {}
    with open(cargo_file, "r", encoding="gbk", newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            item_id = row["货物编号"]
            item_types[item_id] = ItemType(
                item_id,
                type_mapping[row["类型"]],
                (
                    int(row["长(cm)"]),
                    int(row["宽(cm)"]),
                    int(row["高(cm)"]),
                ),
                float(row["单重(kg)"]),
                int(row["数量"]),
            )

    return trucks, item_types

# 三、姿态枚举与计数工具
@lru_cache(maxsize=None)
def unique_orientations(dims: Tuple[int, int, int], category: str, fragile_fixed:
bool = False) -> Tuple[Tuple[int, int, int, str], ...]:
    if category == "directional":
        return ((dims[0], dims[1], dims[2], f"{dims[0]}x{dims[1]}x{dims[2]}"),)

    if category == "fragile" and fragile_fixed:
        return ((dims[0], dims[1], dims[2], f"{dims[0]}x{dims[1]}x{dims[2]}"),)

    out = []
    for p in set(permutations(dims, 3)):
        out.append((p[0], p[1], p[2], f"{p[0]}x{p[1]}x{p[2]}"))

    # 优先大底面、低高度
    out.sort(key=lambda x: (-(x[0] * x[1]), x[2], -max(x[0], x[1]), -min(x[0],
x[1])))
    return tuple(out)

```

```

def counts_from_catalog(catalog: Dict[str, ItemType]) -> Dict[str, int]:
    return {code: catalog[code].quantity for code in ITEM_ORDER}

def counts_tuple(counts: Dict[str, int]) -> Tuple[int, ...]:
    return tuple(counts.get(code, 0) for code in ITEM_ORDER)

def counts_dict(counts_t: Tuple[int, ...]) -> Dict[str, int]:
    return {code: counts_t[i] for i, code in enumerate(ITEM_ORDER)}

def subtract_counts(a: Dict[str, int], b: Dict[str, int]) -> Dict[str, int]:
    out = {}
    for code in ITEM_ORDER:
        v = a.get(code, 0) - b.get(code, 0)
        if v < 0:
            raise ValueError(f"计数相减出现负数: {code}")
        out[code] = v
    return out

def is_zero_counts(counts: Dict[str, int]) -> bool:
    return all(counts.get(code, 0) == 0 for code in ITEM_ORDER)

# 四、下界：最少车辆数的理论下界
def total_remaining_volume(counts: Dict[str, int], catalog: Dict[str, ItemType]) -> int:
    return sum(counts[code] * catalog[code].volume for code in ITEM_ORDER)

def total_remaining_weight(counts: Dict[str, int], catalog: Dict[str, ItemType]) -> float:
    return sum(counts[code] * catalog[code].weight for code in ITEM_ORDER)

def volume_weight_lower_bound(counts: Dict[str, int], catalog: Dict[str, ItemType],
truck: TruckType) -> int:
    total_vol = total_remaining_volume(counts, catalog)
    total_wt = total_remaining_weight(counts, catalog)
    return max(
        math.ceil(total_vol / truck.eff_volume),
        math.ceil(total_wt / truck.max_weight),
    )

# 五、核心装箱器：表面分割 + 货类计数驱动
class SurfacePacker:
    def __init__(
        self,
        truck: TruckType,
        catalog: Dict[str, ItemType],
        remaining_counts: Dict[str, int],
        policy: PackingPolicy,
        fragile_fixed: bool = False,
        projection_ratio: float = DIRECTIONAL_PROJECTION_RATIO,
    ):

```

```

self.truck = truck
self.catalog = catalog
self.policy = policy
self.fragile_fixed = fragile_fixed
self.projection_ratio = projection_ratio

self.remaining_counts = {code: remaining_counts.get(code, 0) for code in
ITEM_ORDER}
self.loaded_counts = {code: 0 for code in ITEM_ORDER}

self.placed: List[PlacedBox] = []
self.by_pid: Dict[int, PlacedBox] = {}
self.next_pid = 1

self.total_weight = 0.0
self.total_volume = 0

# 初始唯一表面: 车厢底板
self-surfaces: List[Surface] = [Surface(0, 0, 0, truck.L, truck.W, None)]

# 同一装箱状态下, 大量候选会反复查询“该表面还能否继续抬高成 G3 平台”。
# 用轻量缓存避免在每次打分时重复枚举。
self._top_fragile_fit_cache: Dict[Tuple[int, int, int], bool] = {}
self._fragile_final_top_cache: Dict[Tuple[int, int, int], int] = {}
self._future_fragile_platform_cache: Dict[Tuple[int, int, int, int, int,
Optional[int], Tuple[int, ...]], int] = {}
self._recursive_fragile_top_cache: Dict[Tuple[int, int, int, int], int] = {}

# 基础几何与稳定性检查
def rectangle_overlap_area(
    self,
    ax: int,
    ay: int,
    al: int,
    aw: int,
    bx: int,
    by: int,
    bl: int,
    bw: int,
) -> int:
    x_overlap = max(0, min(ax + al, bx + bl) - max(ax, bx))
    y_overlap = max(0, min(ay + aw, by + bw) - max(ay, by))
    return x_overlap * y_overlap

def centroid_inside_parent(self, x: int, y: int, l: int, w: int, parent:
PlacedBox) -> bool:
    cx = x + l / 2.0
    cy = y + w / 2.0
    return (parent.x <= cx <= parent.x + parent.l) and (parent.y <= cy <= parent.y
+ parent.w)

def projection_ratio_ok(self, spec: ItemType, x: int, y: int, l: int, w: int,
parent: Optional[PlacedBox]) -> bool:
    if parent is None:
        return True

    overlap = self.rectangle_overlap_area(x, y, l, w, parent.x, parent.y,
parent.l, parent.w)

```

```

ratio = overlap / (l * w)

# 定向件新增“投影面积约束”：采用保守的“整底完整支撑”策略
if spec.category == "directional" and ratio + 1e-12 < self.projection_ratio:
    return False

# 若支撑体本身是定向件，则上层货物同样要求完整落在其投影范围内，避免悬挑
if parent.category == "directional" and ratio + 1e-12 < self.projection_ratio:
    return False

return True

def surface_support_boxes(self, surface: Surface) -> List[PlacedBox]:
    support_boxes: List[PlacedBox] = []
    if surface.support_pids:
        for pid in surface.support_pids:
            box = self.by_pid.get(pid)
            if box is not None:
                support_boxes.append(box)
        return support_boxes
    if surface.parent_pid is not None:
        parent = self.by_pid.get(surface.parent_pid)
        if parent is not None:
            support_boxes.append(parent)
    return support_boxes

def surface_is_standard_support(self, surface: Surface) -> bool:
    boxes = self.surface_support_boxes(surface)
    return bool(boxes) and all(box.category == "standard" for box in boxes)

def compute_support_loads(self, item_weight: float, surface: Surface, x: int,
y: int, l: int, w: int) -> Optional[Dict[int, float]]:
    boxes = self.surface_support_boxes(surface)
    if not boxes:
        return {}
    if len(boxes) == 1:
        return {boxes[0].pid: item_weight}

    overlaps: Dict[int, int] = {}
    total_overlap = 0
    for box in boxes:
        ov = self.rectangle_overlap_area(x, y, l, w, box.x, box.y, box.l, box.w)
        if ov > 0:
            overlaps[box.pid] = ov
            total_overlap += ov
    if total_overlap < l * w:
        return None
    return {pid: item_weight * ov / total_overlap for pid, ov in overlaps.items()}

def can_support(self, spec: ItemType, surface: Surface, l: int, w: int) -> bool:
    support_boxes = self.surface_support_boxes(surface)

    # 落地总是允许
    if not support_boxes:
        return True

    # 任一支撑箱为易碎件都不允许
    if any(box.category == "fragile" for box in support_boxes):

```

```

        return False

    # 组合面仅给 G3 使用；且其支撑箱必须全部为标准件
    if len(support_boxes) >= 2:
        if spec.category != "fragile":
            return False
        if not all(box.category == "standard" for box in support_boxes):
            return False
    else:
        parent = support_boxes[0]
        if spec.category == "fragile" and parent.category != "standard":
            return False
        if not self.centroid_inside_parent(surface.x, surface.y, l, w, parent):
            return False
        if not self.projection_ratio_ok(spec, surface.x, surface.y, l, w,
parent):
            return False

    load_map = self.compute_support_loads(spec.weight, surface, surface.x,
surface.y, l, w)
    if load_map is None:
        return False

    # 承重链约束：组合面按覆盖面积分摊重量
    for pid, delta in load_map.items():
        cur = self.by_pid.get(pid)
        while cur is not None:
            if cur.load_on_top + delta > cur.bearing_capacity + 1e-12:
                return False
            cur = self.by_pid.get(cur.parent_pid)
    return True

def apply_load_update(self, load_map: Dict[int, float]) -> None:
    for pid, delta in load_map.items():
        cur = self.by_pid.get(pid)
        while cur is not None:
            cur.load_on_top += delta
            cur = self.by_pid.get(cur.parent_pid)

def reset_state_caches(self) -> None:
    self._top_fragile_fit_cache.clear()
    self._fragile_final_top_cache.clear()
    self._future_fragile_platform_cache.clear()
    self._recursive_fragile_top_cache.clear()

    def surface_cache_key(self, surface: Surface) -> Tuple[int, int, int, int, int,
Optional[int], Tuple[int, ...]]:
        return (surface.x, surface.y, surface.z, surface.L, surface.W,
surface.parent_pid, tuple(surface.support_pids))

    def remaining_fragile_exists(self) -> bool:
        return self.remaining_counts.get(FRAGILE_CODE, 0) > 0

    def remaining_standard_support_builder_exists(self) -> bool:
        return any(self.remaining_counts.get(code, 0) > 0 for code in
STANDARD_SUPPORT_BUILDERS)

    def best_fragile_final_top_z_for_top(self, L: int, W: int, top_z: int) -> int:

```

```

        key = (L, W, top_z)
        if key in self._fragile_final_top_cache:
            return self._fragile_final_top_cache[key]

        fragile = self.catalog[FRAGILE_CODE]
        best_final_top = -1
        for fl, fw, fh, _ in unique_orientations(fragile.dims, fragile.category,
self.fragile_fixed):
            if fl <= L and fw <= W and top_z + fh <= self.truck.effective_H:
                best_final_top = max(best_final_top, top_z + fh)
            self._fragile_final_top_cache[key] = best_final_top
        return best_final_top

    def _best_recursive_fragile_top_z_from_top(self, L: int, W: int, top_z: int,
depth: int) -> int:
        key = (L, W, top_z, depth)
        if key in self._recursive_fragile_top_cache:
            return self._recursive_fragile_top_cache[key]

        best = self.best_fragile_final_top_z_for_top(L, W, top_z)
        if depth <= 0 or top_z >= self.truck.effective_H or not
self.remaining_standard_support_builder_exists():
            self._recursive_fragile_top_cache[key] = best
            return best

        for code in STANDARD_SUPPORT_BUILDERS:
            if self.remaining_counts.get(code, 0) <= 0:
                continue
            spec = self.catalog[code]
            for nl, nw, nh, _ in unique_orientations(spec.dims, spec.category,
self.fragile_fixed):
                if nl > L or nw > W:
                    continue
                if top_z + nh > self.truck.effective_H:
                    continue
                child = self._best_recursive_fragile_top_z_from_top(nl, nw, top_z +
nh, depth - 1)
                best = max(best, child)

        self._recursive_fragile_top_cache[key] = best
        return best

    def top_can_host_fragile(self, L: int, W: int, top_z: int) -> bool:
        key = (L, W, top_z)
        if key in self._top_fragile_fit_cache:
            return self._top_fragile_fit_cache[key]
        ok = self.best_fragile_final_top_z_for_top(L, W, top_z) >= 0
        self._top_fragile_fit_cache[key] = ok
        return ok

    def merged_adjacent_rect(self, x1: int, y1: int, L1: int, W1: int, x2: int, y2:
int, L2: int, W2: int) -> Optional[Tuple[int, int, int, int]]:
        if x1 == x2 and L1 == L2 and (y1 + W1 == y2 or y2 + W2 == y1):
            return (x1, min(y1, y2), L1, W1 + W2)
        if y1 == y2 and W1 == W2 and (x1 + L1 == x2 or x2 + L2 == x1):
            return (min(x1, x2), y1, L1 + L2, W1)
        return None

    def best_fragile_top_from_existing_neighbors(self, x: int, y: int, L: int, W:

```

```

int, top_z: int, exclude_pids: Tuple[int, ...] = ()) -> int:
    best = -1
    for other in self-surfaces:
        if other.z != top_z or other.parent_pid is None or other.parent_pid in
exclude_pids:
            continue
        parent = self.by_pid.get(other.parent_pid)
        if parent is None or parent.category != "standard":
            continue
        merged = self.merged_adjacent_rect(x, y, L, W, other.x, other.y, other.L,
other.W)
        if merged is None:
            continue
        _, _, mL, mW = merged
        best = max(best, self.best_fragile_final_top_z_for_top(mL, mW, top_z))
    return best

def best_fragile_top_from_same_level_sibling_after_builder(
    self,
    surface: Surface,
    code: str,
    l: int,
    w: int,
    h: int,
) -> int:
    """估计“先抬当前柱、再抬同层相邻柱”后形成更高 G3 支撑面的潜力。

    这是针对“G2 只堆一层就开始放 G3”问题的关键修正：
    当前代码原本只能看到“当前柱抬高后与已经同高的邻柱拼面”，
    却看不到“相邻柱下一步也能同步抬高到同一层”这一常见情形。
    因而会在较低层过早放置 G3。这里显式地把这类两步平台构造
    （当前柱 + 同层相邻柱）纳入前瞻评估。
    """
    if self.remaining_counts.get(code, 0) < 2:
        return -1

    spec = self.catalog[code]
    top_z = surface.z + h
    best = -1

    current_support = tuple(sorted(surface.support_pids)) if
surface.support_pids else (()) if surface.parent_pid is None else
(surface.parent_pid,)

    for other in self-surfaces:
        if other is surface:
            continue
        if other.z != surface.z:
            continue

        other_support_boxes = self.surface_support_boxes(other)
        if not other_support_boxes or not all(box.category == "standard" for box
in other_support_boxes):
            continue

        other_support = tuple(sorted(other.support_pids)) if other.support_pids
else (()) if other.parent_pid is None else (other.parent_pid,)
        if other_support == current_support:

```

```

        # 同一父面的再摆放潜力已经由“同一支撑面再补一件”逻辑覆盖。
        continue

        if l > other.L or w > other.W:
            continue
        if other.z + h > self.truck.effective_H:
            continue
        if not self.can_support(spec, other, l, w):
            continue

        merged = self.merged_adjacent_rect(surface.x, surface.y, l, w, other.x,
other.y, l, w)
        if merged is None:
            continue
        _, _, mL, mW = merged
        best = max(best, self.best_fragile_final_top_z_for_top(mL, mW, top_z))

    return best

    def best_future_fragile_top_after_builder(self, surface: Surface, code: str, l:
int, w: int, h: int) -> int:
        top_z = surface.z + h
        best = self._best_recursive_fragile_top_z_from_top(l, w, top_z,
FRAGILE_PLATFORM_LOOKAHEAD_DEPTH - 1)

        # 情形 A: 当前柱抬高后, 已经存在同高邻柱, 可立即形成更高拼面。
        best = max(best, self.best_fragile_top_from_existing_neighbors(surface.x,
surface.y, l, w, top_z))

        # 情形 B: 当前支撑面本身还足够宽/长, 再补一件同类标准件后形成更高拼面。
        if self.remaining_counts.get(code, 0) >= 2:
            if 2 * l <= surface.L:
                best = max(best, self.best_fragile_final_top_z_for_top(2 * l, w,
top_z))
            if 2 * w <= surface.W:
                best = max(best, self.best_fragile_final_top_z_for_top(l, 2 * w,
top_z))

        # 情形 C: 当前柱先抬高一步, 随后把“同层相邻柱”也抬到同一高度,
        # 再在更高层形成 G3 支撑面。这个分支正是用于抑制“G2 只放一层就开始放 G3”。
        best = max(best,
self.best_fragile_top_from_same_level_sibling_after_builder(surface, code, l, w,
h))

    return best

    def best_future_fragile_top_z_from_surface(self, surface: Surface) -> int:
        key = self.surface_cache_key(surface)
        if key in self._future_fragile_platform_cache:
            return self._future_fragile_platform_cache[key]

        if not self.remaining_fragile_exists() or not
self.remaining_standard_support_builder_exists():
            self._future_fragile_platform_cache[key] = -1
            return -1

        best_final_top = -1
        for code in STANDARD_SUPPORT_BUILDERS:
            if self.remaining_counts.get(code, 0) <= 0:

```

```

        continue
    spec = self.catalog[code]
    if self.total_weight + spec.weight > self.truck.max_weight + 1e-12:
        continue

    for l, w, h, _ in unique_orientations(spec.dims, spec.category,
self.fragile_fixed):
        if l > surface.L or w > surface.W:
            continue
        if surface.z + h > self.truck.effective_H:
            continue
        if not self.can_support(spec, surface, l, w):
            continue

        final_top = self._best_recursive_fragile_top_z_from_top(
            l, w, surface.z + h, FRAGILE_PLATFORM_LOOKAHEAD_DEPTH - 1
        )
        if final_top >= 0:
            best_final_top = max(best_final_top, final_top)

    self._future_fragile_platform_cache[key] = best_final_top
    return best_final_top

def generate_fragile_composite_surfaces(self, candidate_surfaces:
List[Surface]) -> List[Surface]:
    standard_surfaces = [
        s for s in candidate_surfaces
        if s.parent_pid is not None and self.surface_is_standard_support(s)
   ][:COMPOSITE_SURFACE_PAIR_LIMIT]

    best_by_key: Dict[Tuple[int, int, int, int, int, Tuple[int, ...]], Surface]
= {}

    for i, s1 in enumerate(standard_surfaces):
        p1 = self.by_pid.get(s1.parent_pid)
        if p1 is None or p1.category != "standard":
            continue
        for s2 in standard_surfaces[i + 1 :]:
            p2 = self.by_pid.get(s2.parent_pid)
            if p2 is None or p2.category != "standard" or p2.pid == p1.pid:
                continue
            if s1.z != s2.z:
                continue

            merged: Optional[Surface] = None
            # 纵向拼接: 同 x / L, 相邻 y
            if s1.x == s2.x and s1.L == s2.L and (s1.y + s1.W == s2.y or s2.y +
s2.W == s1.y):
                merged = Surface(
                    x=s1.x,
                    y=min(s1.y, s2.y),
                    z=s1.z,
                    L=s1.L,
                    W=s1.W + s2.W,
                    parent_pid=None,
                    support_pids=tuple(sorted((p1.pid, p2.pid))),
                    source_surfaces=(s1, s2),
                )
            # 横向拼接: 同 y / W, 相邻 x

```

```

        elif s1.y == s2.y and s1.W == s2.W and (s1.x + s1.L == s2.x or s2.x
+ s2.L == s1.x):
            merged = Surface(
                x=min(s1.x, s2.x),
                y=s1.y,
                z=s1.z,
                L=s1.L + s2.L,
                W=s1.W,
                parent_pid=None,
                support_pids=tuple(sorted((p1.pid, p2.pid))),
                source_surfaces=(s1, s2),
            )
            if merged is None:
                continue
            if self.best_fragile_final_top_z_for_top(merged.L, merged.W,
merged.z) < 0:
                continue
            ckey = (merged.x, merged.y, merged.z, merged.L, merged.W,
merged.support_pids)
            best_by_key[ckey] = merged
            return list(best_by_key.values())

    def best_supported_fragile_choice(self, candidate_surfaces: List[Surface]) ->
    Tuple[Optional[Tuple[ItemType, Surface, int, int, int, str]], int]:
        if self.remaining_counts.get(FRAGILE_CODE, 0) <= 0:
            return None, -1

        spec = self.catalog[FRAGILE_CODE]
        if self.total_weight + spec.weight > self.truck.max_weight + 1e-12:
            return None, -1

        best_choice = None
        best_key = None
        best_final_top = -1

        for surface in candidate_surfaces:
            support_boxes = self.surface_support_boxes(surface)
            if support_boxes and not all(box.category == "standard" for box in
support_boxes):
                continue
            # 顶层优先阶段只考虑“可抬高后的高位平台”；地板情况交给通用搜索处理。
            if not support_boxes:
                continue

            for l, w, h, orient_name in unique_orientations(spec.dims, spec.category,
self.fragile_fixed):
                if l > surface.L or w > surface.W:
                    continue
                if surface.z + h > self.truck.effective_H:
                    continue
                if not self.can_support(spec, surface, l, w):
                    continue

                final_top = surface.z + h
                composite_bonus = 1 if len(surface.support_pids) >= 2 else 0
                key = (final_top, composite_bonus) + self.candidate_key(spec,
surface, l, w, h)
                if best_key is None or key > best_key:
                    best_key = key

```

```

        best_choice = (spec, surface, l, w, h, orient_name)
        best_final_top = final_top

    return best_choice, best_final_top

    def best_platform_builder_choice(self, candidate_surfaces: List[Surface]) ->
    Tuple[Optional[Tuple[ItemType, Surface, int, int, int, str]], int]:
        if not self.remaining_fragile_exists() or not
self.remaining_standard_support_builder_exists():
            return None, -1

        best_choice = None
        best_key = None
        best_future_final_top = -1

        for code in STANDARD_SUPPORT_BUILDERS:
            if self.remaining_counts.get(code, 0) <= 0:
                continue
            spec = self.catalog[code]
            if self.total_weight + spec.weight > self.truck.max_weight + 1e-12:
                continue

            for surface in candidate_surfaces:
                for l, w, h, orient_name in unique_orientations(spec.dims,
spec.category, self.fragile_fixed):
                    if l > surface.L or w > surface.W:
                        continue
                    if surface.z + h > self.truck.effective_H:
                        continue
                    if not self.can_support(spec, surface, l, w):
                        continue

                    future_final_top =
self.best_future_fragile_top_after_builder(surface, code, l, w, h)
                    if future_final_top < 0:
                        continue

                    key = (future_final_top,) + self.candidate_key(spec, surface, l,
w, h)

                    if best_key is None or key > best_key:
                        best_key = key
                        best_choice = (spec, surface, l, w, h, orient_name)
                        best_future_final_top = future_final_top

        return best_choice, best_future_final_top

    def select_toplayer_fragile_priority_choice(self, candidate_surfaces:
List[Surface]) -> Optional[Tuple[ItemType, Surface, int, int, int, str]]:
        if not self.remaining_fragile_exists():
            return None

        fragile_surfaces = list(candidate_surfaces) +
self.generate_fragile_composite_surfaces(candidate_surfaces)
        fragile_choice, fragile_final_top =
self.best_supported_fragile_choice(fragile_surfaces)
        builder_choice, builder_future_final_top =
self.best_platform_builder_choice(candidate_surfaces)

        if fragile_choice is None and builder_choice is None:

```

```

        return None
    if fragile_choice is None:
        return builder_choice
    if builder_choice is None:
        return fragile_choice

    # 只要还能明显抬高 G3 的最终顶高，就继续造平台；
    # 否则直接把 G3 放到当前更高的位置。
    if builder_future_final_top > fragile_final_top + 1:
        return builder_choice
    return fragile_choice

# 表面维护
def split_surface(self, surface: Surface, l: int, w: int) -> List[Surface]:
    # 切法 A
    a_surfaces: List[Surface] = []
    if surface.L - l > 0:
        a_surfaces.append(Surface(surface.x + l, surface.y, surface.z, surface.L
- l, surface.W, surface.parent_pid))
        if surface.W - w > 0:
            a_surfaces.append(Surface(surface.x, surface.y + w, surface.z, l,
surface.W - w, surface.parent_pid))
        score_a = sum(s.area for s in a_surfaces) - abs((surface.L - l) - l) -
abs((surface.W - w) - w)

    # 切法 B
    b_surfaces: List[Surface] = []
    if surface.W - w > 0:
        b_surfaces.append(Surface(surface.x, surface.y + w, surface.z,
surface.L, surface.W - w, surface.parent_pid))
        if surface.L - l > 0:
            b_surfaces.append(Surface(surface.x + l, surface.y, surface.z, surface.L
- l, w, surface.parent_pid))
        score_b = sum(s.area for s in b_surfaces) - abs((surface.L - l) - l) -
abs((surface.W - w) - w)

    return a_surfaces if score_a >= score_b else b_surfaces

def subtract_rect_from_surface(self, surface: Surface, rx: int, ry: int, rl: int,
rw: int) -> List[Surface]:
    ix0 = max(surface.x, rx)
    iy0 = max(surface.y, ry)
    ix1 = min(surface.x + surface.L, rx + rl)
    iy1 = min(surface.y + surface.W, ry + rw)
    if ix0 >= ix1 or iy0 >= iy1:
        return [surface]

    out: List[Surface] = []
    # left / right strips
    if ix0 > surface.x:
        out.append(Surface(surface.x, surface.y, surface.z, ix0 - surface.x,
surface.W, surface.parent_pid))
    if ix1 < surface.x + surface.L:
        out.append(Surface(ix1, surface.y, surface.z, surface.x + surface.L -
ix1, surface.W, surface.parent_pid))
    # bottom / top strips inside the overlapped x-span
    mid_L = ix1 - ix0
    if iy0 > surface.y:

```

```

        out.append(Surface(ix0, surface.y, surface.z, mid_L, iy0 - surface.y,
surface.parent_pid))
        if iy1 < surface.y + surface.W:
            out.append(Surface(ix0, iy1, surface.z, mid_L, surface.y + surface.W -
iy1, surface.parent_pid))
        return [s for s in out if s.L > 0 and s.W > 0]

def _merge_surfaces_once(self, surfaces: List[Surface]) -> List[Surface]:
    # 先横向合并
    grouped: Dict[Tuple[int, Optional[int], int, int], List[Surface]] = {}
    for s in surfaces:
        grouped.setdefault((s.z, s.parent_pid, s.y, s.W), []).append(s)

    merged_h: List[Surface] = []
    for key, group in grouped.items():
        group = sorted(group, key=lambda s: (s.x, s.L))
        cur = group[0]
        for nxt in group[1:]:
            if cur.x + cur.L == nxt.x:
                cur = Surface(cur.x, cur.y, cur.z, cur.L + nxt.L, cur.W,
cur.parent_pid)
            else:
                merged_h.append(cur)
                cur = nxt
        merged_h.append(cur)

    # 再纵向合并
    grouped2: Dict[Tuple[int, Optional[int], int, int], List[Surface]] = {}
    for s in merged_h:
        grouped2.setdefault((s.z, s.parent_pid, s.x, s.L), []).append(s)

    merged_v: List[Surface] = []
    for key, group in grouped2.items():
        group = sorted(group, key=lambda s: (s.y, s.W))
        cur = group[0]
        for nxt in group[1:]:
            if cur.y + cur.W == nxt.y:
                cur = Surface(cur.x, cur.y, cur.z, cur.L, cur.W + nxt.W,
cur.parent_pid)
            else:
                merged_v.append(cur)
                cur = nxt
        merged_v.append(cur)

    return merged_v

def prune_surfaces(self) -> None:
    surfaces = [
        s for s in self.surfaces
        if s.L > 0 and s.W > 0 and s.z <= self.truck.effective_H
    ]

    # 去除被包含表面
    surfaces.sort(key=lambda s: (-s.area, s.z, s.x, s.y))
    kept: List[Surface] = []
    for s in surfaces:
        covered = False
        for k in kept:

```

```

        if (
            s.parent_pid == k.parent_pid
            and s.z == k.z
            and s.x >= k.x
            and s.y >= k.y
            and s.x + s.L <= k.x + k.L
            and s.y + s.W <= k.y + k.W
        ):
            covered = True
            break
    if not covered:
        kept.append(s)

# 进行两轮邻接面合并
for _ in range(2):
    kept = self._merge_surfaces_once(kept)

kept.sort(key=lambda s: (s.z, s.x, s.y, -s.area))
self.surfaces = kept[:MAX_SURFACES]

# 目标与评分
def utilization(self) -> Dict[str, float]:
    return {
        "空间利用率_raw": self.total_volume / self.truck.raw_volume,
        "空间利用率_eff": self.total_volume / self.truck.eff_volume,
        "载重利用率": self.total_weight / self.truck.max_weight,
        "已装货物体积_m3": self.total_volume / 1e6,
        "已装货物重量_kg": self.total_weight,
    }

def truck_score_tuple(self) -> Tuple[float, float, float, float, float]:
    util = self.utilization()
    v = util["空间利用率_eff"]
    w = util["载重利用率"]
    return (
        min(v, w),
        (v + w) / 2.0,
        -abs(v - w),
        v * w,
        len(self.placed),
    )

def candidate_key(self, spec: ItemType, surface: Surface, l: int, w: int, h: int)
-> Tuple[float, ...]:
    new_v = (self.total_volume + l * w * h) / self.truck.eff_volume
    new_w = (self.total_weight + spec.weight) / self.truck.max_weight
    fill = (l * w) / (surface.L * surface.W)
    cat_bonus = self.policy.category_bonus.get(spec.category, 0.0)
    gap = abs(new_v - new_w)

    support_boxes = self.surface_support_boxes(surface)
    has_standard_support = bool(support_boxes) and all(box.category ==
"standard" for box in support_boxes)
    is_composite_support = len(surface.support_pids) >= 2

    fragile_support_bonus = 0.0
    standard_platform_bonus = 0.0
    top_surface_penalty = 0.0

```

```

top_layer_bonus = 0.0
future_stack_penalty = 0.0
shape_pref = -float(h)

if spec.category == "fragile":
    shape_pref = float(h)
    placed_top_z = surface.z + h
    top_ratio = placed_top_z / self.truck.effective_H
    top_layer_bonus = (
        FRAGILE_TOP_LINEAR_BONUS * top_ratio
        + FRAGILE_TOP_QUADRATIC_BONUS * (top_ratio ** 2)
    )
    if placed_top_z >= self.truck.effective_H - FRAGILE_NEAR_TOP_MARGIN_CM:
        top_layer_bonus += FRAGILE_NEAR_TOP_BONUS

    if has_standard_support:
        fragile_support_bonus = 1.2 + (0.35 if is_composite_support else 0.0)
        if not is_composite_support:
            future_top_z =
self.best_future_fragile_top_z_from_surface(surface)
            if future_top_z > placed_top_z + 1:
                gap_ratio = (future_top_z - placed_top_z) /
self.truck.effective_H
                future_stack_penalty = -(2.8 + 3.5 * gap_ratio)
            elif self.remaining_standard_support_builder_exists():
                future_stack_penalty = -1.5

    if spec.category == "standard" and self.remaining_fragile_exists():
        top_z = surface.z + h
        future_fragile_top =
self.best_future_fragile_top_after_builder(surface, spec.code, l, w, h)
        if future_fragile_top >= 0:
            standard_platform_bonus = 1.0 + 2.8 * (future_fragile_top /
self.truck.effective_H)
            if future_fragile_top > top_z + 1:
                standard_platform_bonus += 0.8
        elif surface.z == 0:
            standard_platform_bonus = 0.1

    if (
        support_boxes
        and len(support_boxes) == 1
        and support_boxes[0].category == "standard"
        and spec.category == "directional"
        and self.remaining_fragile_exists()
    ):
        top_surface_penalty = -0.7

    if self.policy.mode == "balanced":
        main = (min(new_v, new_w), (new_v + new_w) / 2.0, -gap, new_v * new_w)
    elif self.policy.mode == "volume":
        main = (new_v, min(new_v, new_w), (new_v + new_w) / 2.0, -gap)
    elif self.policy.mode == "weight":
        main = (new_w, min(new_v, new_w), (new_v + new_w) / 2.0, -gap)
    elif self.policy.mode == "product":
        main = (new_v * new_w, min(new_v, new_w), (new_v + new_w) / 2.0, -gap)
    else:
        raise ValueError(f"未知策略模式: {self.policy.mode}")

```

```

        return main + (
            fragile_support_bonus,
            standard_platform_bonus,
            top_surface_penalty,
            top_layer_bonus,
            future_stack_penalty,
            shape_pref,
            fill,
            cat_bonus,
            -surface.z,
            -surface.x,
            -surface.y,
        )

    # 放置操作
    def place(self, spec: ItemType, surface: Surface, l: int, w: int, h: int,
orient_name: str) -> None:
        pid = self.next_pid
        self.next_pid += 1

        support_pids = tuple(surface.support_pids) if surface.support_pids else (())
        if surface.parent_pid is None else (surface.parent_pid,)
        support_parent = surface.parent_pid if not surface.support_pids else None
        placed = PlacedBox(
            pid=pid,
            type_code=spec.code,
            category=spec.category,
            weight=spec.weight,
            x=surface.x,
            y=surface.y,
            z=surface.z,
            l=l,
            w=w,
            h=h,
            orientation_name=orient_name,
            parent_pid=support_parent,
            support_pids=support_pids,
        )

        self.placed.append(placed)
        self.by_pid[pid] = placed
        self.total_weight += spec.weight
        self.total_volume += placed.volume
        self.remaining_counts[spec.code] -= 1
        self.loaded_counts[spec.code] += 1

        load_map = self.compute_support_loads(spec.weight, surface, placed.x,
placed.y, l, w)
        if load_map is None:
            raise RuntimeError("组合支撑面重量分配失败。")
        self.apply_load_update(load_map)

        if surface.source_surfaces:
            for src in surface.source_surfaces:
                if src in self.surfaces:
                    self.surfaces.remove(src)
                    self.surfaces.extend(self.subtract_rect_from_surface(src,
placed.x, placed.y, l, w))

```

```

else:
    self.surfaces.remove(surface)
    self.surfaces.extend(self.split_surface(surface, l, w))

if spec.category != "fragile":
    top_z = surface.z + h
    if top_z <= self.truck.effective_H:
        self.surfaces.append(Surface(surface.x, surface.y, top_z, l, w,
pid))

self.prune_surfaces()
self.reset_state_caches()

# 单车装箱主过程
def pack(self) -> "SurfacePacker":
    while self.surfaces and not is_zero_counts(self.remaining_counts):
        self.surfaces.sort(key=lambda s: (s.z, s.x, s.y, -s.area))
        base_candidate_surfaces = self.surfaces[:CANDIDATE_SURFACE_LIMIT]

        priority_choice =
self.select_toplayer_fragile_priority_choice(base_candidate_surfaces)
        if priority_choice is not None:
            spec, surface, l, w, h, orient_name = priority_choice
            self.place(spec, surface, l, w, h, orient_name)
            continue

        best_choice = None
        best_key = None

        for code in ITEM_ORDER:
            if self.remaining_counts.get(code, 0) <= 0:
                continue
            spec = self.catalog[code]

            if self.total_weight + spec.weight > self.truck.max_weight + 1e-12:
                continue

            for surface in base_candidate_surfaces:
                for l, w, h, orient_name in unique_orientations(spec.dims,
spec.category, self.fragile_fixed):
                    if l > surface.L or w > surface.W:
                        continue
                    if surface.z + h > self.truck.effective_H:
                        continue
                    if not self.can_support(spec, surface, l, w):
                        continue

                    key = self.candidate_key(spec, surface, l, w, h)
                    if best_key is None or key > best_key:
                        best_key = key
                        best_choice = (spec, surface, l, w, h, orient_name)

            if best_choice is None:
                break

            spec, surface, l, w, h, orient_name = best_choice
            self.place(spec, surface, l, w, h, orient_name)

    return self

```

六、策略集

```
SINGLE_TRUCK_POLICIES: Tuple[PackingPolicy, ...] = (  
    PackingPolicy(  
        "balanced_platform",  
        "balanced",  
        {"standard": 2.0, "directional": 1.0, "fragile": 0.0},  
    ),  
    PackingPolicy(  
        "balanced_directional",  
        "balanced",  
        {"directional": 2.0, "standard": 1.0, "fragile": 0.0},  
    ),  
    PackingPolicy(  
        "product_mix",  
        "product",  
        {"standard": 1.5, "directional": 1.5, "fragile": 0.2},  
    ),  
    PackingPolicy(  
        "volume_first",  
        "volume",  
        {"standard": 1.5, "directional": 1.0, "fragile": 0.1},  
    ),  
    PackingPolicy(  
        "weight_first",  
        "weight",  
        {"standard": 1.0, "directional": 1.0, "fragile": 0.5},  
    ),  
)
```

```
MIN_TRUCK_POLICIES: Tuple[PackingPolicy, ...] = (  
    PackingPolicy(  
        "cover_volume",  
        "volume",  
        {"standard": 1.8, "directional": 1.0, "fragile": 0.2},  
    ),  
    PackingPolicy(  
        "cover_balanced",  
        "balanced",  
        {"standard": 2.0, "directional": 1.0, "fragile": 0.0},  
    ),  
    PackingPolicy(  
        "cover_product",  
        "product",  
        {"standard": 1.3, "directional": 1.3, "fragile": 0.0},  
    ),  
    PackingPolicy(  
        "fragile_rescue",  
        "volume",  
        {"fragile": 2.0, "standard": 1.0, "directional": 0.5},  
    ),  
    PackingPolicy(  
        "heavy_mix",  
        "weight",  
        {"directional": 1.5, "standard": 1.0, "fragile": 0.2},  
    ),  
)
```

七、候选模式生成、单车择优、最少车辆搜索

```
def compare_balance_packers(packer: SurfacePacker) -> Tuple[float, float, float, float, float]:
```

```
    util = packer.utilization()
```

```
    v = util["空间利用率_eff"]
```

```
    w = util["载重利用率"]
```

```
    return (min(v, w), (v + w) / 2.0, -abs(v - w), v * w, len(packer.placed))
```

```
def compare_cover_packers(packer: SurfacePacker) -> Tuple[float, float, float, float, int]:
```

```
    util = packer.utilization()
```

```
    v = util["空间利用率_eff"]
```

```
    w = util["载重利用率"]
```

```
    return (v, min(v, w), w, v * w, len(packer.placed))
```

```
def pack_single_truck_bestof(
```

```
    counts: Dict[str, int],
```

```
    catalog: Dict[str, ItemType],
```

```
    truck: TruckType,
```

```
    fragile_fixed: bool = False,
```

```
) -> SurfacePacker:
```

```
    best = None
```

```
    best_key = None
```

```
    for policy in SINGLE_TRUCK_POLICIES:
```

```
        packer = SurfacePacker(truck, catalog, counts, policy,
```

```
    fragile_fixed=fragile_fixed).pack()
```

```
        key = compare_balance_packers(packer)
```

```
        if best_key is None or key > best_key:
```

```
            best_key = key
```

```
            best = packer
```

```
    return best
```

```
def generate_candidate_packers(
```

```
    counts: Dict[str, int],
```

```
    catalog: Dict[str, ItemType],
```

```
    truck: TruckType,
```

```
    fragile_fixed: bool = False,
```

```
    top_k: int = 4,
```

```
) -> List[SurfacePacker]:
```

```
    candidates = []
```

```
    best_by_pattern: Dict[Tuple[int, ...], Tuple[Tuple[float, ...], SurfacePacker]] = {}
```

```
    for policy in MIN_TRUCK_POLICIES:
```

```
        packer = SurfacePacker(truck, catalog, counts, policy,
```

```
    fragile_fixed=fragile_fixed).pack()
```

```
        if len(packer.placed) == 0:
```

```
            continue
```

```
        pattern = counts_tuple(packer.loaded_counts)
```

```
        score = compare_cover_packers(packer)
```

```
        old = best_by_pattern.get(pattern)
```

```
        if old is None or score > old[0]:
```

```

        best_by_pattern[pattern] = (score, packer)

    ranked = sorted(best_by_pattern.values(), key=lambda x: x[0], reverse=True)
    return [packer for _, packer in ranked[:top_k]]

def greedy_pack_all(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    truck: TruckType,
    fragile_fixed: bool = False,
) -> List[SurfacePacker]:
    remaining = {code: counts.get(code, 0) for code in ITEM_ORDER}
    solution: List[SurfacePacker] = []

    while not is_zero_counts(remaining):
        candidates = generate_candidate_packers(remaining, catalog, truck,
        fragile_fixed=fragile_fixed, top_k=1)
        if not candidates:
            raise RuntimeError("当前启发式未能再装入任何货物，请检查参数设置。")
        chosen = candidates[0]
        solution.append(chosen)
        remaining = subtract_counts(remaining, chosen.loaded_counts)

    return solution

def can_finish_with_n_trucks(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    truck: TruckType,
    n: int,
    fragile_fixed: bool = False,
    top_k: int = 4,
    memo: Optional[Dict[Tuple[int, Tuple[int, ...]], Tuple[bool,
Optional[List[SurfacePacker]]]]] = None,
    candidate_cache: Optional[Dict[Tuple[Tuple[int, ...], str],
List[SurfacePacker]]] = None,
) -> Tuple[bool, Optional[List[SurfacePacker]]]:
    if memo is None:
        memo = {}
    if candidate_cache is None:
        candidate_cache = {}

    state = counts_tuple(counts)
    key = (n, state)
    if key in memo:
        return memo[key]

    if all(v == 0 for v in state):
        memo[key] = (True, [])
        return memo[key]

    if n == 0:
        memo[key] = (False, None)
        return memo[key]

    if volume_weight_lower_bound(counts, catalog, truck) > n:
        memo[key] = (False, None)

```

```

        return memo[key]

    ckey = (state, truck.name)
    if ckey not in candidate_cache:
        candidate_cache[ckey] = generate_candidate_packers(
            counts,
            catalog,
            truck,
            fragile_fixed=fragile_fixed,
            top_k=top_k,
        )
    candidates = candidate_cache[ckey]

    for packer in candidates:
        new_counts = subtract_counts(counts, packer.loaded_counts)
        ok, tail = can_finish_with_n_trucks(
            new_counts,
            catalog,
            truck,
            n - 1,
            fragile_fixed=fragile_fixed,
            top_k=top_k,
            memo=memo,
            candidate_cache=candidate_cache,
        )
        if ok:
            memo[key] = (True, [packer] + (tail or []))
            return memo[key]

    memo[key] = (False, None)
    return memo[key]

def solve_min_trucks_single_type(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    truck: TruckType,
    fragile_fixed: bool = False,
    max_search_gap: int = 3,
) -> List[SurfacePacker]:
    """固定一种车型时，求“尽量少的车辆数”。

    说明：
    - 先用贪心得到一个可行上界；
    - 只有当“上界 - 下界”不大时，才启动有限深度搜索压缩车辆数；
    - 对于本题放大10倍后的数据，若 gap 过大，直接返回贪心解更稳健，
      这样可以保证程序在比赛环境中运行时间可控。
    """
    lb = volume_weight_lower_bound(counts, catalog, truck)
    greedy_solution = greedy_pack_all(counts, catalog, truck,
    fragile_fixed=fragile_fixed)
    ub = len(greedy_solution)

    if lb == ub:
        return greedy_solution

    if ub - lb > max_search_gap:
        return greedy_solution

```

```

memo: Dict[Tuple[int, Tuple[int, ...]], Tuple[bool,
Optional[List[SurfacePacker]]]] = {}
candidate_cache: Dict[Tuple[Tuple[int, ...], str], List[SurfacePacker]] = {}

for n in range(lb, ub + 1):
    ok, sol = can_finish_with_n_trucks(
        counts,
        catalog,
        truck,
        n,
        fragile_fixed=fragile_fixed,
        top_k=4,
        memo=memo,
        candidate_cache=candidate_cache,
    )
    if ok and sol is not None:
        return sol

return greedy_solution

# 八、导出与结果整理
def uid_width_for_catalog(catalog: Dict[str, ItemType]) -> int:
    return max(4, max(len(str(catalog[code].quantity)) for code in ITEM_ORDER))

def solution_rows_from_packers(packers: List[SurfacePacker], catalog: Dict[str,
ItemType]) -> List[Dict[str, object]]:
    width = uid_width_for_catalog(catalog)
    next_index = {code: 1 for code in ITEM_ORDER}
    rows: List[Dict[str, object]] = []
    category_name_map = {
        "standard": "标准件",
        "fragile": "易碎件",
        "directional": "定向件",
    }

    for truck_idx, packer in enumerate(packers, start=1):
        pid_to_uid: Dict[int, str] = {}
        for p in packer.placed:
            uid = f"{p.type_code}_{next_index[p.type_code]:0{width}d}"
            next_index[p.type_code] += 1
            pid_to_uid[p.pid] = uid

        for p in packer.placed:
            rows.append(
                {
                    "车辆编号": truck_idx,
                    "车型": packer.truck.name,
                    "货物唯一编号": pid_to_uid[p.pid],
                    "货物类型": p.type_code,
                    "货物类别": category_name_map.get(p.category, p.category),
                    "x坐标_右后到前_cm": p.x,
                    "y坐标_右到左_cm": p.y,
                    "z坐标_下到上_cm": p.z,
                    "放置长度_x方向_cm": p.l,
                }
            )

```

```

        "放置宽度_y方向_cm": p.w,
        "放置高度_z方向_cm": p.h,
        "摆放姿态": p.orientation_name,
        "支撑对象编号": (
            "组合支撑:" + "|".join(pid_to_uid[pid] for pid in
p.support_pids)
            if len(p.support_pids) >= 2
            else (pid_to_uid.get(p.parent_pid, "地板") if p.parent_pid is
not None else "地板")
        ),
        "货物重量_kg": p.weight,
        "货物体积_cm3": p.volume,
    }
)
return rows

def summary_rows(title: str, packers: List[SurfacePacker]) -> List[Dict[str,
object]]:
    rows = []
    for idx, packer in enumerate(packers, start=1):
        util = packer.utilization()
        row = {
            "方案": title,
            "车辆序号": idx,
            "车型": packer.truck.name,
            "装载件数": len(packer.placed),
            "空间利用率_raw": util["空间利用率_raw"],
            "空间利用率_eff": util["空间利用率_eff"],
            "载重利用率": util["载重利用率"],
            "已装货物体积_m3": util["已装货物体积_m3"],
            "已装货物重量_kg": util["已装货物重量_kg"],
        }
        for code in ITEM_ORDER:
            row[f"{code}_数量"] = packer.loaded_counts[code]
        rows.append(row)
    return rows

def print_solution_summary(title: str, packers: List[SurfacePacker]) -> None:
    print("=" * 78)
    print(title)
    print("=" * 78)
    print(f"总车辆数: {len(packers)}")
    total_vol = sum(p.total_volume for p in packers) / 1e6
    total_wt = sum(p.total_weight for p in packers)
    print(f"总装载体积: {total_vol:.3f} m^3")
    print(f"总装载重量: {total_wt:.1f} kg")
    print()

    for i, p in enumerate(packers, start=1):
        util = p.utilization()
        print(f"车辆 {i} | {p.truck.name} | 件数 {len(p.placed)}")
        print(f" 空间利用率(raw): {util['空间利用率_raw']:.4f}")
        print(f" 空间利用率(eff): {util['空间利用率_eff']:.4f}")
        print(f" 载重利用率      : {util['载重利用率']:.4f}")

```

```

print(f" 已装体积      : {util['已装货物体积_m3']:.3f} m^3")
print(f" 已装重量      : {util['已装货物重量_kg']:.1f} kg")
print(
    " 货类计数      : "
    + ", ".join(f"{code}={p.loaded_counts[code]}" for code in ITEM_ORDER)
)
print()

# 九、问题1: 单车型精确最少车辆 + 二次均衡优化 (独立运行)
# 说明:
# - 与原先依赖外部基类的 q1 精确版不同, 这里采用完全自包含实现;
# - G3 的支撑平台统一允许来自标准件 G1 / G2, 符合附件约束;
# - 单车最优装载仍使用前面的多策略择优, 最少车辆部分采用“模式库 + MILP 精确覆盖”。

PATTERN_POLICIES: Tuple[PackingPolicy, ...] = (
    PackingPolicy("cover_volume", "volume", {"standard": 1.8, "directional": 1.0,
"fragile": 0.2}),
    PackingPolicy("cover_balanced", "balanced", {"standard": 2.0, "directional":
1.0, "fragile": 0.0}),
    PackingPolicy("mix_product", "product", {"standard": 1.3, "directional": 1.3,
"fragile": 0.0}),
    PackingPolicy("directional_bias", "volume", {"directional": 2.5, "standard":
0.8, "fragile": 0.3}),
)

CAP_GRID: Tuple[Optional[int], ...] = (None, 30, 50, 100, 150, 200, 300)

_PATTERN_CACHE: Dict[Tuple[str, Tuple[int, ...], bool], List[SurfacePacker]] = {}

def _pattern_subsets() -> Tuple[Tuple[str, ...], ...]:
    subsets: List[Tuple[str, ...]] = []
    for r in (1, 2, 3):
        subsets.extend(combinations(ITEM_ORDER, r))
    subsets.extend(
        [
            ("G1", "G2", "G4", "G5"),
            ("G2", "G3", "G4", "G5"),
            ("G1", "G2", "G3", "G5"),
            ("G1", "G3", "G4"),
            ITEM_ORDER,
        ]
    )
    return tuple(subsets)

PATTERN_SUBSETS: Tuple[Tuple[str, ...], ...] = _pattern_subsets()

def _masked_counts(counts: Dict[str, int], subset: Sequence[str], cap:
Optional[int]) -> Dict[str, int]:
    out: Dict[str, int] = {}
    for code in ITEM_ORDER:
        if code not in subset:
            out[code] = 0
        elif cap is None:
            out[code] = counts[code]
        else:

```

```

        out[code] = min(counts[code], cap)
    return out

def _pattern_key(packer: SurfacePacker) -> Tuple[int, ...]:
    return tuple(packer.loaded_counts[code] for code in ITEM_ORDER)

@lru_cache(maxsize=None)
def _pattern_min_fill_from_key(
    loaded_g1: int,
    loaded_g2: int,
    loaded_g3: int,
    loaded_g4: int,
    loaded_g5: int,
    truck_name: str,
) -> float:
    trucks, catalog = build_problem_data()
    truck = next(t for t in trucks if t.name == truck_name)
    total_vol = 0
    total_wt = 0.0
    for code, qty in zip(ITEM_ORDER, (loaded_g1, loaded_g2, loaded_g3, loaded_g4,
loaded_g5)):
        spec = catalog[code]
        total_vol += qty * spec.volume
        total_wt += qty * spec.weight
    v = total_vol / truck.eff_volume
    w = total_wt / truck.max_weight
    return min(v, w)

def pattern_min_fill(packer: SurfacePacker) -> float:
    key = _pattern_key(packer)
    return _pattern_min_fill_from_key(*key, packer.truck.name)

def pattern_score_tuple(packer: SurfacePacker) -> Tuple[float, float, float, float,
int, int]:
    util = packer.utilization()
    v = util["空间利用率_eff"]
    w = util["载重利用率"]
    mix = sum(1 for code in ITEM_ORDER if packer.loaded_counts[code] > 0)
    return (
        min(v, w),
        (v + w) / 2.0,
        v * w,
        v,
        mix,
        len(packer.placed),
    )

def pattern_secondary_score(packer: SurfacePacker) -> float:
    util = packer.utilization()
    v = util["空间利用率_eff"]
    w = util["载重利用率"]
    mix = sum(1 for code in ITEM_ORDER if packer.loaded_counts[code] > 0)
    return 1000.0 * min(v, w) + 250.0 * (v * w) + 20.0 * mix + 0.1 * len(packer.placed)

```

```

def sort_solution_by_fullness(packers: List[SurfacePacker]) ->
List[SurfacePacker]:
    return sorted(packers, key=pattern_score_tuple, reverse=True)

def generate_pattern_library(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    truck: TruckType,
    fragile_fixed: bool = False,
) -> List[SurfacePacker]:
    cache_key = (truck.name, counts_tuple(counts), fragile_fixed)
    if cache_key in _PATTERN_CACHE:
        return _PATTERN_CACHE[cache_key]

    best_by_pattern: Dict[Tuple[int, ...], SurfacePacker] = {}

    greedy_solution = greedy_pack_all(counts, catalog, truck,
    fragile_fixed=fragile_fixed)
    for packer in greedy_solution:
        best_by_pattern[_pattern_key(packer)] = packer

    for subset in PATTERN_SUBSETS:
        for cap in CAP_GRID:
            masked = _masked_counts(counts, subset, cap)
            if not any(masked.values()):
                continue
            for policy in PATTERN_POLICIES:
                packer = SurfacePacker(truck, catalog, masked, policy,
    fragile_fixed=fragile_fixed).pack()
                if not packer.placed:
                    continue
                key = _pattern_key(packer)
                old = best_by_pattern.get(key)
                if old is None or pattern_score_tuple(packer) >
    pattern_score_tuple(old):
                    best_by_pattern[key] = packer

    library = sorted(best_by_pattern.values(), key=pattern_score_tuple,
    reverse=True)
    _PATTERN_CACHE[cache_key] = library
    return library

def _build_count_matrix(packers: List[SurfacePacker]) -> np.ndarray:
    return np.array([[packer.loaded_counts[code] for packer in packers] for code in
    ITEM_ORDER], dtype=float)

def _target_vector(counts: Dict[str, int]) -> np.ndarray:
    return np.array([counts[code] for code in ITEM_ORDER], dtype=float)

def _run_milp_exact_cover(
    counts: Dict[str, int],
    packers: List[SurfacePacker],
    objective: np.ndarray,

```

```

    exact_truck_count: Optional[int] = None,
    time_limit: float = 60.0,
):
    if not packers:
        return None

    matrix = _build_count_matrix(packers)
    target = _target_vector(counts)

    constraints: List[LinearConstraint] = [LinearConstraint(matrix, target,
target)]
    if exact_truck_count is not None:
        ones = np.ones((1, len(packers)), dtype=float)
        exact = np.array([float(exact_truck_count)])
        constraints.append(LinearConstraint(ones, exact, exact))

    bounds = Bounds(lb=np.zeros(len(packers)), ub=np.full(len(packers), np.inf))
    integrality = np.ones(len(packers), dtype=int)
    return milp(
        c=objective,
        constraints=constraints,
        integrality=integrality,
        bounds=bounds,
        options={"time_limit": float(time_limit)},
    )

def _extract_integer_coeffs(result) -> Optional[np.ndarray]:
    if result is None or result.x is None:
        return None
    coeffs = np rint(result.x).astype(int)
    if np.max(np.abs(result.x - coeffs)) > 1e-5:
        return None
    return coeffs

def _coeffs_to_solution(packers: List[SurfacePacker], coeffs: np.ndarray) ->
List[SurfacePacker]:
    out: List[SurfacePacker] = []
    for idx, times in enumerate(coeffs.tolist()):
        if times <= 0:
            continue
        for _ in range(times):
            out.append(copy.deepcopy(packers[idx]))
    return sort_solution_by_fullness(out)

def _solution_counts(packers: List[SurfacePacker]) -> Dict[str, int]:
    acc = {code: 0 for code in ITEM_ORDER}
    for packer in packers:
        for code in ITEM_ORDER:
            acc[code] += packer.loaded_counts[code]
    return acc

def _solution_matches_target(packers: List[SurfacePacker], counts: Dict[str, int])
-> bool:
    return _solution_counts(packers) == {code: counts[code] for code in ITEM_ORDER}

```

```

def solve_min_truck_count_via_mip(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    truck: TruckType,
    fragile_fixed: bool = False,
) -> Tuple[int, List[SurfacePacker], List[SurfacePacker]]:
    library = generate_pattern_library(counts, catalog, truck,
    fragile_fixed=fragile_fixed)
    result = _run_milp_exact_cover(
        counts,
        library,
        objective=np.ones(len(library), dtype=float),
        exact_truck_count=None,
        time_limit=90.0,
    )
    coeffs = _extract_integer_coeffs(result)
    if coeffs is None:
        fallback = greedy_pack_all(counts, catalog, truck,
    fragile_fixed=fragile_fixed)
        return len(fallback), sort_solution_by_fullness(fallback), library

    solution = _coeffs_to_solution(library, coeffs)
    if not _solution_matches_target(solution, counts):
        fallback = greedy_pack_all(counts, catalog, truck,
    fragile_fixed=fragile_fixed)
        return len(fallback), sort_solution_by_fullness(fallback), library

    return len(solution), solution, library

def _highest_feasible_fill_threshold(
    counts: Dict[str, int],
    library: List[SurfacePacker],
    truck_count: int,
) -> float:
    values = sorted({pattern_min_fill(packer) for packer in library})
    if not values:
        return 0.0

    lo = 0
    hi = len(values) - 1
    best = 0.0
    while lo <= hi:
        mid = (lo + hi) // 2
        threshold = values[mid]
        filtered = [packer for packer in library if pattern_min_fill(packer) + 1e-9
    >= threshold]
        result = _run_milp_exact_cover(
            counts,
            filtered,
            objective=np.zeros(len(filtered), dtype=float),
            exact_truck_count=truck_count,
            time_limit=20.0,
        )
        coeffs = _extract_integer_coeffs(result)
        if coeffs is not None:
            solution = _coeffs_to_solution(filtered, coeffs)
            if _solution_matches_target(solution, counts):

```

```

        best = threshold
        lo = mid + 1
        continue
    hi = mid - 1
    return best

def optimize_balance_at_fixed_truck_count(
    counts: Dict[str, int],
    library: List[SurfacePacker],
    truck_count: int,
) -> List[SurfacePacker]:
    threshold = _highest_feasible_fill_threshold(counts, library, truck_count)
    filtered = [packer for packer in library if pattern_min_fill(packer) + 1e-9 >=
threshold]
    if not filtered:
        filtered = library

    objective = -np.array([pattern_secondary_score(packer) for packer in filtered],
dtype=float)
    result = _run_milp_exact_cover(
        counts,
        filtered,
        objective=objective,
        exact_truck_count=truck_count,
        time_limit=60.0,
    )
    coeffs = _extract_integer_coeffs(result)
    if coeffs is not None:
        solution = _coeffs_to_solution(filtered, coeffs)
        if _solution_matches_target(solution, counts):
            return solution

    fallback_result = _run_milp_exact_cover(
        counts,
        library,
        objective=-np.array([pattern_secondary_score(packer) for packer in
library], dtype=float),
        exact_truck_count=truck_count,
        time_limit=60.0,
    )
    fallback_coeffs = _extract_integer_coeffs(fallback_result)
    if fallback_coeffs is not None:
        fallback_solution = _coeffs_to_solution(library, fallback_coeffs)
        if _solution_matches_target(fallback_solution, counts):
            return fallback_solution

    return []

def solve_min_trucks_single_type(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    truck: TruckType,
    fragile_fixed: bool = False,
) -> List[SurfacePacker]:
    baseline = sort_solution_by_fullness(greedy_pack_all(counts, catalog, truck,
fragile_fixed=fragile_fixed))
    best_count, stage1_solution, library = solve_min_truck_count_via_mip(counts,

```

```

catalog, truck, fragile_fixed=fragile_fixed)
    balanced = optimize_balance_at_fixed_truck_count(counts, library, best_count)

    if balanced:
        return sort_solution_by_fullness(balanced)
    if stage1_solution:
        return sort_solution_by_fullness(stage1_solution)
    return baseline

def solve_question1(fragile_fixed: bool = False) -> Dict[str, object]:
    trucks, catalog = build_problem_data()
    counts = counts_from_catalog(catalog)

    truck1, truck2 = trucks

    best_truck1 = pack_single_truck_bestof(counts, catalog, truck1,
    fragile_fixed=fragile_fixed)
    best_truck2 = pack_single_truck_bestof(counts, catalog, truck2,
    fragile_fixed=fragile_fixed)

    min_solution_truck1 = solve_min_trucks_single_type(counts, catalog, truck1,
    fragile_fixed=fragile_fixed)
    min_solution_truck2 = solve_min_trucks_single_type(counts, catalog, truck2,
    fragile_fixed=fragile_fixed)

    return {
        "catalog": catalog,
        "single_truck_best": {
            truck1.name: [best_truck1],
            truck2.name: [best_truck2],
        },
        "min_trucks_single_type": {
            truck1.name: min_solution_truck1,
            truck2.name: min_solution_truck2,
        },
    }

if __name__ == "__main__":
    result = solve_question1(fragile_fixed=False)
    print_solution_summary("Q1(1) 车型1单车最优装载", result["single_truck_best"]["
车型1"])
    print_solution_summary("Q1(1) 车型2单车最优装载", result["single_truck_best"]["
车型2"])
    print_solution_summary("Q1(2) 仅用车型1时车辆数最少方案",
result["min_trucks_single_type"]["车型1"])
    print_solution_summary("Q1(2) 仅用车型2时车辆数最少方案",
result["min_trucks_single_type"]["车型2"])

```

附录二 问题一_输出脚本.py

介绍：用于输出问题一结果，在求解器算法基础上生成装载明细、利用率统计及三维装载可视化图。

```
from __future__ import annotations

import csv
import json
import os
from typing import Dict, List

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

import 问题一_求解器 as solver

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
OUT_DIR = os.path.join(BASE_DIR, '问题一摆放图及空间载重利用率数据')
PNG_DIR = os.path.join(OUT_DIR, '货车图片')
os.makedirs(OUT_DIR, exist_ok=True)
os.makedirs(PNG_DIR, exist_ok=True)

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'Noto Sans CJK SC',
                                   'Arial Unicode MS', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

TRUCK_NAME_ASCII = {
    '车型 1': '车型 1',
    '车型 2': '车型 2',
}

BOX_COLORS = {
    'G1': '#4C78A8',
    'G2': '#F58518',
    'G3': '#E45756',
    'G4': '#54A24B',
    'G5': '#B279A2',
}

PAPER_DPI = 220
FIGSIZE = (13, 9)

def write_csv(path: str, rows: List[Dict[str, object]]) -> None:
    if not rows:
        raise ValueError(f'没有可写入的数据: {path}')
    fieldnames = list(rows[0].keys())
    with open(path, 'w', newline='', encoding='utf-8-sig') as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)

def write_summary_json(path: str, result: Dict[str, object]) -> None:
    out = {
```

```

        '单车最优装载': {},
        '单一车型车辆数最少方案': {},
    }

    for truck_name, packers in result['single_truck_best'].items():
        out['单车最优装载'][truck_name] = [
            {
                '利用率': p.utilization(),
                '装载数量': p.loaded_counts,
                '装载件数': len(p.placed),
            }
            for p in packers
        ]

    for truck_name, packers in result['min_trucks_single_type'].items():
        out['单一车型车辆数最少方案'][truck_name] = [
            {
                '利用率': p.utilization(),
                '装载数量': p.loaded_counts,
                '装载件数': len(p.placed),
            }
            for p in packers
        ]

    with open(path, 'w', encoding='utf-8') as f:
        json.dump(out, f, ensure_ascii=False, indent=2)

# =====
# 3D PNG 渲染模块（仅本地 matplotlib, 不依赖网页图片）
# =====

def cuboid_faces(x: int, y: int, z: int, dx: int, dy: int, dz: int):
    p000 = (x, y, z)
    p100 = (x + dx, y, z)
    p110 = (x + dx, y + dy, z)
    p010 = (x, y + dy, z)
    p001 = (x, y, z + dz)
    p101 = (x + dx, y, z + dz)
    p111 = (x + dx, y + dy, z + dz)
    p011 = (x, y + dy, z + dz)
    return [
        [p000, p100, p110, p010],
        [p001, p101, p111, p011],
        [p000, p100, p101, p001],
        [p010, p110, p111, p011],
        [p000, p010, p011, p001],
        [p100, p110, p111, p101],
    ]

def draw_truck_wireframe(ax, L: int, W: int, H: int) -> None:
    corners = [
        (0, 0, 0), (L, 0, 0), (L, W, 0), (0, W, 0),
        (0, 0, H), (L, 0, H), (L, W, H), (0, W, H),
    ]
    edges = [
        (0, 1), (1, 2), (2, 3), (3, 0),
        (4, 5), (5, 6), (6, 7), (7, 4),
        (0, 4), (1, 5), (2, 6), (3, 7),
    ]

```

```

        for i, j in edges:
            xs = [corners[i][0], corners[j][0]]
            ys = [corners[i][1], corners[j][1]]
            zs = [corners[i][2], corners[j][2]]
            ax.plot(xs, ys, zs, color='black', linewidth=0.9)

def style_3d_axes(ax) -> None:
    ax.grid(False)
    for axis in (ax.xaxis, ax.yaxis, ax.zaxis):
        try:
            axis.pane.set_facecolor((1.0, 1.0, 1.0, 0.0))
            axis.pane.set_edgecolor((1.0, 1.0, 1.0, 0.0))
        except Exception:
            pass
    try:
        ax.xaxis._axinfo['grid']['linewidth'] = 0.0
        ax.yaxis._axinfo['grid']['linewidth'] = 0.0
        ax.zaxis._axinfo['grid']['linewidth'] = 0.0
    except Exception:
        pass

def render_truck_png(packer: solver.SurfacePacker, out_png: str, title: str) -> Dict[str, object]:
    fig = plt.figure(figsize=FIGSIZE, dpi=PAPER_DPI)
    ax = fig.add_subplot(111, projection='3d')

    for p in sorted(packer.placed, key=lambda box: (box.z, box.x, box.y, box.h)):
        faces = cuboid_faces(p.x, p.y, p.z, p.l, p.w, p.h)
        poly = Poly3DCollection(
            faces,
            facecolors=BOX_COLORS.get(p.type_code, '#999999'),
            edgecolors='k',
            linewidths=0.18,
            alpha=0.68,
        )
        ax.add_collection3d(poly)

    L, W, H = packer.truck.L, packer.truck.W, packer.truck.effective_H
    draw_truck_wireframe(ax, L, W, H)
    style_3d_axes(ax)

    ax.set_xlim(0, L)
    ax.set_ylim(0, W)
    ax.set_zlim(0, H)
    ax.set_box_aspect((L, W, H))
    ax.set_xlabel('x 轴: 从右后到前方（厘米）', labelpad=8)
    ax.set_ylabel('y 轴: 从右侧到左侧（厘米）', labelpad=8)
    ax.set_zlabel('z 轴: 从下到上（厘米）', labelpad=8)
    ax.view_init(elev=23, azim=-58)

    util = packer.utilization()
    truck_label = TRUCK_NAME_ASCII.get(packer.truck.name, packer.truck.name)
    counts_text = ' '.join(f'{code}={packer.loaded_counts[code]}' for code in
solver.ITEM_ORDER)
    info_text = (
        f"车型: {truck_label}\n"
        f"装载件数: {len(packer.placed)}\n"
        f"有效空间利用率: {util['空间利用率_eff']:.4f}\n"
        f"载重利用率: {util['载重利用率']:.4f}\n"

```

```

        f"已装体积: {util['已装货物体积_m3']:.3f} 立方米\n"
        f"已装重量: {util['已装货物重量_kg']:.1f} 千克\n"
        f"各类货物数量: {counts_text}"
    )

    legend_handles = [
        Patch(facecolor=BOX_COLORS[code], edgecolor='k', label=code, alpha=0.68)
        for code in solver.ITEM_ORDER
        if packer.loaded_counts.get(code, 0) > 0
    ]
    if legend_handles:
        fig.legend(handles=legend_handles, loc='upper right',
bbox_to_anchor=(0.985, 0.88), frameon=True)

    fig.text(
        0.81,
        0.43,
        info_text,
        ha='left',
        va='top',
        fontsize=10.5,
        bbox=dict(boxstyle='round,pad=0.45', facecolor='white',
edgecolor='#666666', alpha=0.95),
    )

    fig.suptitle(title, y=0.97, fontsize=14)
    plt.tight_layout(rect=(0.0, 0.0, 0.79, 0.95))
    fig.savefig(out_png, bbox_inches='tight')
    plt.close(fig)

    return {
        '车型': packer.truck.name,
        '装载件数': len(packer.placed),
        '空间利用率_eff': util['空间利用率_eff'],
        '载重利用率': util['载重利用率'],
        '已装货物体积_m3': util['已装货物体积_m3'],
        '已装货物重量_kg': util['已装货物重量_kg'],
    }

def export_pngs_for_scheme(packers: List[solver.SurfacePacker], scheme_slug: str,
scheme_title: str) -> List[Dict[str, object]]:
    scheme_dir = os.path.join(PNG_DIR, scheme_slug)
    os.makedirs(scheme_dir, exist_ok=True)

    manifest_rows: List[Dict[str, object]] = []
    for i, packer in enumerate(packers, start=1):
        filename = f'{scheme_slug}_第{i:02d}辆车.png'
        abs_path = os.path.join(scheme_dir, filename)
        title = f'{scheme_title} - 第{i}辆车'
        stats = render_truck_png(packer, abs_path, title)
        manifest_rows.append({
            '方案标识': scheme_slug,
            '方案名称': scheme_title,
            '车辆序号': i,
            '图片文件名': filename,
            '图片相对路径': os.path.relpath(abs_path, OUT_DIR),
            **stats,
        })
    return manifest_rows

```

```

if __name__ == '__main__':
    result = solver.solve_question1(fragile_fixed=False)
    catalog = result['catalog']

    single_t1 = result['single_truck_best']['车型 1']
    single_t2 = result['single_truck_best']['车型 2']
    min_t1 = result['min_trucks_single_type']['车型 1']
    min_t2 = result['min_trucks_single_type']['车型 2']

    # 逐件方案导出
    write_csv(
        os.path.join(OUT_DIR, '问题 1_车型 1 单车最优装载.csv'),
        solver.solution_rows_from_packers(single_t1, catalog),
    )
    write_csv(
        os.path.join(OUT_DIR, '问题 1_车型 2 单车最优装载.csv'),
        solver.solution_rows_from_packers(single_t2, catalog),
    )
    write_csv(
        os.path.join(OUT_DIR, '问题 1_仅用车型 1 车辆数最少方案.csv'),
        solver.solution_rows_from_packers(min_t1, catalog),
    )
    write_csv(
        os.path.join(OUT_DIR, '问题 1_仅用车型 2 车辆数最少方案.csv'),
        solver.solution_rows_from_packers(min_t2, catalog),
    )

    # 汇总表导出
    summary_rows: List[Dict[str, object]] = []
    summary_rows.extend(solver.summary_rows('问题 1_车型 1 单车最优装载',
single_t1))
    summary_rows.extend(solver.summary_rows('问题 1_车型 2 单车最优装载',
single_t2))
    summary_rows.extend(solver.summary_rows('问题 1_仅用车型 1 时车辆数最少方案',
min_t1))
    summary_rows.extend(solver.summary_rows('问题 1_仅用车型 2 时车辆数最少方案',
min_t2))
    write_csv(os.path.join(OUT_DIR, '问题 1_汇总表.csv'), summary_rows)

    write_summary_json(os.path.join(OUT_DIR, '问题 1_汇总信息.json'), result)

    # 新增: 逐车论文用 PNG 导出
    png_manifest: List[Dict[str, object]] = []
    png_manifest.extend(export_pngs_for_scheme(single_t1, '车型 1 单车最优装载', '
问题 1 (1) 车型 1 单车最优装载'))
    png_manifest.extend(export_pngs_for_scheme(single_t2, '车型 2 单车最优装载', '
问题 1 (1) 车型 2 单车最优装载'))
    png_manifest.extend(export_pngs_for_scheme(min_t1, '仅用车型 1 车辆数最少方案',
'问题 1 (2) 仅用车型 1 时车辆数最少方案'))
    png_manifest.extend(export_pngs_for_scheme(min_t2, '仅用车型 2 车辆数最少方案',
'问题 1 (2) 仅用车型 2 时车辆数最少方案'))
    write_csv(os.path.join(OUT_DIR, '问题 1_图片清单.csv'), png_manifest)

    print('已生成更新版输出文件: ')
    for name in [
        '问题 1_车型 1 单车最优装载.csv',
        '问题 1_车型 2 单车最优装载.csv',
        '问题 1_仅用车型 1 车辆数最少方案.csv',
        '问题 1_仅用车型 2 车辆数最少方案.csv',
        '问题 1_汇总表.csv',

```

```
        '问题 1_汇总信息.json',  
        '问题 1_图片清单.csv',  
    ]:  
        print(f'- {os.path.join(OUT_DIR, name)}')  
print(f'- 货车图片目录: {PNG_DIR}')
```

附录四 问题二_求解器.py

介绍：用于求解问题二，在三维装箱启发式算法基础上，结合多车型组合决策与优化方法，求解联合运输下的车辆配置与成本最优方案。

```
from __future__ import annotations

import copy
import math

import csv
from pathlib import Path
from typing import Tuple, List, Dict

import numpy as np
from scipy.optimize import Bounds, LinearConstraint, milp
from dataclasses import dataclass, field
from functools import lru_cache
from itertools import combinations, permutations
from typing import Dict, Iterable, List, Optional, Sequence, Tuple

# 全局参数
SAFETY_GAP_CM = 3
BEARING_LIMIT_KG_PER_M2 = 500.0
BEARING_LIMIT_KG_PER_CM2 = BEARING_LIMIT_KG_PER_M2 / 10000.0 # 0.05 kg / cm^2
MAX_SURFACES = 400
CANDIDATE_SURFACE_LIMIT = 140
DIRECTIONAL_PROJECTION_RATIO = 1.0 # 保守处理：定向件必须整底完全投影在支撑面内
ITEM_ORDER = ("G1", "G2", "G3", "G4", "G5")
FRAGILE_CODE = "G3"
STANDARD_SUPPORT_BUILDERS = ("G1", "G2")
FRAGILE_PLATFORM_LOOKAHEAD_DEPTH = 4
FRAGILE_TOP_LINEAR_BONUS = 4.0
FRAGILE_TOP_QUADRATIC_BONUS = 6.0
FRAGILE_NEAR_TOP_BONUS = 2.0
FRAGILE_NEAR_TOP_MARGIN_CM = 20
COMPOSITE_SURFACE_PAIR_LIMIT = 120

# 一、数据结构

@dataclass(frozen=True)
class TruckType:
    name: str
    L: int
    W: int
    H: int
    max_weight: float
    trip_cost: float

    @property
    def effective_H(self) -> int:
        return self.H - SAFETY_GAP_CM

    @property
    def raw_volume(self) -> int:
        return self.L * self.W * self.H

    @property
    def eff_volume(self) -> int:
        return self.L * self.W * self.effective_H

@dataclass(frozen=True)
```

```

class ItemType:
    code: str
    category: str # standard / fragile / directional
    dims: Tuple[int, int, int]
    weight: float
    quantity: int

    @property
    def volume(self) -> int:
        a, b, c = self.dims
        return a * b * c

@dataclass
class PlacedBox:
    pid: int
    type_code: str
    category: str
    weight: float
    x: int
    y: int
    z: int
    l: int
    w: int
    h: int
    orientation_name: str
    parent_pid: Optional[int] = None
    support_pids: Tuple[int, ...] = ()
    load_on_top: float = 0.0

    @property
    def volume(self) -> int:
        return self.l * self.w * self.h

    @property
    def top_area_cm2(self) -> int:
        return self.l * self.w

    @property
    def bearing_capacity(self) -> float:
        if self.category == "fragile":
            return 0.0
        return BEARING_LIMIT_KG_PER_CM2 * self.top_area_cm2

@dataclass(frozen=True)
class Surface:
    x: int
    y: int
    z: int
    L: int
    W: int
    parent_pid: Optional[int] = None
    support_pids: Tuple[int, ...] = ()
    source_surfaces: Tuple["Surface", ...] = ()

    @property
    def area(self) -> int:
        return self.L * self.W

@dataclass(frozen=True)
class PackingPolicy:
    name: str
    mode: str # balanced / volume / weight / product
    category_bonus: Dict[str, float] = field(default_factory=dict)

```

二、基础数据（从附件中提取）

```
def build_problem_data() -> Tuple[List[TruckType], Dict[str, ItemType]]:
    def _find_file(filename: str) -> Path:
        candidates = [
            Path(filename),
            Path.cwd() / filename,
            Path("/mnt/data") / filename,    # 当前对话附件环境
        ]
        for p in candidates:
            if p.exists():
                return p
        raise FileNotFoundError(f"未找到附件文件: {filename}")

    truck_file = _find_file("附件1_车型数据.csv")
    cargo_file = _find_file("附件1_货物数据.csv")

    trucks = []
    with open(truck_file, "r", encoding="gbk", newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            # 保持原来名称风格: 车型1、车型2
            truck_name = row["车型名称"].split("_")[0] if row["车型名称"] else f"车型{row['车型编号']}"
            trucks.append(
                TruckType(
                    truck_name,
                    int(row["长(cm)"]),
                    int(row["宽(cm)"]),
                    int(row["高(cm)"]),
                    int(row["额定载重(kg)"]),
                    float(row["单次运输成本(元)"]),
                )
            )

    type_mapping = {
        "标准件": "standard",
        "易碎件": "fragile",
        "定向件": "directional",
    }

    item_types = {}
    with open(cargo_file, "r", encoding="gbk", newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            item_id = row["货物编号"]
            item_types[item_id] = ItemType(
                item_id,
                type_mapping[row["类型"]],
                (
                    int(row["长(cm)"]),
                    int(row["宽(cm)"]),
                    int(row["高(cm)"]),
                ),
                float(row["单重(kg)"]),
                int(row["数量"]),
            )

    return trucks, item_types
```

三、姿态枚举与计数工具

@lru_cache(maxsize=None)

```
def unique_orientations(dims: Tuple[int, int, int], category: str, fragile_fixed: bool = False) -> Tuple[Tuple[int, int, int, str], ...]:
```

```

    if category == "directional":
        return ((dims[0], dims[1], dims[2], f"{dims[0]}x{dims[1]}x{dims[2]}"),)

    if category == "fragile" and fragile_fixed:
        return ((dims[0], dims[1], dims[2], f"{dims[0]}x{dims[1]}x{dims[2]}"),)

    out = []
    for p in set(permutations(dims, 3)):
        out.append((p[0], p[1], p[2], f"{p[0]}x{p[1]}x{p[2]}"))

    # 优先大底面、低高度
    out.sort(key=lambda x: (-(x[0] * x[1]), x[2], -max(x[0], x[1]), -min(x[0], x[1])))
    return tuple(out)

def counts_from_catalog(catalog: Dict[str, ItemType]) -> Dict[str, int]:
    return {code: catalog[code].quantity for code in ITEM_ORDER}

def counts_tuple(counts: Dict[str, int]) -> Tuple[int, ...]:
    return tuple(counts.get(code, 0) for code in ITEM_ORDER)

def counts_dict(counts_t: Tuple[int, ...]) -> Dict[str, int]:
    return {code: counts_t[i] for i, code in enumerate(ITEM_ORDER)}

def subtract_counts(a: Dict[str, int], b: Dict[str, int]) -> Dict[str, int]:
    out = {}
    for code in ITEM_ORDER:
        v = a.get(code, 0) - b.get(code, 0)
        if v < 0:
            raise ValueError(f"计数相减出现负数: {code}")
        out[code] = v
    return out

def is_zero_counts(counts: Dict[str, int]) -> bool:
    return all(counts.get(code, 0) == 0 for code in ITEM_ORDER)

# 四、下界：最少车辆数的理论下界
def total_remaining_volume(counts: Dict[str, int], catalog: Dict[str, ItemType]) -> int:
    return sum(counts[code] * catalog[code].volume for code in ITEM_ORDER)

def total_remaining_weight(counts: Dict[str, int], catalog: Dict[str, ItemType]) -> float:
    return sum(counts[code] * catalog[code].weight for code in ITEM_ORDER)

def volume_weight_lower_bound(counts: Dict[str, int], catalog: Dict[str, ItemType],
truck: TruckType) -> int:
    total_vol = total_remaining_volume(counts, catalog)
    total_wt = total_remaining_weight(counts, catalog)
    return max(
        math.ceil(total_vol / truck.eff_volume),
        math.ceil(total_wt / truck.max_weight),
    )

# 五、核心装箱器：表面分割 + 货类计数驱动
class SurfacePacker:
    def __init__(
        self,
        truck: TruckType,

```

```

        catalog: Dict[str, ItemType],
        remaining_counts: Dict[str, int],
        policy: PackingPolicy,
        fragile_fixed: bool = False,
        projection_ratio: float = DIRECTIONAL_PROJECTION_RATIO,
    ):
        self.truck = truck
        self.catalog = catalog
        self.policy = policy
        self.fragile_fixed = fragile_fixed
        self.projection_ratio = projection_ratio

        self.remaining_counts = {code: remaining_counts.get(code, 0) for code in
ITEM_ORDER}
        self.loaded_counts = {code: 0 for code in ITEM_ORDER}

        self.placed: List[PlacedBox] = []
        self.by_pid: Dict[int, PlacedBox] = {}
        self.next_pid = 1

        self.total_weight = 0.0
        self.total_volume = 0

        # 初始唯一表面: 车厢底板
        self.surfaces: List[Surface] = [Surface(0, 0, 0, truck.L, truck.W, None)]

        # 同一装箱状态下, 大量候选会反复查询“该表面还能否继续抬高成 G3 平台”。
        # 用轻量缓存避免在每次打分时重复枚举。
        self._top_fragile_fit_cache: Dict[Tuple[int, int, int], bool] = {}
        self._fragile_final_top_cache: Dict[Tuple[int, int, int], int] = {}
        self._future_fragile_platform_cache: Dict[Tuple[int, int, int, int, int,
Optional[int], Tuple[int, ...]], int] = {}
        self._recursive_fragile_top_cache: Dict[Tuple[int, int, int, int], int] = {}

        # 基础几何与稳定性检查
        def rectangle_overlap_area(
            self,
            ax: int,
            ay: int,
            al: int,
            aw: int,
            bx: int,
            by: int,
            bl: int,
            bw: int,
        ) -> int:
            x_overlap = max(0, min(ax + al, bx + bl) - max(ax, bx))
            y_overlap = max(0, min(ay + aw, by + bw) - max(ay, by))
            return x_overlap * y_overlap

        def centroid_inside_parent(self, x: int, y: int, l: int, w: int, parent: PlacedBox)
-> bool:
            cx = x + l / 2.0
            cy = y + w / 2.0
            return (parent.x <= cx <= parent.x + parent.l) and (parent.y <= cy <= parent.y
+ parent.w)

        def projection_ratio_ok(self, spec: ItemType, x: int, y: int, l: int, w: int, parent:
Optional[PlacedBox]) -> bool:
            if parent is None:
                return True

            overlap = self.rectangle_overlap_area(x, y, l, w, parent.x, parent.y, parent.l,
parent.w)
            ratio = overlap / (l * w)

        # 定向件新增“投影面积约束”: 采用保守的“整底完整支撑”策略

```

```

        if spec.category == "directional" and ratio + 1e-12 < self.projection_ratio:
            return False

        # 若支撑体本身是定向件，则上层货物同样要求完整落在其投影范围内，避免悬挑
        if parent.category == "directional" and ratio + 1e-12 < self.projection_ratio:
            return False

        return True

    def surface_support_boxes(self, surface: Surface) -> List[PlacedBox]:
        support_boxes: List[PlacedBox] = []
        if surface.support_pids:
            for pid in surface.support_pids:
                box = self.by_pid.get(pid)
                if box is not None:
                    support_boxes.append(box)
            return support_boxes
        if surface.parent_pid is not None:
            parent = self.by_pid.get(surface.parent_pid)
            if parent is not None:
                support_boxes.append(parent)
        return support_boxes

    def surface_is_standard_support(self, surface: Surface) -> bool:
        boxes = self.surface_support_boxes(surface)
        return bool(boxes) and all(box.category == "standard" for box in boxes)

    def compute_support_loads(self, item_weight: float, surface: Surface, x: int, y:
int, l: int, w: int) -> Optional[Dict[int, float]]:
        boxes = self.surface_support_boxes(surface)
        if not boxes:
            return {}
        if len(boxes) == 1:
            return {boxes[0].pid: item_weight}

        overlaps: Dict[int, int] = {}
        total_overlap = 0
        for box in boxes:
            ov = self.rectangle_overlap_area(x, y, l, w, box.x, box.y, box.l, box.w)
            if ov > 0:
                overlaps[box.pid] = ov
                total_overlap += ov
        if total_overlap < l * w:
            return None
        return {pid: item_weight * ov / total_overlap for pid, ov in overlaps.items()}

    def can_support(self, spec: ItemType, surface: Surface, l: int, w: int) -> bool:
        support_boxes = self.surface_support_boxes(surface)

        # 落地总是允许
        if not support_boxes:
            return True

        # 任一支撑箱为易碎件都不允许
        if any(box.category == "fragile" for box in support_boxes):
            return False

        # 组合面仅给 G3 使用；且其支撑箱必须全部为标准件
        if len(support_boxes) >= 2:
            if spec.category != "fragile":
                return False
            if not all(box.category == "standard" for box in support_boxes):
                return False
        else:
            parent = support_boxes[0]
            if spec.category == "fragile" and parent.category != "standard":
                return False

```

```

        if not self.centroid_inside_parent(surface.x, surface.y, l, w, parent):
            return False
        if not self.projection_ratio_ok(spec, surface.x, surface.y, l, w, parent):
            return False

        load_map = self.compute_support_loads(spec.weight, surface, surface.x,
        surface.y, l, w)
        if load_map is None:
            return False

        # 承重链约束: 组合面按覆盖面积分摊重量
        for pid, delta in load_map.items():
            cur = self.by_pid.get(pid)
            while cur is not None:
                if cur.load_on_top + delta > cur.bearing_capacity + 1e-12:
                    return False
                cur = self.by_pid.get(cur.parent_pid)
        return True

    def apply_load_update(self, load_map: Dict[int, float]) -> None:
        for pid, delta in load_map.items():
            cur = self.by_pid.get(pid)
            while cur is not None:
                cur.load_on_top += delta
                cur = self.by_pid.get(cur.parent_pid)

    def reset_state_caches(self) -> None:
        self._top_fragile_fit_cache.clear()
        self._fragile_final_top_cache.clear()
        self._future_fragile_platform_cache.clear()
        self._recursive_fragile_top_cache.clear()

    def surface_cache_key(self, surface: Surface) -> Tuple[int, int, int, int, int,
Optional[int], Tuple[int, ...]]:
        return (surface.x, surface.y, surface.z, surface.L, surface.W,
        surface.parent_pid, tuple(surface.support_pids))

    def remaining_fragile_exists(self) -> bool:
        return self.remaining_counts.get(FRAGILE_CODE, 0) > 0

    def remaining_standard_support_builder_exists(self) -> bool:
        return any(self.remaining_counts.get(code, 0) > 0 for code in
        STANDARD_SUPPORT_BUILDERS)

    def best_fragile_final_top_z_for_top(self, L: int, W: int, top_z: int) -> int:
        key = (L, W, top_z)
        if key in self._fragile_final_top_cache:
            return self._fragile_final_top_cache[key]

        fragile = self.catalog[FRAGILE_CODE]
        best_final_top = -1
        for fl, fw, fh, _ in unique_orientations(fragile.dims, fragile.category,
        self.fragile_fixed):
            if fl <= L and fw <= W and top_z + fh <= self.truck.effective_H:
                best_final_top = max(best_final_top, top_z + fh)
        self._fragile_final_top_cache[key] = best_final_top
        return best_final_top

    def _best_recursive_fragile_top_z_from_top(self, L: int, W: int, top_z: int, depth:
int) -> int:
        key = (L, W, top_z, depth)
        if key in self._recursive_fragile_top_cache:
            return self._recursive_fragile_top_cache[key]

        best = self.best_fragile_final_top_z_for_top(L, W, top_z)
        if depth <= 0 or top_z >= self.truck.effective_H or not
        self.remaining_standard_support_builder_exists():

```

```

        self._recursive_fragile_top_cache[key] = best
        return best

    for code in STANDARD_SUPPORT_BUILDERS:
        if self.remaining_counts.get(code, 0) <= 0:
            continue
        spec = self.catalog[code]
        for nl, nw, nh, _ in unique_orientations(spec.dims, spec.category,
self.fragile_fixed):
            if nl > L or nw > W:
                continue
            if top_z + nh > self.truck.effective_H:
                continue
            child = self._best_recursive_fragile_top_z_from_top(nl, nw, top_z + nh,
depth - 1)
            best = max(best, child)

        self._recursive_fragile_top_cache[key] = best
        return best

    def top_can_host_fragile(self, L: int, W: int, top_z: int) -> bool:
        key = (L, W, top_z)
        if key in self._top_fragile_fit_cache:
            return self._top_fragile_fit_cache[key]
        ok = self.best_fragile_final_top_z_for_top(L, W, top_z) >= 0
        self._top_fragile_fit_cache[key] = ok
        return ok

    def merged_adjacent_rect(self, x1: int, y1: int, L1: int, W1: int, x2: int, y2: int,
L2: int, W2: int) -> Optional[Tuple[int, int, int, int]]:
        if x1 == x2 and L1 == L2 and (y1 + W1 == y2 or y2 + W2 == y1):
            return (x1, min(y1, y2), L1, W1 + W2)
        if y1 == y2 and W1 == W2 and (x1 + L1 == x2 or x2 + L2 == x1):
            return (min(x1, x2), y1, L1 + L2, W1)
        return None

    def best_fragile_top_from_existing_neighbors(self, x: int, y: int, L: int, W: int,
top_z: int, exclude_pids: Tuple[int, ...] = ()) -> int:
        best = -1
        for other in self-surfaces:
            if other.z != top_z or other.parent_pid is None or other.parent_pid in
exclude_pids:
                continue
            parent = self.by_pid.get(other.parent_pid)
            if parent is None or parent.category != "standard":
                continue
            merged = self.merged_adjacent_rect(x, y, L, W, other.x, other.y, other.L,
other.W)
            if merged is None:
                continue
            _, _, mL, mW = merged
            best = max(best, self.best_fragile_final_top_z_for_top(mL, mW, top_z))
        return best

    def best_fragile_top_from_same_level_sibling_after_builder(
        self,
        surface: Surface,
        code: str,
        l: int,
        w: int,
        h: int,
    ) -> int:
        """估计“先抬当前柱、再抬同层相邻柱”后形成更高 G3 支撑面的潜力。

```

这是针对“G2 只堆一层就开始放 G3”问题的关键修正：
 当前代码原本只能看到“当前柱抬高后与已经同高的邻柱拼面”，
 却看不到“相邻柱下一步也能同步抬高到同一层”这一常见情形。

因而会在较低层过早放置 G3。这里显式地把这类两步平台构造（当前柱 + 同层相邻柱）纳入前瞻评估。

```

"""
if self.remaining_counts.get(code, 0) < 2:
    return -1

spec = self.catalog[code]
top_z = surface.z + h
best = -1

    current_support = tuple(sorted(surface.support_pids)) if surface.support_pids
else (()) if surface.parent_pid is None else (surface.parent_pid,)

    for other in self-surfaces:
        if other is surface:
            continue
        if other.z != surface.z:
            continue

        other_support_boxes = self.surface_support_boxes(other)
        if not other_support_boxes or not all(box.category == "standard" for box
in other_support_boxes):
            continue

        other_support = tuple(sorted(other.support_pids)) if other.support_pids
else (()) if other.parent_pid is None else (other.parent_pid,)
        if other_support == current_support:
            # 同一父面的再摆放潜力已经由“同一支撑面再补一件”逻辑覆盖。
            continue

        if l > other.L or w > other.W:
            continue
        if other.z + h > self.truck.effective_H:
            continue
        if not self.can_support(spec, other, l, w):
            continue

        merged = self.merged_adjacent_rect(surface.x, surface.y, l, w, other.x,
other.y, l, w)
        if merged is None:
            continue
        _, _, mL, mW = merged
        best = max(best, self.best_fragile_final_top_z_for_top(mL, mW, top_z))

    return best

def best_future_fragile_top_after_builder(self, surface: Surface, code: str, l:
int, w: int, h: int) -> int:
    top_z = surface.z + h
    best = self._best_recursive_fragile_top_z_from_top(l, w, top_z,
FRAGILE_PLATFORM_LOOKAHEAD_DEPTH - 1)

    # 情形 A: 当前柱抬高后, 已经存在同高邻柱, 可立即形成更高拼面。
    best = max(best, self.best_fragile_top_from_existing_neighbors(surface.x,
surface.y, l, w, top_z))

    # 情形 B: 当前支撑面本身还足够宽/长, 再补一件同类标准件后形成更高拼面。
    if self.remaining_counts.get(code, 0) >= 2:
        if 2 * l <= surface.L:
            best = max(best, self.best_fragile_final_top_z_for_top(2 * l, w,
top_z))
        if 2 * w <= surface.W:
            best = max(best, self.best_fragile_final_top_z_for_top(l, 2 * w,
top_z))

    # 情形 C: 当前柱先抬高一步, 随后把“同层相邻柱”也抬到同一高度,

```

```

        # 再在更高层形成 G3 支撑面。这个分支正是用于抑制“G2 只放一层就开始放 G3”。
        best = max(best,
self.best_fragile_top_from_same_level_sibling_after_builder(surface, code, l, w, h))
        return best

    def best_future_fragile_top_z_from_surface(self, surface: Surface) -> int:
        key = self.surface_cache_key(surface)
        if key in self._future_fragile_platform_cache:
            return self._future_fragile_platform_cache[key]

        if not self.remaining_fragile_exists() or not
self.remaining_standard_support_builder_exists():
            self._future_fragile_platform_cache[key] = -1
            return -1

        best_final_top = -1
        for code in STANDARD_SUPPORT_BUILDERS:
            if self.remaining_counts.get(code, 0) <= 0:
                continue
            spec = self.catalog[code]
            if self.total_weight + spec.weight > self.truck.max_weight + 1e-12:
                continue

            for l, w, h, _ in unique_orientations(spec.dims, spec.category,
self.fragile_fixed):
                if l > surface.L or w > surface.W:
                    continue
                if surface.z + h > self.truck.effective_H:
                    continue
                if not self.can_support(spec, surface, l, w):
                    continue

                final_top = self._best_recursive_fragile_top_z_from_top(
                    l, w, surface.z + h, FRAGILE_PLATFORM_LOOKAHEAD_DEPTH - 1
                )
                if final_top >= 0:
                    best_final_top = max(best_final_top, final_top)

            self._future_fragile_platform_cache[key] = best_final_top
            return best_final_top

    def generate_fragile_composite_surfaces(self, candidate_surfaces: List[Surface])
-> List[Surface]:
        standard_surfaces = [
            s for s in candidate_surfaces
            if s.parent_pid is not None and self.surface_is_standard_support(s)
       ][:COMPOSITE_SURFACE_PAIR_LIMIT]

        best_by_key: Dict[Tuple[int, int, int, int, int, Tuple[int, ...]], Surface] =
{}

        for i, s1 in enumerate(standard_surfaces):
            p1 = self.by_pid.get(s1.parent_pid)
            if p1 is None or p1.category != "standard":
                continue
            for s2 in standard_surfaces[i + 1 :]:
                p2 = self.by_pid.get(s2.parent_pid)
                if p2 is None or p2.category != "standard" or p2.pid == p1.pid:
                    continue
                if s1.z != s2.z:
                    continue

                merged: Optional[Surface] = None
                # 纵向拼接: 同 x / L, 相邻 y
                if s1.x == s2.x and s1.L == s2.L and (s1.y + s1.W == s2.y or s2.y + s2.W
== s1.y):
                    merged = Surface(
                        x=s1.x,

```

```

        y=min(s1.y, s2.y),
        z=s1.z,
        L=s1.L,
        W=s1.W + s2.W,
        parent_pid=None,
        support_pids=tuple(sorted((p1.pid, p2.pid))),
        source_surfaces=(s1, s2),
    )
    # 横向拼接: 同 y / W, 相邻 x
    elif s1.y == s2.y and s1.W == s2.W and (s1.x + s1.L == s2.x or s2.x +
s2.L == s1.x):
        merged = Surface(
            x=min(s1.x, s2.x),
            y=s1.y,
            z=s1.z,
            L=s1.L + s2.L,
            W=s1.W,
            parent_pid=None,
            support_pids=tuple(sorted((p1.pid, p2.pid))),
            source_surfaces=(s1, s2),
        )
        if merged is None:
            continue
        if self.best_fragile_final_top_z_for_top(merged.L, merged.W,
merged.z) < 0:
            continue
        ckey = (merged.x, merged.y, merged.z, merged.L, merged.W,
merged.support_pids)
        best_by_key[ckey] = merged
        return list(best_by_key.values())

    def best_supported_fragile_choice(self, candidate_surfaces: List[Surface]) ->
Tuple[Optional[Tuple[ItemType, Surface, int, int, int, str]], int]:
        if self.remaining_counts.get(FRAGILE_CODE, 0) <= 0:
            return None, -1

        spec = self.catalog[FRAGILE_CODE]
        if self.total_weight + spec.weight > self.truck.max_weight + 1e-12:
            return None, -1

        best_choice = None
        best_key = None
        best_final_top = -1

        for surface in candidate_surfaces:
            support_boxes = self.surface_support_boxes(surface)
            if support_boxes and not all(box.category == "standard" for box in
support_boxes):
                continue
            # 顶层优先阶段只考虑“可抬高后的高位平台”; 地板情况交给通用搜索处理。
            if not support_boxes:
                continue

            for l, w, h, orient_name in unique_orientations(spec.dims, spec.category,
self.fragile_fixed):
                if l > surface.L or w > surface.W:
                    continue
                if surface.z + h > self.truck.effective_H:
                    continue
                if not self.can_support(spec, surface, l, w):
                    continue

                final_top = surface.z + h
                composite_bonus = 1 if len(surface.support_pids) >= 2 else 0
                key = (final_top, composite_bonus) + self.candidate_key(spec, surface,
l, w, h)
                if best_key is None or key > best_key:

```

```

        best_key = key
        best_choice = (spec, surface, l, w, h, orient_name)
        best_final_top = final_top

    return best_choice, best_final_top

def best_platform_builder_choice(self, candidate_surfaces: List[Surface]) ->
Tuple[Optional[Tuple[ItemType, Surface, int, int, int, str]], int]:
    if not self.remaining_fragile_exists() or not
self.remaining_standard_support_builder_exists():
        return None, -1

    best_choice = None
    best_key = None
    best_future_final_top = -1

    for code in STANDARD_SUPPORT_BUILDERS:
        if self.remaining_counts.get(code, 0) <= 0:
            continue
        spec = self.catalog[code]
        if self.total_weight + spec.weight > self.truck.max_weight + 1e-12:
            continue

        for surface in candidate_surfaces:
            for l, w, h, orient_name in unique_orientations(spec.dims,
spec.category, self.fragile_fixed):
                if l > surface.L or w > surface.W:
                    continue
                if surface.z + h > self.truck.effective_H:
                    continue
                if not self.can_support(spec, surface, l, w):
                    continue

                future_final_top =
self.best_future_fragile_top_after_builder(surface, code, l, w, h)
                if future_final_top < 0:
                    continue

                key = (future_final_top,) + self.candidate_key(spec, surface, l, w,
h)

                if best_key is None or key > best_key:
                    best_key = key
                    best_choice = (spec, surface, l, w, h, orient_name)
                    best_future_final_top = future_final_top

    return best_choice, best_future_final_top

def select_toplayer_fragile_priority_choice(self, candidate_surfaces:
List[Surface]) -> Optional[Tuple[ItemType, Surface, int, int, int, str]]:
    if not self.remaining_fragile_exists():
        return None

    fragile_surfaces = list(candidate_surfaces) +
self.generate_fragile_composite_surfaces(candidate_surfaces)
    fragile_choice, fragile_final_top =
self.best_supported_fragile_choice(fragile_surfaces)
    builder_choice, builder_future_final_top =
self.best_platform_builder_choice(candidate_surfaces)

    if fragile_choice is None and builder_choice is None:
        return None
    if fragile_choice is None:
        return builder_choice
    if builder_choice is None:
        return fragile_choice

# 只要还能明显抬高 G3 的最终顶高，就继续造平台；

```

```

# 否则直接把 G3 放到当前更高的位置。
if builder_future_final_top > fragile_final_top + 1:
    return builder_choice
return fragile_choice

# 表面维护
def split_surface(self, surface: Surface, l: int, w: int) -> List[Surface]:
    # 切法 A
    a_surfaces: List[Surface] = []
    if surface.L - l > 0:
        a_surfaces.append(Surface(surface.x + l, surface.y, surface.z, surface.L
- l, surface.W, surface.parent_pid))
        if surface.W - w > 0:
            a_surfaces.append(Surface(surface.x, surface.y + w, surface.z, l, surface.W
- w, surface.parent_pid))
        score_a = sum(s.area for s in a_surfaces) - abs((surface.L - l) - l) -
abs((surface.W - w) - w)

    # 切法 B
    b_surfaces: List[Surface] = []
    if surface.W - w > 0:
        b_surfaces.append(Surface(surface.x, surface.y + w, surface.z, surface.L,
surface.W - w, surface.parent_pid))
        if surface.L - l > 0:
            b_surfaces.append(Surface(surface.x + l, surface.y, surface.z, surface.L
- l, w, surface.parent_pid))
        score_b = sum(s.area for s in b_surfaces) - abs((surface.L - l) - l) -
abs((surface.W - w) - w)

    return a_surfaces if score_a >= score_b else b_surfaces

def subtract_rect_from_surface(self, surface: Surface, rx: int, ry: int, rl: int,
rw: int) -> List[Surface]:
    ix0 = max(surface.x, rx)
    iy0 = max(surface.y, ry)
    ix1 = min(surface.x + surface.L, rx + rl)
    iy1 = min(surface.y + surface.W, ry + rw)
    if ix0 >= ix1 or iy0 >= iy1:
        return [surface]

    out: List[Surface] = []
    # left / right strips
    if ix0 > surface.x:
        out.append(Surface(surface.x, surface.y, surface.z, ix0 - surface.x,
surface.W, surface.parent_pid))
    if ix1 < surface.x + surface.L:
        out.append(Surface(ix1, surface.y, surface.z, surface.x + surface.L - ix1,
surface.W, surface.parent_pid))
    # bottom / top strips inside the overlapped x-span
    mid_L = ix1 - ix0
    if iy0 > surface.y:
        out.append(Surface(ix0, surface.y, surface.z, mid_L, iy0 - surface.y,
surface.parent_pid))
    if iy1 < surface.y + surface.W:
        out.append(Surface(ix0, iy1, surface.z, mid_L, surface.y + surface.W - iy1,
surface.parent_pid))
    return [s for s in out if s.L > 0 and s.W > 0]

def _merge_surfaces_once(self, surfaces: List[Surface]) -> List[Surface]:
    # 先横向合并
    grouped: Dict[Tuple[int, Optional[int], int, int], List[Surface]] = {}
    for s in surfaces:
        grouped.setdefault((s.z, s.parent_pid, s.y, s.W), []).append(s)

    merged_h: List[Surface] = []
    for key, group in grouped.items():
        group = sorted(group, key=lambda s: (s.x, s.L))

```

```

        cur = group[0]
        for nxt in group[1:]:
            if cur.x + cur.L == nxt.x:
                cur = Surface(cur.x, cur.y, cur.z, cur.L + nxt.L, cur.W,
cur.parent_pid)
            else:
                merged_h.append(cur)
                cur = nxt
        merged_h.append(cur)

# 再纵向合并
grouped2: Dict[Tuple[int, Optional[int], int, int], List[Surface]] = {}
for s in merged_h:
    grouped2.setdefault((s.z, s.parent_pid, s.x, s.L), []).append(s)

merged_v: List[Surface] = []
for key, group in grouped2.items():
    group = sorted(group, key=lambda s: (s.y, s.W))
    cur = group[0]
    for nxt in group[1:]:
        if cur.y + cur.W == nxt.y:
            cur = Surface(cur.x, cur.y, cur.z, cur.L, cur.W + nxt.W,
cur.parent_pid)
        else:
            merged_v.append(cur)
            cur = nxt
    merged_v.append(cur)

return merged_v

def prune_surfaces(self) -> None:
    surfaces = [
        s for s in self.surfaces
        if s.L > 0 and s.W > 0 and s.z <= self.truck.effective_H
    ]

# 去除被包含表面
surfaces.sort(key=lambda s: (-s.area, s.z, s.x, s.y))
kept: List[Surface] = []
for s in surfaces:
    covered = False
    for k in kept:
        if (
            s.parent_pid == k.parent_pid
            and s.z == k.z
            and s.x >= k.x
            and s.y >= k.y
            and s.x + s.L <= k.x + k.L
            and s.y + s.W <= k.y + k.W
        ):
            covered = True
            break
    if not covered:
        kept.append(s)

# 进行两轮邻接面合并
for _ in range(2):
    kept = self._merge_surfaces_once(kept)

kept.sort(key=lambda s: (s.z, s.x, s.y, -s.area))
self.surfaces = kept[:MAX_SURFACES]

# 目标与评分
def utilization(self) -> Dict[str, float]:
    return {
        "空间利用率_raw": self.total_volume / self.truck.raw_volume,
        "空间利用率_eff": self.total_volume / self.truck.eff_volume,

```

```

        "载重利用率": self.total_weight / self.truck.max_weight,
        "已装货物体积_m3": self.total_volume / 1e6,
        "已装货物重量_kg": self.total_weight,
    }

    def truck_score_tuple(self) -> Tuple[float, float, float, float, float]:
        util = self.utilization()
        v = util["空间利用率_eff"]
        w = util["载重利用率"]
        return (
            min(v, w),
            (v + w) / 2.0,
            -abs(v - w),
            v * w,
            len(self.placed),
        )

    def candidate_key(self, spec: ItemType, surface: Surface, l: int, w: int, h: int)
-> Tuple[float, ...]:
        new_v = (self.total_volume + l * w * h) / self.truck.eff_volume
        new_w = (self.total_weight + spec.weight) / self.truck.max_weight
        fill = (l * w) / (surface.L * surface.W)
        cat_bonus = self.policy.category_bonus.get(spec.category, 0.0)
        gap = abs(new_v - new_w)

        support_boxes = self.surface_support_boxes(surface)
        has_standard_support = bool(support_boxes) and all(box.category == "standard"
for box in support_boxes)
        is_composite_support = len(surface.support_pids) >= 2

        fragile_support_bonus = 0.0
        standard_platform_bonus = 0.0
        top_surface_penalty = 0.0
        top_layer_bonus = 0.0
        future_stack_penalty = 0.0
        shape_pref = -float(h)

        if spec.category == "fragile":
            shape_pref = float(h)
            placed_top_z = surface.z + h
            top_ratio = placed_top_z / self.truck.effective_H
            top_layer_bonus = (
                FRAGILE_TOP_LINEAR_BONUS * top_ratio
                + FRAGILE_TOP_QUADRATIC_BONUS * (top_ratio ** 2)
            )
            if placed_top_z >= self.truck.effective_H - FRAGILE_NEAR_TOP_MARGIN_CM:
                top_layer_bonus += FRAGILE_NEAR_TOP_BONUS

            if has_standard_support:
                fragile_support_bonus = 1.2 + (0.35 if is_composite_support else 0.0)
                if not is_composite_support:
                    future_top_z =
self.best_future_fragile_top_z_from_surface(surface)
                    if future_top_z > placed_top_z + 1:
                        gap_ratio = (future_top_z - placed_top_z) /
self.truck.effective_H
                        future_stack_penalty = -(2.8 + 3.5 * gap_ratio)
                    elif self.remaining_standard_support_builder_exists():
                        future_stack_penalty = -1.5

            if spec.category == "standard" and self.remaining_fragile_exists():
                top_z = surface.z + h
                future_fragile_top = self.best_future_fragile_top_after_builder(surface,
spec.code, l, w, h)
                if future_fragile_top >= 0:
                    standard_platform_bonus = 1.0 + 2.8 * (future_fragile_top /
self.truck.effective_H)

```

```

        if future_fragile_top > top_z + 1:
            standard_platform_bonus += 0.8
    elif surface.z == 0:
        standard_platform_bonus = 0.1

    if (
        support_boxes
        and len(support_boxes) == 1
        and support_boxes[0].category == "standard"
        and spec.category == "directional"
        and self.remaining_fragile_exists()
    ):
        top_surface_penalty = -0.7

    if self.policy.mode == "balanced":
        main = (min(new_v, new_w), (new_v + new_w) / 2.0, -gap, new_v * new_w)
    elif self.policy.mode == "volume":
        main = (new_v, min(new_v, new_w), (new_v + new_w) / 2.0, -gap)
    elif self.policy.mode == "weight":
        main = (new_w, min(new_v, new_w), (new_v + new_w) / 2.0, -gap)
    elif self.policy.mode == "product":
        main = (new_v * new_w, min(new_v, new_w), (new_v + new_w) / 2.0, -gap)
    else:
        raise ValueError(f"未知策略模式: {self.policy.mode}")

    return main + (
        fragile_support_bonus,
        standard_platform_bonus,
        top_surface_penalty,
        top_layer_bonus,
        future_stack_penalty,
        shape_pref,
        fill,
        cat_bonus,
        -surface.z,
        -surface.x,
        -surface.y,
    )

# 放置操作
def place(self, spec: ItemType, surface: Surface, l: int, w: int, h: int,
orient_name: str) -> None:
    pid = self.next_pid
    self.next_pid += 1

    support_pids = tuple(surface.support_pids) if surface.support_pids else (()) if
surface.parent_pid is None else (surface.parent_pid,)
    support_parent = surface.parent_pid if not surface.support_pids else None
    placed = PlacedBox(
        pid=pid,
        type_code=spec.code,
        category=spec.category,
        weight=spec.weight,
        x=surface.x,
        y=surface.y,
        z=surface.z,
        l=l,
        w=w,
        h=h,
        orientation_name=orient_name,
        parent_pid=support_parent,
        support_pids=support_pids,
    )

    self.placed.append(placed)
    self.by_pid[pid] = placed
    self.total_weight += spec.weight

```

```

        self.total_volume += placed.volume
        self.remaining_counts[spec.code] -= 1
        self.loaded_counts[spec.code] += 1

        load_map = self.compute_support_loads(spec.weight, surface, placed.x,
placed.y, l, w)
        if load_map is None:
            raise RuntimeError("组合支撑面重量分配失败。")
        self.apply_load_update(load_map)

        if surface.source_surfaces:
            for src in surface.source_surfaces:
                if src in self.surfaces:
                    self.surfaces.remove(src)
                    self.surfaces.extend(self.subtract_rect_from_surface(src,
placed.x, placed.y, l, w))
                else:
                    self.surfaces.remove(surface)
                    self.surfaces.extend(self.split_surface(surface, l, w))

        if spec.category != "fragile":
            top_z = surface.z + h
            if top_z <= self.truck.effective_H:
                self.surfaces.append(Surface(surface.x, surface.y, top_z, l, w, pid))

        self.prune_surfaces()
        self.reset_state_caches()

# 单车装箱主过程
def pack(self) -> "SurfacePacker":
    while self.surfaces and not is_zero_counts(self.remaining_counts):
        self.surfaces.sort(key=lambda s: (s.z, s.x, s.y, -s.area))
        base_candidate_surfaces = self.surfaces[:CANDIDATE_SURFACE_LIMIT]

        priority_choice =
self.select_toplayer_fragile_priority_choice(base_candidate_surfaces)
        if priority_choice is not None:
            spec, surface, l, w, h, orient_name = priority_choice
            self.place(spec, surface, l, w, h, orient_name)
            continue

        best_choice = None
        best_key = None

        for code in ITEM_ORDER:
            if self.remaining_counts.get(code, 0) <= 0:
                continue
            spec = self.catalog[code]

            if self.total_weight + spec.weight > self.truck.max_weight + 1e-12:
                continue

            for surface in base_candidate_surfaces:
                for l, w, h, orient_name in unique_orientations(spec.dims,
spec.category, self.fragile_fixed):
                    if l > surface.L or w > surface.W:
                        continue
                    if surface.z + h > self.truck.effective_H:
                        continue
                    if not self.can_support(spec, surface, l, w):
                        continue

                    key = self.candidate_key(spec, surface, l, w, h)
                    if best_key is None or key > best_key:
                        best_key = key
                        best_choice = (spec, surface, l, w, h, orient_name)

```

```

        if best_choice is None:
            break

        spec, surface, l, w, h, orient_name = best_choice
        self.place(spec, surface, l, w, h, orient_name)

    return self

# 六、策略集
SINGLE_TRUCK_POLICIES: Tuple[PackingPolicy, ...] = (
    PackingPolicy(
        "balanced_platform",
        "balanced",
        {"standard": 2.0, "directional": 1.0, "fragile": 0.0},
    ),
    PackingPolicy(
        "balanced_directional",
        "balanced",
        {"directional": 2.0, "standard": 1.0, "fragile": 0.0},
    ),
    PackingPolicy(
        "product_mix",
        "product",
        {"standard": 1.5, "directional": 1.5, "fragile": 0.2},
    ),
    PackingPolicy(
        "volume_first",
        "volume",
        {"standard": 1.5, "directional": 1.0, "fragile": 0.1},
    ),
    PackingPolicy(
        "weight_first",
        "weight",
        {"standard": 1.0, "directional": 1.0, "fragile": 0.5},
    ),
)

MIN_TRUCK_POLICIES: Tuple[PackingPolicy, ...] = (
    PackingPolicy(
        "cover_volume",
        "volume",
        {"standard": 1.8, "directional": 1.0, "fragile": 0.2},
    ),
    PackingPolicy(
        "cover_balanced",
        "balanced",
        {"standard": 2.0, "directional": 1.0, "fragile": 0.0},
    ),
    PackingPolicy(
        "cover_product",
        "product",
        {"standard": 1.3, "directional": 1.3, "fragile": 0.0},
    ),
    PackingPolicy(
        "fragile_rescue",
        "volume",
        {"fragile": 2.0, "standard": 1.0, "directional": 0.5},
    ),
    PackingPolicy(
        "heavy_mix",
        "weight",
        {"directional": 1.5, "standard": 1.0, "fragile": 0.2},
    ),
)

```

七、候选模式生成、单车择优、最少车辆搜索

```
def compare_balance_packers(packer: SurfacePacker) -> Tuple[float, float, float, float, float]:
```

```
    util = packer.utilization()
```

```
    v = util["空间利用率_eff"]
```

```
    w = util["载重利用率"]
```

```
    return (min(v, w), (v + w) / 2.0, -abs(v - w), v * w, len(packer.placed))
```

```
def compare_cover_packers(packer: SurfacePacker) -> Tuple[float, float, float, float, float, int]:
```

```
    util = packer.utilization()
```

```
    v = util["空间利用率_eff"]
```

```
    w = util["载重利用率"]
```

```
    return (v, min(v, w), w, v * w, len(packer.placed))
```

```
def pack_single_truck_bestof(
```

```
    counts: Dict[str, int],
```

```
    catalog: Dict[str, ItemType],
```

```
    truck: TruckType,
```

```
    fragile_fixed: bool = False,
```

```
) -> SurfacePacker:
```

```
    best = None
```

```
    best_key = None
```

```
    for policy in SINGLE_TRUCK_POLICIES:
```

```
        packer = SurfacePacker(truck, catalog, counts, policy,
```

```
    fragile_fixed=fragile_fixed).pack()
```

```
        key = compare_balance_packers(packer)
```

```
        if best_key is None or key > best_key:
```

```
            best_key = key
```

```
            best = packer
```

```
    return best
```

```
def generate_candidate_packers(
```

```
    counts: Dict[str, int],
```

```
    catalog: Dict[str, ItemType],
```

```
    truck: TruckType,
```

```
    fragile_fixed: bool = False,
```

```
    top_k: int = 4,
```

```
) -> List[SurfacePacker]:
```

```
    candidates = []
```

```
    best_by_pattern: Dict[Tuple[int, ...], Tuple[Tuple[float, ...], SurfacePacker]] = {}
```

```
    for policy in MIN_TRUCK_POLICIES:
```

```
        packer = SurfacePacker(truck, catalog, counts, policy,
```

```
    fragile_fixed=fragile_fixed).pack()
```

```
        if len(packer.placed) == 0:
```

```
            continue
```

```
        pattern = counts_tuple(packer.loaded_counts)
```

```
        score = compare_cover_packers(packer)
```

```
        old = best_by_pattern.get(pattern)
```

```
        if old is None or score > old[0]:
```

```
            best_by_pattern[pattern] = (score, packer)
```

```
    ranked = sorted(best_by_pattern.values(), key=lambda x: x[0], reverse=True)
```

```
    return [packer for _, packer in ranked[:top_k]]
```

```
def greedy_pack_all(
```

```
    counts: Dict[str, int],
```

```
    catalog: Dict[str, ItemType],
```

```
    truck: TruckType,
```

```

        fragile_fixed: bool = False,
    ) -> List[SurfacePacker]:
        remaining = {code: counts.get(code, 0) for code in ITEM_ORDER}
        solution: List[SurfacePacker] = []

        while not is_zero_counts(remaining):
            candidates = generate_candidate_packers(remaining, catalog, truck,
            fragile_fixed=fragile_fixed, top_k=1)
            if not candidates:
                raise RuntimeError("当前启发式未能再装入任何货物，请检查参数设置。")
            chosen = candidates[0]
            solution.append(chosen)
            remaining = subtract_counts(remaining, chosen.loaded_counts)

        return solution

def can_finish_with_n_trucks(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    truck: TruckType,
    n: int,
    fragile_fixed: bool = False,
    top_k: int = 4,
    memo: Optional[Dict[Tuple[int, Tuple[int, ...]], Tuple[bool,
Optional[List[SurfacePacker]]]]] = None,
    candidate_cache: Optional[Dict[Tuple[Tuple[int, ...], str], List[SurfacePacker]]]
= None,
) -> Tuple[bool, Optional[List[SurfacePacker]]]:
    if memo is None:
        memo = {}
    if candidate_cache is None:
        candidate_cache = {}

    state = counts_tuple(counts)
    key = (n, state)
    if key in memo:
        return memo[key]

    if all(v == 0 for v in state):
        memo[key] = (True, [])
        return memo[key]

    if n == 0:
        memo[key] = (False, None)
        return memo[key]

    if volume_weight_lower_bound(counts, catalog, truck) > n:
        memo[key] = (False, None)
        return memo[key]

    ckey = (state, truck.name)
    if ckey not in candidate_cache:
        candidate_cache[ckey] = generate_candidate_packers(
            counts,
            catalog,
            truck,
            fragile_fixed=fragile_fixed,
            top_k=top_k,
        )
    candidates = candidate_cache[ckey]

    for packer in candidates:
        new_counts = subtract_counts(counts, packer.loaded_counts)
        ok, tail = can_finish_with_n_trucks(
            new_counts,
            catalog,

```

```

        truck,
        n - 1,
        fragile_fixed=fragile_fixed,
        top_k=top_k,
        memo=memo,
        candidate_cache=candidate_cache,
    )
    if ok:
        memo[key] = (True, [packer] + (tail or []))
        return memo[key]

memo[key] = (False, None)
return memo[key]

def solve_min_trucks_single_type(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    truck: TruckType,
    fragile_fixed: bool = False,
    max_search_gap: int = 3,
) -> List[SurfacePacker]:
    """固定一种车型时，求“尽量少的车辆数”。

    说明：
    - 先用贪心得到一个可行上界；
    - 只有当“上界 - 下界”不大时，才启动有限深度搜索压缩车辆数；
    - 对于本题放大10倍后的数据，若 gap 过大，直接返回贪心解更稳健，
      这样可以保证程序在比赛环境中运行时间可控。
    """
    lb = volume_weight_lower_bound(counts, catalog, truck)
    greedy_solution = greedy_pack_all(counts, catalog, truck,
    fragile_fixed=fragile_fixed)
    ub = len(greedy_solution)

    if lb == ub:
        return greedy_solution

    if ub - lb > max_search_gap:
        return greedy_solution

    memo: Dict[Tuple[int, Tuple[int, ...]], Tuple[bool,
Optional[List[SurfacePacker]]]] = {}
    candidate_cache: Dict[Tuple[Tuple[int, ...], str], List[SurfacePacker]] = {}

    for n in range(lb, ub + 1):
        ok, sol = can_finish_with_n_trucks(
            counts,
            catalog,
            truck,
            n,
            fragile_fixed=fragile_fixed,
            top_k=4,
            memo=memo,
            candidate_cache=candidate_cache,
        )
        if ok and sol is not None:
            return sol

    return greedy_solution

# 八、导出与结果整理
def uid_width_for_catalog(catalog: Dict[str, ItemType]) -> int:
    return max(4, max(len(str(catalog[code].quantity)) for code in ITEM_ORDER))

```

```

def solution_rows_from_packers(packers: List[SurfacePacker], catalog: Dict[str,
ItemType]) -> List[Dict[str, object]]:
    width = uid_width_for_catalog(catalog)
    next_index = {code: 1 for code in ITEM_ORDER}
    rows: List[Dict[str, object]] = []
    category_name_map = {
        "standard": "标准件",
        "fragile": "易碎件",
        "directional": "定向件",
    }

    for truck_idx, packer in enumerate(packers, start=1):
        pid_to_uid: Dict[int, str] = {}
        for p in packer.placed:
            uid = f"{p.type_code}_{next_index[p.type_code]:0{width}d}"
            next_index[p.type_code] += 1
            pid_to_uid[p.pid] = uid

        for p in packer.placed:
            rows.append(
                {
                    "车辆编号": truck_idx,
                    "车型": packer.truck.name,
                    "货物唯一编号": pid_to_uid[p.pid],
                    "货物类型": p.type_code,
                    "货物类别": category_name_map.get(p.category, p.category),
                    "x坐标_右后到前_cm": p.x,
                    "y坐标_右到左_cm": p.y,
                    "z坐标_下到上_cm": p.z,
                    "放置长度_x方向_cm": p.l,
                    "放置宽度_y方向_cm": p.w,
                    "放置高度_z方向_cm": p.h,
                    "摆放姿态": p.orientation_name,
                    "支撑对象编号": (
                        "组合支撑:" + "|".join(pid_to_uid[pid] for pid in
p.support_pids)
                        if len(p.support_pids) >= 2
                        else (pid_to_uid.get(p.parent_pid, "地板") if p.parent_pid is
not None else "地板")
                    ),
                    "货物重量_kg": p.weight,
                    "货物体积_cm3": p.volume,
                }
            )
        return rows

def summary_rows(title: str, packers: List[SurfacePacker]) -> List[Dict[str, object]]:
    rows = []
    for idx, packer in enumerate(packers, start=1):
        util = packer.utilization()
        row = {
            "方案": title,
            "车辆序号": idx,
            "车型": packer.truck.name,
            "装载件数": len(packer.placed),
            "空间利用率_raw": util["空间利用率_raw"],
            "空间利用率_eff": util["空间利用率_eff"],
            "载重利用率": util["载重利用率"],
            "已装货物体积_m3": util["已装货物体积_m3"],
            "已装货物重量_kg": util["已装货物重量_kg"],
        }
        for code in ITEM_ORDER:

```

```

        row[f"{code}_数量"] = packer.loaded_counts[code]
    rows.append(row)
    return rows

def print_solution_summary(title: str, packers: List[SurfacePacker]) -> None:
    print("=" * 78)
    print(title)
    print("=" * 78)
    print(f"总车辆数: {len(packers)}")
    total_vol = sum(p.total_volume for p in packers) / 1e6
    total_wt = sum(p.total_weight for p in packers)
    print(f"总装载体积: {total_vol:.3f} m^3")
    print(f"总装载重量: {total_wt:.1f} kg")
    print()

    for i, p in enumerate(packers, start=1):
        util = p.utilization()
        print(f"车辆 {i} | {p.truck.name} | 件数 {len(p.placed)}")
        print(f" 空间利用率(raw): {util['空间利用率_raw']:.4f}")
        print(f" 空间利用率(eff): {util['空间利用率_eff']:.4f}")
        print(f" 载重利用率      : {util['载重利用率']:.4f}")
        print(f" 已装体积          : {util['已装货物体积_m3']:.3f} m^3")
        print(f" 已装重量          : {util['已装货物重量_kg']:.1f} kg")
        print(
            " 货类计数      : "
            + ", ".join(f"{code}={p.loaded_counts[code]}" for code in ITEM_ORDER)
        )
        print()

# 九、问题2: 多车型组合配送（独立于问题1的精确求解）
PATTERN_POLICIES_Q2: Tuple[PackingPolicy, ...] = (
    PackingPolicy("cover_volume", "volume", {"standard": 1.8, "directional": 1.0,
"fragile": 0.2}),
    PackingPolicy("cover_balanced", "balanced", {"standard": 2.0, "directional": 1.0,
"fragile": 0.0}),
    PackingPolicy("directional_bias", "volume", {"directional": 2.5, "standard": 0.8,
"fragile": 0.3}),
    PackingPolicy("heavy_mix", "weight", {"directional": 1.5, "standard": 1.0,
"fragile": 0.2}),
)

# 比原版更丰富的截断网格，有利于生成更多高质量单车模式，
# 再由 MILP 做“最少车/最低成本”精确覆盖。
CAP_GRID_Q2: Tuple[Optional[int], ...] = (None, 30, 80, 150, 300)

_PATTERN_CACHE_Q2: Dict[Tuple[str, Tuple[int, ...], bool], List[SurfacePacker]] = {}

# 运行时间控制：对 Q2 的模式库做“主评分 + 性价比”双通道裁剪，
# 同时缩短若干辅助 MILP 的时限，避免最少成本分支因为时间上限返回较差可行解。
Q2_TRUCK_LIBRARY_KEEP_MAIN = 90
Q2_TRUCK_LIBRARY_KEEP_COST = 60
Q2_JOINT_LIBRARY_KEEP_MAIN = 150
Q2_JOINT_LIBRARY_KEEP_COST = 100
Q2_TIME_LIMIT_MIN_TRUCKS_STAGE1 = 90.0
Q2_TIME_LIMIT_MIN_TRUCKS_STAGE2 = 45.0
Q2_TIME_LIMIT_COST_STAGE1 = 35.0
Q2_TIME_LIMIT_COST_STAGE2 = 35.0
Q2_TIME_LIMIT_BALANCE = 30.0
Q2_TIME_LIMIT_THRESHOLD = 10.0
Q2_TIME_LIMIT_CHEAPER_CHECK = 8.0
Q2_TIME_LIMIT_CHEAPER_SEARCH = 15.0

def _pattern_subsets_q2() -> Tuple[Tuple[str, ...], ...]:

```

```

subsets: List[Tuple[str, ...]] = []
for r in (1, 2, 3):
    subsets.extend(combinations(ITEM_ORDER, r))
subsets.extend(
    [
        ("G1", "G2", "G4", "G5"),
        ("G2", "G3", "G4", "G5"),
        ("G1", "G2", "G3", "G5"),
        ("G1", "G2", "G3", "G4"),
        ("G1", "G3", "G4"),
        ITEM_ORDER,
    ]
)
return tuple(subsets)

PATTERN_SUBSETS_Q2: Tuple[Tuple[str, ...], ...] = _pattern_subsets_q2()

def _masked_counts_q2(counts: Dict[str, int], subset: Sequence[str], cap:
Optional[int]) -> Dict[str, int]:
    out: Dict[str, int] = {}
    for code in ITEM_ORDER:
        if code not in subset:
            out[code] = 0
        elif cap is None:
            out[code] = counts[code]
        else:
            out[code] = min(counts[code], cap)
    return out

def _pattern_key_q2(packer: SurfacePacker) -> Tuple[str, Tuple[int, ...]]:
    return packer.truck.name, tuple(packer.loaded_counts[code] for code in ITEM_ORDER)

@lru_cache(maxsize=None)
def _pattern_min_fill_from_key_q2(
    loaded_g1: int,
    loaded_g2: int,
    loaded_g3: int,
    loaded_g4: int,
    loaded_g5: int,
    truck_name: str,
) -> float:
    trucks, catalog = build_problem_data()
    truck = next(t for t in trucks if t.name == truck_name)
    total_vol = 0
    total_wt = 0.0
    for code, qty in zip(ITEM_ORDER, (loaded_g1, loaded_g2, loaded_g3, loaded_g4,
loaded_g5)):
        spec = catalog[code]
        total_vol += qty * spec.volume
        total_wt += qty * spec.weight
    v = total_vol / truck.eff_volume
    w = total_wt / truck.max_weight
    return min(v, w)

def pattern_min_fill_q2(packer: SurfacePacker) -> float:
    key = tuple(packer.loaded_counts[code] for code in ITEM_ORDER)
    return _pattern_min_fill_from_key_q2(*key, packer.truck.name)

def pattern_score_tuple_q2(packer: SurfacePacker) -> Tuple[float, float, float, float,

```

```

int, int, float]:
    util = packer.utilization()
    v = util["空间利用率_eff"]
    w = util["载重利用率"]
    mix = sum(1 for code in ITEM_ORDER if packer.loaded_counts[code] > 0)
    return (
        min(v, w),
        (v + w) / 2.0,
        v * w,
        v,
        mix,
        len(packer.placed),
        -packer.truck.trip_cost,
    )

def pattern_secondary_score_q2(packer: SurfacePacker) -> float:
    util = packer.utilization()
    v = util["空间利用率_eff"]
    w = util["载重利用率"]
    mix = sum(1 for code in ITEM_ORDER if packer.loaded_counts[code] > 0)
    return 1000.0 * min(v, w) + 250.0 * (v * w) + 20.0 * mix + 0.1 * len(packer.placed)

def sort_solution_by_fullness_q2(packers: List[SurfacePacker]) ->
List[SurfacePacker]:
    return sorted(packers, key=pattern_score_tuple_q2, reverse=True)

def pattern_cost_efficiency_q2(packer: SurfacePacker) -> Tuple[float, float, float,
float, int, float]:
    util = packer.utilization()
    v = util["空间利用率_eff"]
    w = util["载重利用率"]
    trip_cost = max(1.0, float(packer.truck.trip_cost))
    return (
        packer.total_volume / trip_cost,
        min(v, w),
        packer.total_weight / trip_cost,
        v * w,
        len(packer.placed),
        -packer.truck.trip_cost,
    )

def _clip_pattern_library_q2(
    packers: List[SurfacePacker],
    keep_main: int,
    keep_cost: int,
    seed_packers: Optional[Sequence[SurfacePacker]] = None,
) -> List[SurfacePacker]:
    seed_packers = tuple(seed_packers or ())
    unique: Dict[Tuple[str, Tuple[int, ...]], SurfacePacker] = {}
    for packer in list(seed_packers) + list(packers):
        key = _pattern_key_q2(packer)
        old = unique.get(key)
        if old is None or pattern_score_tuple_q2(packer) > pattern_score_tuple_q2(old):
            unique[key] = packer

    ranked_main = sorted(unique.values(), key=pattern_score_tuple_q2, reverse=True)
    ranked_cost = sorted(unique.values(), key=pattern_cost_efficiency_q2,
reverse=True)

```

```

out: List[SurfacePacker] = []
seen: set[Tuple[str, Tuple[int, ...]]] = set()

def add_from(seq: Sequence[SurfacePacker], limit: Optional[int] = None) -> None:
    taken = 0
    for packer in seq:
        key = _pattern_key_q2(packer)
        if key in seen:
            continue
        seen.add(key)
        out.append(packer)
        taken += 1
        if limit is not None and taken >= limit:
            break

    add_from(seed_packers)
    add_from(ranked_main, keep_main)
    add_from(ranked_cost, keep_cost)
    return sorted(out, key=pattern_score_tuple_q2, reverse=True)

def generate_pattern_library_q2(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    truck: TruckType,
    fragile_fixed: bool = False,
) -> List[SurfacePacker]:
    cache_key = (truck.name, counts_tuple(counts), fragile_fixed)
    if cache_key in _PATTERN_CACHE_Q2:
        return _PATTERN_CACHE_Q2[cache_key]

    best_by_pattern: Dict[Tuple[str, Tuple[int, ...]], SurfacePacker] = {}

    greedy_solution = greedy_pack_all(counts, catalog, truck,
    fragile_fixed=fragile_fixed)
    for packer in greedy_solution:
        best_by_pattern[_pattern_key_q2(packer)] = packer

    for subset in PATTERN_SUBSETS_Q2:
        for cap in CAP_GRID_Q2:
            masked = _masked_counts_q2(counts, subset, cap)
            if not any(masked.values()):
                continue
            for policy in PATTERN_POLICIES_Q2:
                packer = SurfacePacker(truck, catalog, masked, policy,
    fragile_fixed=fragile_fixed).pack()
                if not packer.placed:
                    continue
                key = _pattern_key_q2(packer)
                old = best_by_pattern.get(key)
                if old is None or pattern_score_tuple_q2(packer) >
    pattern_score_tuple_q2(old):
                    best_by_pattern[key] = packer

    library = _clip_pattern_library_q2(
        list(best_by_pattern.values()),
        keep_main=Q2_TRUCK_LIBRARY_KEEP_MAIN,
        keep_cost=Q2_TRUCK_LIBRARY_KEEP_COST,
        seed_packers=greedy_solution,
    )
    _PATTERN_CACHE_Q2[cache_key] = library
    return library

def generate_joint_pattern_library_q2(

```

```

counts: Dict[str, int],
catalog: Dict[str, ItemType],
trucks: Sequence[TruckType],
fragile_fixed: bool = False,
) -> List[SurfacePacker]:
    all_patterns: List[SurfacePacker] = []
    for truck in trucks:
        all_patterns.extend(generate_pattern_library_q2(counts, catalog, truck,
fragile_fixed=fragile_fixed))
    best_by_pattern: Dict[Tuple[str, Tuple[int, ...]], SurfacePacker] = {}
    for packer in all_patterns:
        key = _pattern_key_q2(packer)
        old = best_by_pattern.get(key)
        if old is None or pattern_score_tuple_q2(packer) > pattern_score_tuple_q2(old):
            best_by_pattern[key] = packer
    return _clip_pattern_library_q2(
        list(best_by_pattern.values()),
        keep_main=Q2_JOINT_LIBRARY_KEEP_MAIN,
        keep_cost=Q2_JOINT_LIBRARY_KEEP_COST,
    )

def _build_count_matrix_q2(packers: List[SurfacePacker]) -> np.ndarray:
    return np.array([[packer.loaded_counts[code] for packer in packers] for code in
ITEM_ORDER], dtype=float)

def _target_vector_q2(counts: Dict[str, int]) -> np.ndarray:
    return np.array([counts[code] for code in ITEM_ORDER], dtype=float)

def _truck_count_vector_q2(packers: List[SurfacePacker]) -> np.ndarray:
    return np.ones(len(packers), dtype=float)

def _cost_vector_q2(packers: List[SurfacePacker]) -> np.ndarray:
    return np.array([packer.truck.trip_cost for packer in packers], dtype=float)

def _type_indicator_q2(packers: List[SurfacePacker], truck_name: str) -> np.ndarray:
    return np.array([1.0 if packer.truck.name == truck_name else 0.0 for packer in
packers], dtype=float)

def _run_milp_exact_cover_q2(
    counts: Dict[str, int],
    packers: List[SurfacePacker],
    objective: np.ndarray,
    exact_truck_count: Optional[int] = None,
    exact_total_cost: Optional[float] = None,
    exact_type_counts: Optional[Dict[str, int]] = None,
    max_total_cost: Optional[float] = None,
    time_limit: float = 90.0,
):
    if not packers:
        return None

    matrix = _build_count_matrix_q2(packers)
    target = _target_vector_q2(counts)

    constraints: List[LinearConstraint] = [LinearConstraint(matrix, target, target)]

```

```

    if exact_truck_count is not None:
        ones = _truck_count_vector_q2(packers).reshape(1, -1)
        target_vec = np.array([float(exact_truck_count)])
        constraints.append(LinearConstraint(ones, target_vec, target_vec))

    if exact_total_cost is not None:
        costs = _cost_vector_q2(packers).reshape(1, -1)
        target_vec = np.array([float(exact_total_cost)])
        constraints.append(LinearConstraint(costs, target_vec, target_vec))

    if max_total_cost is not None:
        costs = _cost_vector_q2(packers).reshape(1, -1)
        lb = np.array([-np.inf], dtype=float)
        ub = np.array([float(max_total_cost)], dtype=float)
        constraints.append(LinearConstraint(costs, lb, ub))

    if exact_type_counts:
        for truck_name, wanted in exact_type_counts.items():
            ind = _type_indicator_q2(packers, truck_name).reshape(1, -1)
            target_vec = np.array([float(wanted)])
            constraints.append(LinearConstraint(ind, target_vec, target_vec))

    bounds = Bounds(lb=np.zeros(len(packers)), ub=np.full(len(packers), np.inf))
    integrality = np.ones(len(packers), dtype=int)
    return milp(
        c=objective,
        constraints=constraints,
        integrality=integrality,
        bounds=bounds,
        options={"time_limit": float(time_limit)},
    )

def _extract_integer_coeffs_q2(result) -> Optional[np.ndarray]:
    if result is None or result.x is None:
        return None
    coeffs = np rint(result.x).astype(int)
    if np.max(np.abs(result.x - coeffs)) > 1e-5:
        return None
    return coeffs

def _coeffs_to_solution_q2(packers: List[SurfacePacker], coeffs: np.ndarray) -> List[SurfacePacker]:
    out: List[SurfacePacker] = []
    for idx, times in enumerate(coeffs.tolist()):
        if times <= 0:
            continue
        for _ in range(times):
            out.append(copy.deepcopy(packers[idx]))
    return sort_solution_by_fullness_q2(out)

def _solution_counts_q2(packers: List[SurfacePacker]) -> Dict[str, int]:
    acc = {code: 0 for code in ITEM_ORDER}
    for packer in packers:
        for code in ITEM_ORDER:
            acc[code] += packer.loaded_counts[code]
    return acc

def _solution_matches_target_q2(packers: List[SurfacePacker], counts: Dict[str, int])

```

```

-> bool:
    return _solution_counts_q2(packers) == {code: counts[code] for code in ITEM_ORDER}

def scheme_total_cost(packers: List[SurfacePacker]) -> float:
    return float(sum(packer.truck.trip_cost for packer in packers))

def _rounded_cost_q2(value: float) -> int:
    return int(round(float(value)))

def _solution_satisfies_constraints_q2(
    packers: List[SurfacePacker],
    counts: Dict[str, int],
    exact_truck_count: Optional[int] = None,
    exact_total_cost: Optional[float] = None,
    exact_type_counts: Optional[Dict[str, int]] = None,
    max_total_cost: Optional[float] = None,
) -> bool:
    if not _solution_matches_target_q2(packers, counts):
        return False
    if exact_truck_count is not None and len(packers) != int(exact_truck_count):
        return False
    total_cost = _rounded_cost_q2(scheme_total_cost(packers))
    if exact_total_cost is not None and total_cost !=
    _rounded_cost_q2(exact_total_cost):
        return False
    if max_total_cost is not None and total_cost > _rounded_cost_q2(max_total_cost):
        return False
    if exact_type_counts:
        type_counts = scheme_truck_type_counts(packers)
        for truck_name, wanted in exact_type_counts.items():
            if type_counts.get(truck_name, 0) != int(wanted):
                return False
    return True

def _extract_valid_solution_from_result_q2(
    result,
    packers: List[SurfacePacker],
    counts: Dict[str, int],
    exact_truck_count: Optional[int] = None,
    exact_total_cost: Optional[float] = None,
    exact_type_counts: Optional[Dict[str, int]] = None,
    max_total_cost: Optional[float] = None,
    require_success: bool = False,
) -> Optional[List[SurfacePacker]]:
    if result is None or result.x is None:
        return None
    if require_success and not bool(getattr(result, "success", False)):
        return None
    coeffs = _extract_integer_coeffs_q2(result)
    if coeffs is None:
        return None
    solution = _coeffs_to_solution_q2(packers, coeffs)
    if not _solution_satisfies_constraints_q2(
        solution,
        counts,
        exact_truck_count=exact_truck_count,
        exact_total_cost=exact_total_cost,
        exact_type_counts=exact_type_counts,
        max_total_cost=max_total_cost,

```

```

    ):
        return None
    return solution

def _truck_cost_step_q2(trucks: Sequence[TruckType]) -> int:
    step = 0
    for truck in trucks:
        step = math.gcd(step, int(round(float(truck.trip_cost))))
    return max(1, step)

def scheme_truck_type_counts(packers: List[SurfacePacker]) -> Dict[str, int]:
    counts: Dict[str, int] = {}
    for packer in packers:
        counts[packer.truck.name] = counts.get(packer.truck.name, 0) + 1
    return counts

def scheme_fill_stats(packers: List[SurfacePacker]) -> Dict[str, float]:
    if not packers:
        return {
            "min_空间利用率_eff": 0.0,
            "avg_空间利用率_eff": 0.0,
            "min_载重利用率": 0.0,
            "avg_载重利用率": 0.0,
            "min_min利用率": 0.0,
            "avg_min利用率": 0.0,
        }
    v_list = [p.utilization()["空间利用率_eff"] for p in packers]
    w_list = [p.utilization()["载重利用率"] for p in packers]
    min_list = [min(v, w) for v, w in zip(v_list, w_list)]
    return {
        "min_空间利用率_eff": float(min(v_list)),
        "avg_空间利用率_eff": float(sum(v_list) / len(v_list)),
        "min_载重利用率": float(min(w_list)),
        "avg_载重利用率": float(sum(w_list) / len(w_list)),
        "min_min利用率": float(min(min_list)),
        "avg_min利用率": float(sum(min_list) / len(min_list)),
    }

def build_scheme_bundle_q2(name: str, packers: List[SurfacePacker]) -> Dict[str, object]:
    type_counts = scheme_truck_type_counts(packers)
    fill_stats = scheme_fill_stats(packers)
    return {
        "scheme_name": name,
        "packers": sort_solution_by_fullness_q2(packers),
        "truck_count": len(packers),
        "total_cost": scheme_total_cost(packers),
        "truck_type_counts": type_counts,
        **fill_stats,
    }

def _fallback_greedy_multi_type_q2(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    trucks: Sequence[TruckType],
    objective: str,

```

```

        fragile_fixed: bool = False,
    ) -> List[SurfacePacker]:
        remaining = {code: counts[code] for code in ITEM_ORDER}
        solution: List[SurfacePacker] = []

        while not is_zero_counts(remaining):
            best: Optional[SurfacePacker] = None
            best_key = None
            for truck in trucks:
                candidates = generate_candidate_packers(remaining, catalog, truck,
fragile_fixed=fragile_fixed, top_k=2)
                for cand in candidates:
                    util = cand.utilization()
                    v = util["空间利用率_eff"]
                    w = util["载重利用率"]
                    if objective == "cost":
                        key = (
                            cand.total_volume / truck.trip_cost,
                            cand.total_weight / truck.trip_cost,
                            min(v, w),
                            v,
                            -truck.trip_cost,
                            len(cand.placed),
                        )
                    else:
                        key = (
                            cand.total_volume,
                            cand.total_weight,
                            min(v, w),
                            v,
                            -truck.trip_cost,
                            len(cand.placed),
                        )
                    if best_key is None or key > best_key:
                        best_key = key
                        best = cand
            if best is None:
                raise RuntimeError("多车型贪心回退未能继续装载，建议检查候选模式。")
            solution.append(best)
            remaining = subtract_counts(remaining, best.loaded_counts)

        return sort_solution_by_fullness_q2(solution)

def _find_any_solution_with_cost_cap_q2(
    counts: Dict[str, int],
    library: List[SurfacePacker],
    cost_cap: int,
    time_limit: float,
) -> Optional[List[SurfacePacker]]:
    if cost_cap < 0:
        return None
    result = _run_milp_exact_cover_q2(
        counts,
        library,
        objective=np.zeros(len(library), dtype=float),
        exact_truck_count=None,
        exact_total_cost=None,
        exact_type_counts=None,
        max_total_cost=float(cost_cap),
        time_limit=time_limit,
    )
    return _extract_valid_solution_from_result_q2(
        result,
        library,
        counts,

```

```

        max_total_cost=float(cost_cap),
    )

def _find_min_feasible_cost_under_upper_q2(
    counts: Dict[str, int],
    library: List[SurfacePacker],
    upper_cost: int,
    cost_step: int,
) -> Tuple[int, Optional[List[SurfacePacker]]]:
    upper_cost = _rounded_cost_q2(upper_cost)
    cheaper_cap = upper_cost - cost_step
    if cheaper_cap < 0:
        return upper_cost, None

    quick_solution = _find_any_solution_with_cost_cap_q2(
        counts,
        library,
        cheaper_cap,
        time_limit=Q2_TIME_LIMIT_CHEAPER_CHECK,
    )
    if quick_solution is None:
        return upper_cost, None

    caps = list(range(0, upper_cost + cost_step, cost_step))
    lo = 0
    hi = len(caps) - 1
    best_cap = upper_cost
    best_solution: Optional[List[SurfacePacker]] = None

    while lo <= hi:
        mid = (lo + hi) // 2
        cap = caps[mid]
        solution = _find_any_solution_with_cost_cap_q2(
            counts,
            library,
            cap,
            time_limit=Q2_TIME_LIMIT_CHEAPER_SEARCH,
        )
        if solution is not None:
            best_cap = cap
            best_solution = solution
            hi = mid - 1
        else:
            lo = mid + 1

    if best_solution is None:
        return upper_cost, None
    return _rounded_cost_q2(scheme_total_cost(best_solution)), best_solution

def _highest_feasible_fill_threshold_q2(
    counts: Dict[str, int],
    library: List[SurfacePacker],
    exact_truck_count: Optional[int] = None,
    exact_total_cost: Optional[float] = None,
    exact_type_counts: Optional[Dict[str, int]] = None,
) -> float:
    values = sorted({pattern_min_fill_q2(packer) for packer in library})
    if not values:
        return 0.0

    lo = 0
    hi = len(values) - 1
    best = 0.0

```

```

while lo <= hi:
    mid = (lo + hi) // 2
    threshold = values[mid]
    filtered = [packer for packer in library if pattern_min_fill_q2(packer) + 1e-9
>= threshold]
    result = _run_milp_exact_cover_q2(
        counts,
        filtered,
        objective=np.zeros(len(filtered), dtype=float),
        exact_truck_count=exact_truck_count,
        exact_total_cost=exact_total_cost,
        exact_type_counts=exact_type_counts,
        time_limit=Q2_TIME_LIMIT_THRESHOLD,
    )
    solution = _extract_valid_solution_from_result_q2(
        result,
        filtered,
        counts,
        exact_truck_count=exact_truck_count,
        exact_total_cost=exact_total_cost,
        exact_type_counts=exact_type_counts,
    )
    if solution is not None:
        best = threshold
        lo = mid + 1
        continue
    hi = mid - 1
return best

def optimize_balance_with_constraints_q2(
    counts: Dict[str, int],
    library: List[SurfacePacker],
    exact_truck_count: Optional[int] = None,
    exact_total_cost: Optional[float] = None,
    exact_type_counts: Optional[Dict[str, int]] = None,
) -> List[SurfacePacker]:
    threshold = _highest_feasible_fill_threshold_q2(
        counts,
        library,
        exact_truck_count=exact_truck_count,
        exact_total_cost=exact_total_cost,
        exact_type_counts=exact_type_counts,
    )
    filtered = [packer for packer in library if pattern_min_fill_q2(packer) + 1e-9 >=
threshold]
    if not filtered:
        filtered = library

    objective = -np.array([pattern_secondary_score_q2(packer) for packer in filtered],
dtype=float)
    result = _run_milp_exact_cover_q2(
        counts,
        filtered,
        objective=objective,
        exact_truck_count=exact_truck_count,
        exact_total_cost=exact_total_cost,
        exact_type_counts=exact_type_counts,
        time_limit=Q2_TIME_LIMIT_BALANCE,
    )
    solution = _extract_valid_solution_from_result_q2(
        result,
        filtered,
        counts,
        exact_truck_count=exact_truck_count,
        exact_total_cost=exact_total_cost,

```

```

        exact_type_counts=exact_type_counts,
    )
    if solution is not None:
        return solution

    fallback_result = _run_milp_exact_cover_q2(
        counts,
        library,
        objective=-np.array([pattern_secondary_score_q2(packer) for packer in
library], dtype=float),
        exact_truck_count=exact_truck_count,
        exact_total_cost=exact_total_cost,
        exact_type_counts=exact_type_counts,
        time_limit=Q2_TIME_LIMIT_BALANCE,
    )
    fallback_solution = _extract_valid_solution_from_result_q2(
        fallback_result,
        library,
        counts,
        exact_truck_count=exact_truck_count,
        exact_total_cost=exact_total_cost,
        exact_type_counts=exact_type_counts,
    )
    if fallback_solution is not None:
        return fallback_solution

    return []

def solve_min_total_trucks_q2(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    trucks: Sequence[TruckType],
    fragile_fixed: bool = False,
) -> Dict[str, object]:
    library = generate_joint_pattern_library_q2(counts, catalog, trucks,
fragile_fixed=fragile_fixed)
    result1 = _run_milp_exact_cover_q2(
        counts,
        library,
        objective=np.ones(len(library), dtype=float),
        exact_truck_count=None,
        exact_total_cost=None,
        time_limit=Q2_TIME_LIMIT_MIN_TRUCKS_STAGE1,
    )
    stage1_solution = _extract_valid_solution_from_result_q2(result1, library,
counts)
    if stage1_solution is None:
        fallback = _fallback_greedy_multi_type_q2(counts, catalog, trucks,
objective="truck_count", fragile_fixed=fragile_fixed)
        return build_scheme_bundle_q2("Q2(1)_min_total_trucks_fallback", fallback)

    truck_count = len(stage1_solution)

    costs = _cost_vector_q2(library)
    result2 = _run_milp_exact_cover_q2(
        counts,
        library,
        objective=costs,
        exact_truck_count=truck_count,
        exact_total_cost=None,
        time_limit=Q2_TIME_LIMIT_MIN_TRUCKS_STAGE2,
    )
    stage2_solution = _extract_valid_solution_from_result_q2(
        result2,
        library,

```

```

        counts,
        exact_truck_count=truck_count,
    )
    if stage2_solution is None:
        return build_scheme_bundle_q2("Q2(1)_min_total_trucks", stage1_solution)

    min_cost_at_min_trucks = scheme_total_cost(stage2_solution)
    balanced = optimize_balance_with_constraints_q2(
        counts,
        library,
        exact_truck_count=truck_count,
        exact_total_cost=min_cost_at_min_trucks,
    )
    if balanced:
        return build_scheme_bundle_q2("Q2(1)_min_total_trucks", balanced)

    return build_scheme_bundle_q2("Q2(1)_min_total_trucks", stage2_solution)

def solve_min_total_cost_q2(
    counts: Dict[str, int],
    catalog: Dict[str, ItemType],
    trucks: Sequence[TruckType],
    fragile_fixed: bool = False,
    upper_bound_bundle: Optional[Dict[str, object]] = None,
) -> Dict[str, object]:
    library = generate_joint_pattern_library_q2(counts, catalog, trucks,
    fragile_fixed=fragile_fixed)
    costs = _cost_vector_q2(library)

    fallback_packers = _fallback_greedy_multi_type_q2(counts, catalog, trucks,
    objective="cost", fragile_fixed=fragile_fixed)
    best_bundle = build_scheme_bundle_q2("Q2(2)_min_total_cost_fallback",
    fallback_packers)
    if upper_bound_bundle is not None and upper_bound_bundle["total_cost"] <
    best_bundle["total_cost"]:
        best_bundle = build_scheme_bundle_q2("Q2(2)_min_total_cost_upper_bound",
    upper_bound_bundle["packers"])

    # 先做一轮较短的“成本最小化”MILP；只有当它真的给出了更低成本的可行方案时才采纳。
    result1 = _run_milp_exact_cover_q2(
        counts,
        library,
        objective=costs,
        exact_truck_count=None,
        exact_total_cost=None,
        time_limit=Q2_TIME_LIMIT_COST_STAGE1,
    )
    stage1_solution = _extract_valid_solution_from_result_q2(result1, library,
    counts)
    stage1_proven_optimal = stage1_solution is not None and bool(getattr(result1,
    "success", False))
    if stage1_solution is not None:
        stage1_bundle = build_scheme_bundle_q2("Q2(2)_min_total_cost_stage1",
    stage1_solution)
        if stage1_bundle["total_cost"] < best_bundle["total_cost"]:
            best_bundle = stage1_bundle

    cost_step = _truck_cost_step_q2(trucks)
    upper_cost = _rounded_cost_q2(best_bundle["total_cost"])

    # 如果当前最好成本已经有可行上界，先快速检查“再便宜一个步长”是否可行；
    # 不可行则直接返回该上界，避免因为 MILP 超时把更贵方案误标成最低成本。
    if not (stage1_proven_optimal and _rounded_cost_q2(best_bundle["total_cost"]) ==
    upper_cost):
        cheaper_cost, cheaper_solution = _find_min_feasible_cost_under_upper_q2(

```

```

        counts,
        library,
        upper_cost,
        cost_step,
    )
    if cheaper_solution is not None and cheaper_cost < upper_cost:
        best_bundle = build_scheme_bundle_q2("Q2(2)_min_total_cost",
cheaper_solution)
        upper_cost = cheaper_cost
    elif upper_bound_bundle is not None and
_rounded_cost_q2(upper_bound_bundle["total_cost"]) == upper_cost:
        # 当前库里没有找到比上界更便宜的方案，直接返回这个已知更优可行解。
        return build_scheme_bundle_q2("Q2(2)_min_total_cost",
upper_bound_bundle["packers"])

    target_cost = _rounded_cost_q2(best_bundle["total_cost"])
    result2 = _run_milp_exact_cover_q2(
        counts,
        library,
        objective=np.ones(len(library), dtype=float),
        exact_truck_count=None,
        exact_total_cost=target_cost,
        time_limit=Q2_TIME_LIMIT_COST_STAGE2,
    )
    stage2_solution = _extract_valid_solution_from_result_q2(
        result2,
        library,
        counts,
        exact_total_cost=target_cost,
    )
    if stage2_solution is None:
        return build_scheme_bundle_q2("Q2(2)_min_total_cost",
best_bundle["packers"])

    truck_count = len(stage2_solution)
    balanced = optimize_balance_with_constraints_q2(
        counts,
        library,
        exact_truck_count=truck_count,
        exact_total_cost=target_cost,
    )
    if balanced:
        return build_scheme_bundle_q2("Q2(2)_min_total_cost", balanced)

    return build_scheme_bundle_q2("Q2(2)_min_total_cost", stage2_solution)

def compare_q2_schemes(min_trucks_bundle: Dict[str, object], min_cost_bundle:
Dict[str, object]) -> Dict[str, object]:
    return {
        "min_total_trucks_vehicle_count": int(min_trucks_bundle["truck_count"]),
        "min_total_trucks_total_cost": float(min_trucks_bundle["total_cost"]),
        "min_total_trucks_type_counts":
dict(min_trucks_bundle["truck_type_counts"]),
        "min_total_cost_vehicle_count": int(min_cost_bundle["truck_count"]),
        "min_total_cost_total_cost": float(min_cost_bundle["total_cost"]),
        "min_total_cost_type_counts": dict(min_cost_bundle["truck_type_counts"]),
        "cost_gap_min_trucks_minus_min_cost": float(min_trucks_bundle["total_cost"]
- min_cost_bundle["total_cost"]),
        "vehicle_gap_min_cost_minus_min_trucks": int(min_cost_bundle["truck_count"]
- min_trucks_bundle["truck_count"]),
    }

def solve_question2(fragile_fixed: bool = False) -> Dict[str, object]:

```

```

trucks, catalog = build_problem_data()
counts = counts_from_catalog(catalog)

min_trucks_bundle = solve_min_total_trucks_q2(counts, catalog, trucks,
fragile_fixed=fragile_fixed)
min_cost_bundle = solve_min_total_cost_q2(
    counts,
    catalog,
    trucks,
    fragile_fixed=fragile_fixed,
    upper_bound_bundle=min_trucks_bundle,
)

return {
    "catalog": catalog,
    "min_total_trucks": min_trucks_bundle,
    "min_total_cost": min_cost_bundle,
    "comparison": compare_q2_schemes(min_trucks_bundle, min_cost_bundle),
}

def print_scheme_summary_q2(title: str, bundle: Dict[str, object]) -> None:
    packers = bundle["packers"]
    print("=" * 78)
    print(title)
    print("=" * 78)
    print(f"总车辆数: {bundle['truck_count']}")
    print(f"总运输成本: {bundle['total_cost']:.0f} 元")
    print("车型分布: " + ", ".join(f"{k}={v}" for k, v in
sorted(bundle["truck_type_counts"].items())))
    print(f"最差 min(满容,满载): {bundle['min_min利用率']:.4f}")
    print(f"平均 min(满容,满载): {bundle['avg_min利用率']:.4f}")
    print()
    for i, p in enumerate(packers, start=1):
        util = p.utilization()
        print(f"车辆 {i} | {p.truck.name} | 件数 {len(p.placed)} | 成本
{p.truck.trip_cost:.0f}")
        print(f"  空间利用率(eff): {util['空间利用率_eff']:.4f}")
        print(f"  载重利用率      : {util['载重利用率']:.4f}")
        print(f"  已装体积      : {util['已装货物体积_m3']:.3f} m^3")
        print(f"  已装重量      : {util['已装货物重量_kg']:.1f} kg")
        print("  货类计数      : " + ", ".join(f"{code}={p.loaded_counts[code]}" for
code in ITEM_ORDER))
        print()

if __name__ == "__main__":
    result = solve_question2(fragile_fixed=False)
    print_scheme_summary_q2("Q2(1) 两车型下总车辆数最少", result["min_total_trucks"])
    print_scheme_summary_q2("Q2(2) 两车型下总运输成本最低", result["min_total_cost"])
    print("=" * 78)
    print("方案对比")
    print("=" * 78)
    for k, v in result["comparison"].items():
        print(f"{k}: {v}")

```

附录五 问题二_输出脚本.py

介绍：用于输出问题二结果，根据求解算法生成各运输方案的装载明细、汇总指标及三维装载示意图。

```
from __future__ import annotations

import csv
import json
import os
from typing import Dict, List

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

import 问题二_求解器 as solver

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
OUT_DIR = os.path.join(BASE_DIR, '问题二最佳方案摆放图及空间载重利用率数据')
PNG_DIR = os.path.join(OUT_DIR, '货车图片')
os.makedirs(OUT_DIR, exist_ok=True)
os.makedirs(PNG_DIR, exist_ok=True)

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'Noto Sans CJK SC',
'Arial Unicode MS', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

TRUCK_NAME_ASCII = {
    '车型 1': '车型 1',
    '车型 2': '车型 2',
}

BOX_COLORS = {
    'G1': '#4C78A8',
    'G2': '#F58518',
    'G3': '#E45756',
    'G4': '#54A24B',
```

```

        'G5': '#B279A2',
    }

PAPER_DPI = 220
FIGSIZE = (13, 9)

def write_csv(path: str, rows: List[Dict[str, object]]) -> None:
    if not rows:
        raise ValueError(f'没有可写入的数据: {path}')
    fieldnames = list(rows[0].keys())
    with open(path, 'w', newline='', encoding='utf-8-sig') as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)

# =====
# 3D PNG 渲染模块 (仅本地 matplotlib)
# =====

def cuboid_faces(x: int, y: int, z: int, dx: int, dy: int, dz: int):
    p000 = (x, y, z)
    p100 = (x + dx, y, z)
    p110 = (x + dx, y + dy, z)
    p010 = (x, y + dy, z)
    p001 = (x, y, z + dz)
    p101 = (x + dx, y, z + dz)
    p111 = (x + dx, y + dy, z + dz)
    p011 = (x, y + dy, z + dz)
    return [
        [p000, p100, p110, p010],
        [p001, p101, p111, p011],
        [p000, p100, p101, p001],
        [p010, p110, p111, p011],
        [p000, p010, p011, p001],
        [p100, p110, p111, p101],
    ]

```

```
def draw_truck_wireframe(ax, L: int, W: int, H: int) -> None:
```

```
    corners = [  
        (0, 0, 0), (L, 0, 0), (L, W, 0), (0, W, 0),  
        (0, 0, H), (L, 0, H), (L, W, H), (0, W, H),  
    ]
```

```
    edges = [  
        (0, 1), (1, 2), (2, 3), (3, 0),  
        (4, 5), (5, 6), (6, 7), (7, 4),  
        (0, 4), (1, 5), (2, 6), (3, 7),  
    ]
```

```
    for i, j in edges:  
        xs = [corners[i][0], corners[j][0]]  
        ys = [corners[i][1], corners[j][1]]  
        zs = [corners[i][2], corners[j][2]]  
        ax.plot(xs, ys, zs, color='black', linewidth=0.9)
```

```
def style_3d_axes(ax) -> None:
```

```
    ax.grid(False)  
    for axis in (ax.xaxis, ax.yaxis, ax.zaxis):  
        try:  
            axis.pane.set_facecolor((1.0, 1.0, 1.0, 0.0))  
            axis.pane.set_edgecolor((1.0, 1.0, 1.0, 0.0))  
        except Exception:  
            pass
```

```
    try:  
        ax.xaxis._axinfo['grid']['linewidth'] = 0.0  
        ax.yaxis._axinfo['grid']['linewidth'] = 0.0  
        ax.zaxis._axinfo['grid']['linewidth'] = 0.0
```

```
    except Exception:  
        pass
```

```
def render_truck_png(packer: solver.SurfacePacker, out_png: str, title: str) ->  
Dict[str, object]:
```

```
    fig = plt.figure(figsize=FIGSIZE, dpi=PAPER_DPI)
```

```

ax = fig.add_subplot(111, projection='3d')

for p in sorted(packer.placed, key=lambda box: (box.z, box.x, box.y, box.h)):
    faces = cuboid_faces(p.x, p.y, p.z, p.l, p.w, p.h)
    poly = Poly3DCollection(
        faces,
        facecolors=BOX_COLORS.get(p.type_code, '#999999'),
        edgecolors='k',
        linewidths=0.18,
        alpha=0.68,
    )
    ax.add_collection3d(poly)

L, W, H = packer.truck.L, packer.truck.W, packer.truck.effective_H
draw_truck_wireframe(ax, L, W, H)
style_3d_axes(ax)

ax.set_xlim(0, L)
ax.set_ylim(0, W)
ax.set_zlim(0, H)
ax.set_box_aspect((L, W, H))
ax.set_xlabel('x 轴: 从右后到前方（厘米）', labelpad=8)
ax.set_ylabel('y 轴: 从右侧到左侧（厘米）', labelpad=8)
ax.set_zlabel('z 轴: 从下到上（厘米）', labelpad=8)
ax.view_init(elev=23, azim=-58)

util = packer.utilization()
truck_label = TRUCK_NAME_ASCII.get(packer.truck.name, packer.truck.name)
counts_text = ' '.join(f'{code}={packer.loaded_counts[code]}' for code in
solver.ITEM_ORDER)
info_text = (
    f"车型: {truck_label}\n"
    f"装载件数: {len(packer.placed)}\n"
    f"有效空间利用率: {util['空间利用率_eff']:.4f}\n"
    f"载重利用率: {util['载重利用率']:.4f}\n"
    f"单车运输成本: {packer.truck.trip_cost:.0f} 元\n"
    f"已装体积: {util['已装货物体积_m3']:.3f} 立方米\n"
    f"已装重量: {util['已装货物重量_kg']:.1f} 千克\n"

```

```

        f"各类货物数量: {counts_text}"
    )

    legend_handles = [
        Patch(facecolor=BOX_COLORS[code], edgecolor='k', label=code, alpha=0.68)
        for code in solver.ITEM_ORDER
        if packer.loaded_counts.get(code, 0) > 0
    ]
    if legend_handles:
        fig.legend(handles=legend_handles, loc='upper right',
bbox_to_anchor=(0.985, 0.88), frameon=True)

    fig.text(
        0.81,
        0.43,
        info_text,
        ha='left',
        va='top',
        fontsize=10.5,
        bbox=dict(boxstyle='round,pad=0.45', facecolor='white',
edgecolor='#666666', alpha=0.95),
    )

    fig.suptitle(title, y=0.97, fontsize=14)
    plt.tight_layout(rect=(0.0, 0.0, 0.79, 0.95))
    fig.savefig(out_png, bbox_inches='tight')
    plt.close(fig)

    return {
        '车型': packer.truck.name,
        '单车运输成本': packer.truck.trip_cost,
        '装载件数': len(packer.placed),
        '空间利用率_eff': util['空间利用率_eff'],
        '载重利用率': util['载重利用率'],
        '已装货物体积_m3': util['已装货物体积_m3'],
        '已装货物重量_kg': util['已装货物重量_kg'],
    }

```

```

def export_pngs_for_scheme(packers: List[solver.SurfacePacker], scheme_slug: str,
scheme_title: str) -> List[Dict[str, object]]:
    scheme_dir = os.path.join(PNG_DIR, scheme_slug)
    os.makedirs(scheme_dir, exist_ok=True)

    manifest_rows: List[Dict[str, object]] = []
    for i, packer in enumerate(packers, start=1):
        filename = f'{scheme_slug}_第{i:02d}辆车.png'
        abs_path = os.path.join(scheme_dir, filename)
        title = f'{scheme_title} - 第{i}辆车'
        stats = render_truck_png(packer, abs_path, title)
        manifest_rows.append({
            '方案标识': scheme_slug,
            '方案名称': scheme_title,
            '车辆序号': i,
            '图片文件名': filename,
            '图片相对路径': os.path.relpath(abs_path, OUT_DIR),
            **stats,
        })
    return manifest_rows

def bundle_to_overview_row(bundle: Dict[str, object]) -> Dict[str, object]:
    packers = bundle['packers']
    type_counts = bundle['truck_type_counts']
    scheme_name_map = {
        'Q2(1)_min_total_trucks': '问题 2（1）总车辆数最少方案',
        'Q2(1)_min_total_trucks_fallback': '问题 2（1）总车辆数最少方案',
        'Q2(2)_min_total_cost': '问题 2（2）总运输成本最低方案',
        'Q2(2)_min_total_cost_fallback': '问题 2（2）总运输成本最低方案',
        'Q2(2)_min_total_cost_upper_bound': '问题 2（2）总运输成本最低方案',
        'Q2(2)_min_total_cost_stage1': '问题 2（2）总运输成本最低方案',
    }
    return {
        '方 案 名 称': scheme_name_map.get(bundle['scheme_name'],
bundle['scheme_name']),
        '总车辆数': bundle['truck_count'],

```

```

        '车型 1 数量': type_counts.get('车型 1', 0),
        '车型 2 数量': type_counts.get('车型 2', 0),
        '总运输成本': bundle['total_cost'],
        '最小空间利用率_eff': bundle['min_空间利用率_eff'],
        '平均空间利用率_eff': bundle['avg_空间利用率_eff'],
        '最小载重利用率': bundle['min_载重利用率'],
        '平均载重利用率': bundle['avg_载重利用率'],
        '最小最小利用率': bundle['min_min 利用率'],
        '平均最小利用率': bundle['avg_min 利用率'],
        '总装载体积_m3': sum(p.total_volume for p in packers) / 1e6,
        '总装载重量_kg': sum(p.total_weight for p in packers),
    }

def vehicle_summary_rows(bundle: Dict[str, object]) -> List[Dict[str, object]]:
    rows = []
    scheme_name_map = {
        'Q2(1)_min_total_trucks': '问题 2（1）总车辆数最少方案',
        'Q2(1)_min_total_trucks_fallback': '问题 2（1）总车辆数最少方案',
        'Q2(2)_min_total_cost': '问题 2（2）总运输成本最低方案',
        'Q2(2)_min_total_cost_fallback': '问题 2（2）总运输成本最低方案',
        'Q2(2)_min_total_cost_upper_bound': '问题 2（2）总运输成本最低方案',
        'Q2(2)_min_total_cost_stage1': '问题 2（2）总运输成本最低方案',
    }
    for idx, packer in enumerate(bundle['packers'], start=1):
        util = packer.utilization()
        row = {
            '方 案 名 称': scheme_name_map.get(bundle['scheme_name'],
bundle['scheme_name']),
            '车辆序号': idx,
            '车型': packer.truck.name,
            '单车运输成本': packer.truck.trip_cost,
            '装载件数': len(packer.placed),
            '空间利用率_raw': util['空间利用率_raw'],
            '空间利用率_eff': util['空间利用率_eff'],
            '载重利用率': util['载重利用率'],
            '已装货物体积_m3': util['已装货物体积_m3'],
            '已装货物重量_kg': util['已装货物重量_kg'],

```

```

    }
    for code in solver.ITEM_ORDER:
        row[f'{code}_数量'] = packer.loaded_counts[code]
    rows.append(row)
return rows

def result_json_view(result: Dict[str, object]) -> Dict[str, object]:
    scheme_name_map = {
        'Q2(1)_min_total_trucks': '问题 2（1）总车辆数最少方案',
        'Q2(1)_min_total_trucks_fallback': '问题 2（1）总车辆数最少方案',
        'Q2(2)_min_total_cost': '问题 2（2）总运输成本最低方案',
        'Q2(2)_min_total_cost_fallback': '问题 2（2）总运输成本最低方案',
        'Q2(2)_min_total_cost_upper_bound': '问题 2（2）总运输成本最低方案',
        'Q2(2)_min_total_cost_stage1': '问题 2（2）总运输成本最低方案',
    }
    comparison = result['comparison']
    out: Dict[str, object] = {
        '方案对比': {
            '总车辆数最少方案_车辆数':
comparison['min_total_trucks_vehicle_count'],
            '总车辆数最少方案_总成本': comparison['min_total_trucks_total_cost'],
            '总车辆数最少方案_车型数量':
comparison['min_total_trucks_type_counts'],
            '总成本最低方案_车辆数': comparison['min_total_cost_vehicle_count'],
            '总成本最低方案_总成本': comparison['min_total_cost_total_cost'],
            '总成本最低方案_车型数量': comparison['min_total_cost_type_counts'],
            '两方案成本差额_总车辆数最少减总成本最低':
comparison['cost_gap_min_trucks_minus_min_cost'],
            '两方案车辆数差额_总成本最低减总车辆数最少':
comparison['vehicle_gap_min_cost_minus_min_trucks'],
        }
    }
    key_name_map = {
        'min_total_trucks': '总车辆数最少方案',
        'min_total_cost': '总运输成本最低方案',
    }
    for key in ('min_total_trucks', 'min_total_cost'):

```

```

        bundle = result[key]
        out[key_name_map[key]] = {
            ' 方 案 名 称 ': scheme_name_map.get(bundle['scheme_name'],
bundle['scheme_name']),
            '车辆数': bundle['truck_count'],
            '总运输成本': bundle['total_cost'],
            '车型数量': bundle['truck_type_counts'],
            '最小空间利用率_eff': bundle['min_空间利用率_eff'],
            '平均空间利用率_eff': bundle['avg_空间利用率_eff'],
            '最小载重利用率': bundle['min_载重利用率'],
            '平均载重利用率': bundle['avg_载重利用率'],
            '最小最小利用率': bundle['min_min 利用率'],
            '平均最小利用率': bundle['avg_min 利用率'],
            '车辆明细': [
                {
                    '车型': p.truck.name,
                    '单车运输成本': p.truck.trip_cost,
                    '利用率': p.utilization(),
                    '装载数量': p.loaded_counts,
                    '装载件数': len(p.placed),
                }
                for p in bundle['packers']
            ],
        }
    return out

if __name__ == '__main__':
    result = solver.solve_question2(fragile_fixed=False)
    catalog = result['catalog']

    scheme_min_trucks = result['min_total_trucks']
    scheme_min_cost = result['min_total_cost']

    # 明细坐标方案导出
    write_csv(
        os.path.join(OUT_DIR, '问题 2_总车辆数最少方案_装载明细.csv'),
        solver.solution_rows_from_packers(scheme_min_trucks['packers'],

```

```

catalog),
    )
    write_csv(
        os.path.join(OUT_DIR, '问题 2_总运输成本最低方案_装载明细.csv'),
        solver.solution_rows_from_packers(scheme_min_cost['packers'], catalog),
    )

# 方案概览与逐车汇总
overview_rows = [
    bundle_to_overview_row(scheme_min_trucks),
    bundle_to_overview_row(scheme_min_cost),
]
write_csv(os.path.join(OUT_DIR, '问题 2_方案概览.csv'), overview_rows)

vehicle_rows = []
vehicle_rows.extend(vehicle_summary_rows(scheme_min_trucks))
vehicle_rows.extend(vehicle_summary_rows(scheme_min_cost))
write_csv(os.path.join(OUT_DIR, '问题 2_逐车汇总.csv'), vehicle_rows)

# JSON 汇总
with open(os.path.join(OUT_DIR, '问题 2_结果汇总.json'), 'w', encoding='utf-8')
as f:
    json.dump(result_json_view(result), f, ensure_ascii=False, indent=2)

# PNG 导出
png_manifest: List[Dict[str, object]] = []
png_manifest.extend(export_pngs_for_scheme(scheme_min_trucks['packers'], '总
车辆数最少方案', '问题 2（1）总车辆数最少方案'))
png_manifest.extend(export_pngs_for_scheme(scheme_min_cost['packers'], '总运
输成本最低方案', '问题 2（2）总运输成本最低方案'))
write_csv(os.path.join(OUT_DIR, '问题 2_图片清单.csv'), png_manifest)

print('已生成问题 2 输出文件: ')
for name in [
    '问题 2_总车辆数最少方案_装载明细.csv',
    '问题 2_总运输成本最低方案_装载明细.csv',
    '问题 2_方案概览.csv',
    '问题 2_逐车汇总.csv',

```

```
        '问题 2_结果汇总.json',  
        '问题 2_图片清单.csv',  
    ]:  
        print(f'- {os.path.join(OUT_DIR, name)}')  
print(f'- 货车图片目录: {PNG_DIR}')
```